

VETTORE

144 pages of Array Programming

- Pittenger on APL and J 67
- Kondrashev on Internet Programming 74
- Smith on HTML Forms 91
- Burke on J4 and OOPS 103
- Skomorokhov on OpenGL & Dyalog 128



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.14 No.4 April 1998

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1998-99

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

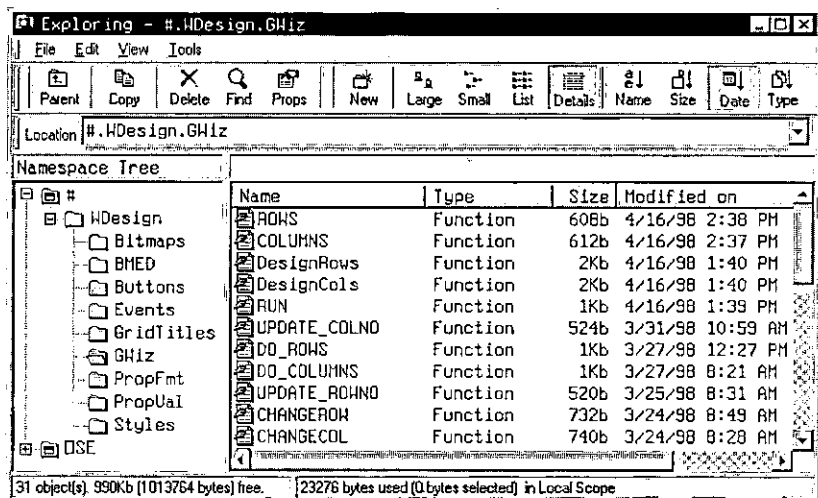
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385 Email: apl385@compuserve.com

Contents

		Page
Editorial	Stefano Lanzavecchia	3
APL NEWS		
Quick Reference Diary		7
Correspondence		
Thomson, Clough, Henriod, Karman, Last, Selfridge		9
News from Sustaining Members		17
APL Product Guide	Gill Smith	19
Review of RainPro	Bob Byers	33
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	38
J-ottings 16	Norman Thomson	48
Volutes	Howard Peelle	53
GENERAL ARTICLES		
APL and J:		
Working Together, yet Separately	Donald Pittenger	67
Internet Programming with the		
Internet Server Auxiliary Processor	Andrei Kondrashev	74
HTML Basics for APLers (IV):		
Forms and Servers	Adrian Smith	91
J4 and OOP	Chris Burke	103
TECHNICAL SECTION		
Hackers' Corner: Six Megabytes in 640K?	Anthony Camacho	112
Technical Correspondence		
Cannon, Clough, Henriod		117
At Play with J:		
A Newer Random Link Generator	Gene McDonnell	122
Using OpenGL Graphics in Dyalog APL	Sasha Skomorokhov	128
Index to Advertisers		143

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- ➔ Organise your workspace into **self-contained** sub-systems
- ➔ **Encapsulate** functions and variables *within* GUI objects
- ➔ **Isolate** utilities from your application code
- ➔ Construct **COM/DCOM** object hierarchies
- ➔ Support **multiple instances** in APL web server applications

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



programmatore che entra nel ventunesimo secolo deve essere in grado di padroneggiare tecniche che spesso sono incompatibili con gli interpreti array-oriented dei nostri giorni. Non credo esista il linguaggio perfetto: credo invece, che, dato un problema, esista il linguaggio ideale per la sua soluzione. Maggiore è il numero di linguaggi che si conosce e minore è la fatica che si dovrà compiere per consegnare un prodotto funzionante e facilmente mantenibile. Spesso, il tempo speso nell'aggiornamento delle proprie conoscenze, è più che ripagato dalla flessibilità guadagnata.

In fondo un bambino che sappia usare solo la paletta (avrei forse dovuto dire, seguendo Ian Sharp, "un temperino a lama singola") non è un gran compagno di giochi...

Let's play with the world

Not just an invitation. More than a hope. With the same spirit of kids wondering about everything they see. Going back to the joy of a knowledge without reason, to the humility of a sailor who challenges the sea and respects it because he knows that it is bigger than he is. So that one can recognise himself in what he has built, in the words and in the smiling look of others. In harmony with the music of the atoms and of the celestial spheres.

Stefano M. L. Lanzavecchia, December 1994

*Like the ghost inside the atom
That spins it round and round
There's magic in some words
Something you can't explain
That conjures up that feeling
Of the sun inside the rain*

J.C.

Lost in the research of the perfect language (Assembler, J or Java?), of the ideal algorithm, of the elegance, too often we forget that behind the seriousness of our job there is still the same game we began when we were children. I was slightly more than a child when I played with my first APL interpreter. A bit later I browsed my first issue of Vector: the happy combination of intelligent humour and competence impressed me. Then, in the dreams of a child, the legend of APL took the shape of a world inhabited by smiling and powerful people, like the wizards of the fairy tales. Only two years ago I shook hands with one of these

Editorial

by Stefano Lanzavecchia

Giochiamo col mondo

Non solo un invito. Più di una speranza. Con lo stesso spirito di bambini che guardano con meraviglia tutto quello che capita loro sotto gli occhi. Ritornando alla gioia di voler sapere senza ragione, all'umiltà del marinaio che sfida il mare e lo rispetta perché sa che è più grande di lui. Per potersi riconoscere in quanto si è costruito, nelle parole e nello sguardo sorridente degli altri. In armonia con la musica degli atomi e delle sfere celesti.

Stefano M. L. Lanzavecchia, Dicembre 1994

*Like the ghost inside the atom
That spins it round and round
There's magic in some words
Something you can't explain
That conjures up that feeling
Of the sun inside the rain*

J.C.

Persi nella ricerca del linguaggio perfetto (Assembler, J o Java?), dell'algoritmo ideale, dell'eleganza stilistica, troppo spesso finiamo col dimenticare che dietro alla serietà del lavoro c'è ancora e sempre lo stesso gioco che abbiamo cominciato da bambini. Ero per l'appunto poco più di un bambino quando mi è capitato tra le mani il mio primo interprete APL. Non per caso poco tempo dopo ho sfogliato il mio primo numero di Vector: la felice combinazione di umorismo intelligente e competenza mi avevano impressionato. Allora, nei sogni ad occhi aperti di un bambino, la leggenda dell'APL aveva preso la forma di un mondo abitato da personaggi sorridenti e potentissimi, come i maghi delle favole. Soltanto due anni fa ho stretto per la prima volta la mano ad uno di questi nomi senza sembianze, scambiato un sorriso e finalmente associato un volto alla firma sotto il titolo di un articolo. E con mio grande piacere ho scoperto che la persona non tradiva i ricordi incantati del bambino. Numerosi incontri sono seguiti al primo ma purtroppo non sempre ho ritrovato quella stessa giocosa spensieratezza.

E, dopo tutto, è l'APL il linguaggio perfetto? Io credo di no. Nonostante i suoi numerosi pregi che non ripeterò in questa sede, e nonostante il fatto che la classe di problemi risolvibili velocemente, efficacemente ed elegantemente in APL sia straordinariamente vasta, altrettanto vasta è quella delle applicazioni in cui l'uso di APL o derivati sarebbe un errore. È sempre più evidente che un

anonymous names, exchanged a smile and finally associated a face to the signature under the title of an article. To my pleasure I've discovered that the person did not betray the enchanted souvenirs of the kid. Many more encounters followed the first one but unfortunately I haven't always found that same joie de vivre.

And, after all, is APL the perfect language? I don't think so. Despite its many advantages which I won't repeat here, and despite the fact that the class of problems that can be solved quickly, efficiently and elegantly in APL is very large, equally large is the class of the applications where the use of APL and friends would be a mistake. It is clearer and clearer that a developer who enters the 21st century ought to master techniques which are very often incompatible with the array-oriented interpreters available nowadays. I don't believe in the existence of the perfect language: I believe instead that, given a problem, there exists the ideal language for its solution. The more languages one knows, the less effort one will have to make in order to deliver a product which works and is easily maintainable. Often, the time spent in the update of one's knowledge is more than repaid by the flexibility gained.

In the end, a kid who can only play with a spade (maybe I should have said, quoting Ian Sharp, "a single-bladed knife") is not a great playmate...



Lingo Allegro USA, Inc.

+1 800 LINGO-A1-1

(+1 800 546-4621-1)

visit our web site at

<http://www.lingo.com>

Quick Reference Diary 1998-1999

Date	Venue	Event
July 27-31	Rome	APL98
August 10-14 1999	Scranton, Penn.	APL99

Dates for Future Issues of VECTOR

	Vol.15 No.1	Vol.15 No.2	Vol.15 No.3
Copy date	3rd July	4th Sept	4th Dec
Ad booking	10th July	11th Sept	11th Dec
Ad Copy	19th July	18th Sept	18th Dec
Distribution	August 98	October 98	January 99
Edited by	Stefano Lanzavecchia	Alan Sykes & Alan Mayer	TBA

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ**

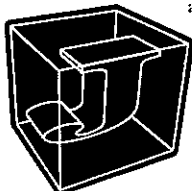
Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.



IT'S SURPRISING
HOW JSOFTWARE
MAKES YOUR DATA
JUMP OUT!

J for Windows 95/NT from Iverson Software Inc.

provides everything you need to solve problems quickly. You can build complete stand alone Windows applications, or easily integrate J with other programs. For example, use J as an ActiveX control for a web page, an OLE server for Excel,



J in a Box

or a 32-bit DLL called from a C program. J is also available on a variety of other platforms including Windows CE, Mac, Linux, Solaris and AIX. The language is identical in all versions, and programs not making use of platform-specific features will work unchanged on all systems.

Strand Software 19235 Covington Court Shorewood, MN 55331

Tel: (612) 470-7345 Fax: (612) 470-9202

www.jsoftware.com

CORRESPONDENCE

A False Simplification of atop?

From: Norman Thomson

22 March 1998

The final sentence of J-ottings 16 (see page 48 in this issue), viz. "... a thorough understanding of with, atop and hook is a major step to confidence in handling J" was written *before* I read Nick Cox's letter in Vector 14.2, and so it is clear that we share at least a broad point of view. However, I want to point out the false simplifications inherent in the transliteration of J symbols into everyday words.

Mr. Cox suggests thinking of *atop* as "applied to" or "after". To me the ordinary processes of taking logs in order to multiply can be articulated as "addition *after* taking logs" or "addition *applied to* logs", yet this is precisely the situation in which one must not use atop. (see J-ottings 16 for details). Also Mr. Cox talks about a *monadic* hook, yet the whole point about any composition is that a new verb is defined *independently of the data*. A hook can be *applied* monadically, but in itself it is neither monadic nor dyadic. The rationale for a notation such as J, or for that matter APL, has always seemed to me to be to eliminate the very sort of ambiguities and uncertainties which Mr. Cox appears to be recommending!

Calculating Medians for Nested Arrays

From: Norman Thomson

5 April 1998

Alan Sykes and Tom Stroud in "APL and Nested Arrays — A Dream for Statistical Computation" (Vector 14 No 2, pp 58-70) provide excellent demonstrations of the versatility of applying APL in the context of imputation. I would like to suggest that matters are even simpler than they suggest.

For example in the final section "Calculating Medians", although *median* is not a scalar function (which is what I meant by "not dimensionally extendible") this does not preclude its extension using *each*:

```
R←MEDIAN X
[1] R←ΔMEDIAN"⊆[1]X←1/X
```

```

R←ΔMEDIAN X
[1] R←0.5×+/X[0.5×0 1+ρX+X[ΔX]]

MEDIAN D←1,[1.5]8 9 11 13 16 18 6 4 20 25
1 12

```

Then, given the data structures which Alan and Tom use, in which *Y* is defined as shown below, their imputed means *IMEDIAN* are given by

```

Y←(,10)(10 14)(,14)(16 14)(,16)(,15)(,20)(,4)(,18)(,22)

R←NVTOMAT V
[1] R←( [/ ,="ρ"V)ρV

MEDIAN"NVTOMAT"←[1]D,Y
1 12 15.5 14.5

```

One minor irritation in current APL life is the minor divergences between Dyalog and IBM APL, the latter of which requires the third character of the definition of *NVTOMAT* to be ρ as opposed to $\uparrow$ in Dyalog.

Sorites — J vs Prolog

From: Eddie Clough

9 April 1998

I was delighted to see Norman Thomson's J-ottings 15 appear in *Vector* Vol. 14, No. 3. You may be interested to know how I came to set Norman off on this particular trail. I wanted to see if I could incorporate within a computer program the transformation rules used when analysing any piece of symbolic logic (e.g. "not not A" is equivalent to "A"; "B implies A" is equivalent to "A or not B"; hence "[not B] implies A" is equivalent to "A or B", where "or" is the inclusive or) but I also wished to determine how well J stood up to another language on its own ground.

By its very name, Prolog seemed the natural language choice and I did produce a long, convoluted Prolog program of about 140 lines using the usual recursive Prolog techniques — but I found that I could reproduce the same results with just a few lines of J script.

Sorites are normally solved by using rules to eliminate propositions which appear in two of the given premises (e.g. if "A implies B" and "B implies C", then "A implies C"). Norman quickly extended my J results to produce all the

valid implications from the sorites, not just the one left by this elimination process.

The outcome of our correspondence was J-ottings 15.

I hope that others have found this an entrancing by-road of J.

E.A.Clough
'Conifers', Leighton Beck Road
Slack Head, Beetham
Milnthorpe
Cumbria LA7 7AZ

On Niven Numbers

From: Claude Henriod (*trans. Sylvia Camacho*)

31 March 1998

On first reading *Vector* Vol. 14.2 I was attracted by one particular article, for three reasons: its Title, the first APL function and the Tables of timing comparisons.

- The Title — I did not know about Niven numbers.
- The function *NIVEN1* of 4 lines with a missing closing parenthesis. Is it a copying error? David would say this is 20% error in 5 parentheses.
- Comparison tables taking up more space than the functions!

In my opinion, differences of 2-3% are not significant and do not justify a choice of one coding over another. There are plenty of parameters which affect such cases and the ratios are not proportional.

- the microprocessor (Motorola, Intel, Pentium ...) + their MHz
- the mathematical co-processor, if there is one
- the operating system (mono or multi-tasking or TSO)
- the interpreter (Dyalog APL, APL 2000, IBM APL2 family ...)
- the available memory (maybe swapping)
- the memory management system (*WS FULL*)

etc., can be deemed to have an effect of +/- 5% on the performance of the code. On the other hand, a yield of a 25th begins to be interesting.

The author of the article , Joseph De Kerf, asks for:

“an algorithm which drastically improves this performance”

- I have not, unfortunately, found such an algorithm.
- I have, fortunately, at my disposal a programming language which allows even me to improve the performance but to achieve this I have had to develop variations and a new approach to *NI VEN*.

How? You will find the appropriate instructions and examples in the *Technical Correspondence* of this issue. The code is available on the Vector Web site.

Before placing the final full stop in my letter, I received Vector 14.3 with David Piper's propositions. I agree with him completely on some points but certainly not with all his assertions.

Use of single character names!

Functions and variables must be understandable, but ...

- APL is of its nature concise (no key words in the standard language); each primitive is limited to an ideogram
- In some applications the number of local variables may be important (10 letter variables take up space!)*
- I am persuaded that in the UK, as in Switzerland, the first lesson in formal algebra is:
 $A + B = C$ but certainly not:
 $\text{Asparagus} + \text{Banana} = \text{Chocolate}$

Multiple assignments on a single line. Why not?

- It is necessary to pay attention to large volumes of data but also to think of each line as independent, allowing the operating system to take control between instructions. (Elapsed time?)
- It is also necessary to think of the Vector Production team. (Economising on paper?)

Complex path analysis — Fighting dirty!

There is not 75% branching but 3 conditional GOTOs on 4 lines. In other words 3 simple conditions among 20 primitives.

While *NIVEND* also has 3 GOTOs spread over 11 lines, one to a label which does not exist! (This is an absurd example: full of names hiding the logic of the whole.) On the other hand, David does not mention which interpreter is being used.

My Conclusion

The search for a good performance in APL must not be about time* at all costs but an alliance of:

"APL logic + code + simplicity + time"

And, as a complement, trying to extract a positive message, it remains only for me to propose that you look at some of the possible versions of *NIVEN* under Technical Correspondence.

* Writing and typesetting also take time.

Claude Henriod
Rte de Monteiller
CH - 1965 Savièse
Switzerland

Tel:++41(27)3951391 Fax:++41(27)3951340 eMail:chenriod@vtx.ch

Y2K and Publicising APL

From: Jan Karman

28th March 1998

In *Vector* Vol.14 No.2 October 1997, Anne D. Wilson asked what the next panic problem would be. According to R.L. Glass in his article 'The next date crisis' (ACM, January 1997) this will be the Unix machine in the year 2035; the clock will ask for the not available next bit. (By the way, I was planning and preparing an article on 'Dates' after Bob Brown's letter, but in spite of the fact that I'm quite familiar with this material the hurdles had easily been underestimated. Moreover, I agree with David Piper, that the code to be published should be more than OK, so altogether I definitely need more time.) But also the change from the native currencies in Europe to the Euro next year will give panic problem related headaches in the IT-industry.

Re the Editorial in *Vector* Vol.14 No.3, I would like to remark that it might be true that in general APL-ers are preaching to their own parish. Every mature APL-er however has an easy opportunity to write in other (general) papers or magazines. A few years ago I promised myself to respond to articles in such

papers when the subject did fit well in an APL (or J or K) context. This has resulted in some articles on APL in 'Computable', a Dutch weekly, some being a half to an entire page (about A3), most of them with a technical touch, algorithms and code or pictures.

The titles I submitted included (translated): *The future lies with Symbolic Languages* (on APL), *Use APL when it comes to Inversion* (on databases), *Programming without Calls* (on Object Orientation), *Math doesn't hurt* (on Functional Programming & Interpreters) and *Leap year – horror scene?* (on miscellaneous date problems).

Not a single one was ever refused. They usually will be refused when the tendency is too promotive, or when vendors try to sneak in. Just write informatively. If I can do it, everybody can.

For some articles I even got responses from interested novices (mostly for J) and I could direct them to the appropriate vendors. Of course, nobody knows the final results, it's just a matter of 'be good, tell it, never look behind'. Above all it should be fun.

Nevertheless, the best way to promote APL & Cousins remains in providing reliable commercial applications in a shorter time. If APL is your tool you can do it.

Abysmal Prrof Reading Standards

From: Disgusted of Evesham

27 March 1998

I am writing to complain about the abysmal standard of proof-reading which is persistently applied at VECTOR. Every time I send in my gems of code in order to rant against someone else's crazy ideas or bad coding style or to enlighten your readership with some of my own the proof-reader fails to correct even the most obvious of my errors. I shall be sending back my Blue Peter badge in protest. You won't be hearing from me again.

Exemplary Code!

From: Phil Last (Phil_Last@compuserve.com)

1st April 1998

Genuflections are due to David Piper for gracing us once again with such an exemplary piece of code. Imagine my distress when I discovered that the page

containing the definition of *again* was missing from my copy of VECTOR. I feel sure that David has come up with something really special here so I should like to attempt to reconstruct it.

To an untrained eye it might at first seem that *again* is a variable. Now we can reasonably assume that the intention of the final branch in each function is to return to the line labelled *next*:. So if *again* is a variable it must take *next*'s value. Ah, but this is 5 in the thoughtfully decommented version of *NIVEND* but 4 in *NIVENE*. Now we know that David would not stoop to changing it without making the change quite explicit, so *again* cannot be a variable. What about a niladic function? Something like:

```

▽ r←again
A generalised branch back destination function
r←next
▽

```

I can't really see David doing this as there is a hidden dependency between *again* and any function that calls it. How about?:

```

▽ r←again
A generalised branch back destination function
r←5+>1+□lc
▽

```

This is much better because it no longer requires the target label to have a fixed name. Though it's unfortunate if you need to branch back by anything other than 5 lines. This must be it!:

```

▽ r←again;z
A generalised branch back destination function
A lines in caller containing colon
r←r~': 'v.=~r+□cr>1+□si
A up to but excluding first colon in each line
r←(pz)ρ(,z+z°.z1[/z+/z)\(,z+^r~': ')/,r
A evaluate labels selecting latest before current line
r←1[/z/~(>1+□lc)>z+1'0',, ' ,r~1=□nc r
▽

```

David, I am amazed. This is a masterpiece. I congratulate you. Only three concise well documented lines of code to determine whether the calling function should branch back. I particularly admire the way you have avoided the slavish following of your own arbitrary rules regarding identifier length and multiple assignment so that the standard algorithms (the long words of APL) shine through. Also the...

What? an error? Excuse me, the fool sitting next to me is rambling about *again's* being a coding error. No, look, David tested his code, he even ran timings on it, a hundred thousand iterations ...

Phil Last
Wistaria Lodge
146 Crossbrook Street
Cheshunt, Hertfordshire
EN8 8JY

Re APL Null Vector Vol 14 No 3

From: R.G.Selfridge (selfrid@nervm.nerdc.ufl.edu)

3 April 1998

Here is your answer in words. When a scalar is combined with an array the resultant shape is nearly always the array shape. This is true for most of the arithmetic operations, e.g. $= + \times$ etc.

A reduction over an empty vector is always a 'fringe' condition and will try to return the identity of the primitive. Thus 'or' reduce returns zero, 'and' reduce returns one, times reduce returns one, plus reduce returns zero. Things like divide reduce are up against the fact that divide is not symmetric.

Now $2 \text{ 'and' } = 0$ / numeric vector first does the $=$, resulting in an empty vector, and then (second) does 'and' reduce, giving one. Using character arguments doesn't change the result. It is important to remember the order of an inner product.

Primitives like epsilon or underscore epsilon don't follow this easily, with epsilon returning a shape dependent on its left argument, epsilon underscore returning a shape dependent on its right argument. Again if you think what these primitives are meant to do both responses seem normal.

The question of an empty left argument with epsilon underscore is a result with right argument shape, and an empty array is (arguably) an element at every position of the right argument.

R.G. Selfridge
Box 116120
University of Florida
Gainesville, FL 32611 USA

News from Sustaining Members

Dyadic Systems Ltd

Dyadic continues to release a steady stream of enhancements to Dyalog APL, which are immediately available to DSS subscribers as downloads from <http://www.dyadic.com>.

APL users throughout the world have long been accustomed to the component file system (`⎕FSTIE`, `⎕FREAD`, `⎕FAPPEND` etc.) and the native file system (`⎕NTIE`, `⎕NREAD`, `⎕NAPPEND` etc.) which are supported by most modern implementations of APL. Recently, Dyadic has implemented two apparently small but significant extensions to these file systems. These extensions are *upwards compatible*, i.e. existing APL code will work unchanged.

The first extension is to provide for the automatic allocation of a *tie number*. If you specify 0 as the right argument to `⎕FCREATE`, `⎕FTIE`, `⎕FSTIE`, `⎕NCREATE` and `⎕NTIE`, Dyalog APL assigns the next available tie number and returns it as the result of the system function. This avoids the need to calculate the tie number yourself. More importantly, it is an atomic operation and a *safe* method of obtaining a tie number when your workspace is attached to a number of concurrent users as may be the case if it is acting as an OLE Automation server or a Web server.

The second extension is that all component and native file system functions that did not previously return a result, now return a *shy* result. (A *shy* result may be assigned to a variable, but if it is *not* assigned, the result is *not* displayed). This means that you can perform several file operations in a single expression; for example, to delete a component file called *TEMP* you can use the expression:

```
'TEMP'⎕FERASE 'TEMP'⎕FTIE 0
```

which is much neater than doing:

```
TIENO←1+⌈/0,⎕FNUMS
'TEMP' ⎕FTIE TIENO
'TEMP' ⎕FERASE TIENO
```

This extension permits more convenient use of file system functions in dynamic functions. For example, a dynamic function *RENAME* to rename a file could be expressed as follows:

```

RENAME+{
    □NUNTIE α □NRENAME ω □NTIE 0
}

'NEW' RENAME 'OLD' α Rename file from OLD to NEW

```

Another important change is an enhancement to the □OR, □NS and □WC system functions, which in Dyalog APL, may be used to *clone* or make an *instance* of a namespace (or GUI object); for example:

```
'Instance'□NS □OR'Class'
```

This creates a new namespace called *Instance* which contains all of the objects (functions, variables and sub-namespaces) defined in the namespace *Class*. However, the objects are not actually *copied* from *Class* into *Instance*. Instead, *Instance* merely contains *pointers* to the objects in *Class*. Only when you change an object in *Instance* will it be de-coupled from the original object in *Class* and consume additional workspace.

This enhancement makes it practical to create large numbers of instances of namespace objects (e.g. an OLE Server) and to make extensive use of *inheritance* in your applications.

Causeway Graphical Systems Ltd

We are delighted to announce that Joachim Hoffmann (of J Austria) has agreed to work with us for a year — this brings us up to the team size that we always felt was 'about right' in terms of productivity and minimal management overhead. Joachim will continue to support his Austrian customers from our Malton office, as well as bringing J expertise and a thorough knowledge of the Dyalog Gui to our team.

The *CausewayPro* documentation (Tutorial and Reference Manual) will be following this Vector to the printer. It will be posted on the web site in HTML format and will be mailed out to existing customers very soon after you read this. Additional classes (for example a simple wrapper for the editing capabilities in *RainPro*) will be available as add-ins from our web site, priced between £25 and £50 depending on complexity.

The latest version of *RainPro* is also available on the web to all customers on current maintenance (call us for the ZIP password). This supports graphical data editing (line-graphs and barcharts only for now), drill-down, and in-place update of data series, headings and labels. We are also working on OpenGL support for the 3D charts and expect to have a demo version available in Rome.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
Beautiful Systems	APL2/PC	poa	IBM's APL 2 for the PC.
	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted Interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1060	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	

	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLIWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$850	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility.
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$50	
	J Standard Edition	\$150	
	J Professional	\$325	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	

	J User Manual	\$50	
	J Phrases	\$40	
	J Primer	\$40	
	Set of the above 4 books	\$160	
	Concrete Math	\$40	
	Fractals, Visualization & J	\$50	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
	APLWIN	\$70	For 386/PC under Windows 3.1
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis Systems	Dyalog APL	poa	Dyalog Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.

RE Time Tracker Co	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Sollton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
ADAPTA Software	MPS - Master Production Scheduling FBS - Forecasting and Budgeting System DRP - Distribution Requirements Planning	pos	
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
The APL Group	Qualed	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler

(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400/\$600	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support system for top management. Handles large multi-dimensional tables, data analysis, EIS presentations; generates HTML and Latex output. Platforms: DOS, APL+II, Windows 3.1/95 for Dyalog APL, LAN support. Ideal for APL customisation, more than 100 installed.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson. Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (I/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASi	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs

	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as Sequelink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; Includes full-screen interface and stand-alone functions. Price depends on number of users.
Lescasse Consulting	APL+Win Monthly Training Program	\$500	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming ...	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for +Win	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC Interface for APL+Win
	SQAPL Pro	poa	ODBC Interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FRESCO Business Graphics	poa	Fast and easy business graphics DLL
	AP126/PC	poa	GDDM Interface for Dyalog APL/W
	AP127/PC	poa	ODBC Interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMs together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server..
Soliton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Alay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, Internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
Entropy Software Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL Interpreters and C.

IAC/Human Interfaces Consultancy		poa	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Ling Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Mackay Kinloch Ltd	Consultancy	200-400	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis Systems	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling -- mainframe to PC, performance tuning.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies Consultancy		poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL.
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.

Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.
--------------	-------------	-----	--

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-395	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS/APL	\$24 p.a.	703-528-7817, 1200-14400b, N-9-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Oasis Systems	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVS/LINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 Interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join

Roma/Italy SIG	Roma, Italy			
RusAPL	Moscow, Russia	APL Club	Seminars and Annual Meeting	100,000R (students 20,000)
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qdly SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Glimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION CONTACT

ADAPTA Software GmbH	Michael Baas
Adaptable Systems	Lois & Richard Hill
Adaytum Software	Douglas Rowley
Adfee	Bernard Smoor
Ajay Askoolum	Ajay Askoolum
Andrews	Dr Anne D Wilson
APL-385	Adrian Smith
APL Bay Area APLBUG	Curtis Jones (Sec)
APL Club Austria	Harald F. Nelson
APL Club Germany	Dieter Lattermann
The APL Group Inc	Stuart Sawabini
APL Solutions Inc	Eric Landau
Association Francophone pour la promotion d'APL	Ludmila Lemagnen
AUSCAN Software Ltd	Richard Procter
BACUS	Joseph De Kerf
Beautiful Systems, Inc.	Jim Goff
Bloomsbury Software	Peter Day
Camacho	Anthony Camacho
Ray Cannon	
Paul Chapman	
Causeway Graphical Systems Ltd	Adrian Smith
Cinerea AB	Rolf Kornemark
CODEWORK	Mauro Guazzo

ADDRESS, TELEPHONE, FAX, EMAIL etc.

Marlenhoehe 86, 25451 Quickborn, Germany Tel: +49 4106 60977 Fax: +49 4106 67869 Email: 101523.1757@compuserve.com
49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737 771643 Email: 106173.3347@compuserve.com
12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK Tel: 01539-731205
Brook House, Gilling East, York YO62 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
Rheinstraße 23, D-69190 Walldorf, Germany, Tel: +49 6227-63489 Compuserve: 100332,1461
844 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108 Email: ssawabini@aol.com, eshaw@aplgroup.com
1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cals.com
174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
Roclinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481 Fax: 0171-436 0524 Email: pd@bloomsbury-software.co.uk
11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036, email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874697 Email: 100430.740@compuserve.com
51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
The Maltings, Castlegate, MALTON, North Yorks YO17 7DP UK Tel: 01653-696760 Fax: 01653-697719 Email: causeway@compuserve.com
Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email rol@cinerea.se
Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@lnrete.it

ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375. Email: glerlov@ibm.net
Danish User Group	Per Gjerlöv	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND.
Dinosoft Oy	Pertti Kalliojärvi	Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
Entropy Software Ltd	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ UK Tel: 01442-834015 Email: 100412.1305@compuserve.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410 Email: olla.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel/fax: 0181-864 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N5C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-8381 Email: info@godin.on.ca
H.M.W.Trading Systems Ltd		Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2832@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	5 Thorsden Court, Guildford Road, Woking, Surrey, GU22 7QS UK Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770398 Email: 101740.1203@compuserve.com
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-526803. Compuserve: 100021.3073
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760038 email: acamacho@cix.compulink.co.uk Reutemet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M48/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (~2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
INFOTROY	Alexei Miroshnikov	3 S. TulenIn Lane, St. Petersburg 191188 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: alm@infotroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119C, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
J Austria	Joachim Hoffmann	c/o Causeway Graphical Systems. Email: JoHo@ping.at
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Bldg.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022 Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00381@niftyserve.or.jp
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY UK. Tel: 01992 633807 Fax: 0121 359 0375 Email: phil_last@compuserve.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Steven J Halasz	1105 Chicago Avenue, Suite 155, Oak Park, IL 60302, USA. Tel: +1 800 546 4821-1 Email: sjhalasz@interaccess.com
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voicemail: 01426 98 3858 Email: Alastair_Kinloch@compuserve.com
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375

MicroAPL Ltd.	Richard Nabavi	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8868 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE UK. Tel: 01730-263843 Email: Ellis@mrtrfm.demon.co.uk
Oasis Systems B.V.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: oasisbv@pl.net or zwart@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-8004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: Info@object-oriented.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK Tel: 0181-763 2490 Fax: 0181-763 2491
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: 100564.2544@compuserve.com
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (203) 270-9729
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, USA. Tel: not known. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
RusAPL	Boris Makeev	box 971 (for Makeev), Dmitrovskoe Sh., 2, 127434, Moscow, Russia Tel/fax: +7 95 210-7783 Email: makeev@atom.al.x-atom.net
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768668 Fax: 0181 8758660 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eeog.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykas Systems Inc, 4649 Wilens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@sollton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Orlinsk-5, Kaluga Region, Russia Tel: +7(08439)31463 Email: askom2@kaluga.ru@mail.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rheswain@acm.org
SwedAPL	Christer Ulfhjelms	Novator Consulting Group AB, Svärdivägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServe: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Wilens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: Info@torontoapl.org
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG Tel: 01273-327238 Email: centre@mlstral.co.uk
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7808

WORLD WIDE WEB SITES

ORGANISATION	URL
AFAPL	www.ensmp.fr/~scherer/flangle/ (Journal available on line)
APL2000	www.APL2000.com
APL-385	www.demon.co.uk/apl385
The APL Group Inc	www.aplgroup.com
AUSCAN	www.interlog.com/~rjp/auscan
Capital PC User Group	http://cpcug.org
Causeway	www.causeway.co.uk
CODEWORK	www.inrete.it/codwk/eng/home.html
COSY (Bob Armstrong)	www.cosy.com
Dinosoft Oy	http://yritys.kolumbus.fi/dinosoft
Dyadic Systems Ltd	www.dyadic.com
FinnAPL	www.pyr.fi/apl
Godin London Inc	www.godin.com
Hoekstra Systems	www.khamsin.demon.co.uk/about/hs/iframe.html
IBM APL2	www.torolab.ibm.com/ap/apl2.html
Infostroy	www.insight.dk/infostroy
Insight Systems ApS	www.insight.dk
Iverson Software Inc	www.jsoftware.com
Japan APL Association	www.naska.co.jp/JAPLA/
Kestrel Consulting	www.kestrel-consulting.com
Lescasse Consulting	www.lescasse.com
Lingo Allegro USA Inc	www.lingo.com
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch
MicroAPL Ltd	www.microapl.co.uk
RE Time Tracker Oy	www.rett.fi
Shepp & Associates	www.digitravel.com
SigAPL	www.acm.org/sigapl
Rex Swain	www.pcnet.com/~rhswain
The APL Group Inc	www.aplgroup.com
Toronto SIG	www.torontoapl.org
Jim Weigang	www.chilton.com/~jimw

FTP SITES

ORGANISATION	DOMAIN NAME
IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

RainPro Review

by Robert Byers (73043.1745@compuserve.com)

A natural extension to data analysis is to be able to graph it. Your audience will, rightly or wrongly, be more impressed with fancy graphs than just tables of information. If this is true, hopefully, you will end up showing something useful. APL is great for data analysis but most vendors do not include a native graphics sub-system with the interpreter. If you get any graphics support it is probably not as full-featured as you need. As a result, I am willing to wager that most APLers will resort to exporting data to a graphics-only software application or a popular spreadsheet to get the job done.

So what about using APL graphics packages? There are some APL tools worth consideration. The ones I know about are AGSS (A Graphical Statistical System) and Graphpak which are available for the APL2 family, APL*PLUS (DOS version) has some built in graphic capability and for APL/W there is Fresco from Lingo Allegro and RainPro which is the subject of this review. In a recent issue, *Vector* ran an article on a port of APL2's AP207 and Graphpak to Dyalog APL/W. Because I am largely unfamiliar with these other packages I am unable to do a comparative review. Alternatively, if you use APL/W or APL+Win you can integrate a non-APL solution, namely one of the many OCX/ActiveX graphic tools available.

When evaluating a product like RainPro it has to fulfil many requirements and provide performance. Everybody has different needs, some of which get overlooked during the review process. Creating a list of requirements involves thinking in very specific terms about chart types, chart properties or characteristics as well as performance. RainPro fits most of my needs.

Causeway Graphical Systems Ltd. distributes *RainPro*. It was developed by Adrian Smith, one of the principal owners of Causeway, who is very familiar to most *Vector* readers. RainPro is a commercial APL add-in workspace designed primarily for business graphs. A shareware version, simply called *Rain*, ships with each copy of Dyalog APL/W.

The history of Rain dates back to 1992 and was first available for APL*PLUS/PC. The Dyalog version first appeared in 1993 and incorporated PostScript language support. The result of PostScript was publication quality output from your Windows printer. RainPro is also available for APL+Win. RainPro made its debut in September 1996. RainPro extends Rain primarily for its 3D graphics capability,

documentation and support. The current production release is 4.2. The next release 4.3 is now in beta and Adrian sent me a copy of it. I will touch on the new features in 4.3 later in this review.

In the Dyalog version, all the code is contained in two namespaces *ch* and *PostScrp*. Namespaces are a Dyalog APL/W feature that allow you to subdivide your workspace into self-contained partitions. The benefit of namespaces really shows off with an add-in product like RainPro. Just copy the two RainPro namespaces into your workspace, and you now have all the functions required to produce high-quality graphs. Later, if you do not want to keep the code around just erase the two namespaces.

RainPro contains the functionality to satisfy most graphical needs. For example, RainPro will create any of the following types of charts: Area, Bar, Pie, Step, Line, HiLo, Gantt and a number of 3D types.

To use it effectively you must learn its syntax. To assist you, both Rain and RainPro have many examples that show the many types of graphs possible. Use these examples to learn and to get ideas to produce your own graphs. You can choose to produce graphs interactively from within the APL session or store RainPro code in defined functions. Graphs produced in RainPro can be displayed and printed using the supplied viewer function or you can integrate graphs onto an object in the Dyalog GUI. Other options include saving your variables in the workspace or in a component file. You can also export graphs to other applications in encapsulated postscript (EPS); direct PostScript; or windows metafile format.

RainPro does not come with a pre-defined GUI. You must be prepared to do some programming. If you want you can develop your own GUI front-end and use RainPro code as your callback functions as long as you acknowledge Causeway in your list of credits. Adrian warns the user about this and states: "Note that Rain is a language, and is intended for APL techies, not end-users!"

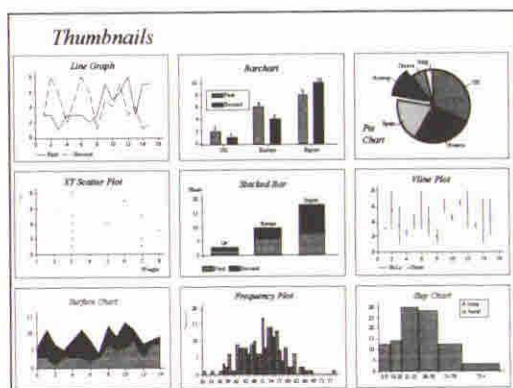
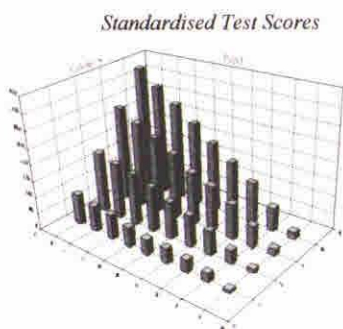
I found that learning the syntax is not difficult and can be fun as you experiment a bit with the code. Each graph is coded in the same sequence. First, you start by setting up your graph using the function *ch.Init* (i.e. Initialization), *ch.New* sets up the chart area, *ch.Set* sets properties, then plot the graph (i.e. *fns ch.Plot, ch.Bar, ch.Cloud, ch.Tower*), finally finish up with *ch.Close*. This last function produces the PostScript code which you can now view or print.

Simple graphs can be created with just a few lines of code. The more detailed or complex your graph, the more code you need to write. Setting properties and

positioning things like labels, notes, legends, titles, font size take the most time to get it right. Occasionally, an error happens. When it does, you usually get a number of edit windows opened of RainPro code and the error can be quite difficult to determine. Sometimes only trial and error can solve the problem. Usually the problem is simple like a statement is out of order. Bug fixes and enhancement releases are distributed from Causeway's website.

RainPro syntax is very similar to the syntax used in NewLeaf, an APL reporting package also from Causeway. Graphs produced in RainPro are directly supported in NewLeaf. If you use both packages together, as I do, you have a powerful reporting and graphic capability from within APL.

To show how easy it is to create a graph, examine these two examples. The "Thumbnails" graph, on the right, shows the variety of graphs. Note the number of graphs you can combine on one form. The "Standardised Test Scores" graph, on the left, and function is shown below.



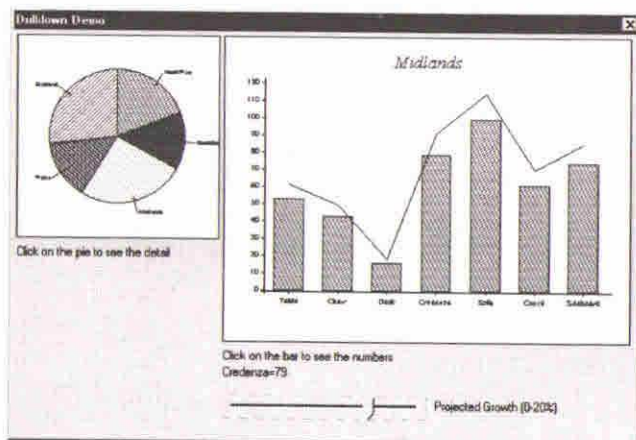
```

r←Tower
a Play with tower charts
a Gap is set to 2*tower base with GGap here (try 0 for fun)
ch.Set'Head' 'Standardised Test Scores'
ch.Set('Ycap' 'Category')('Xcap' 'Pupil')('GGap' 2)
ch.Set'Grid'
a Best to get the bigger numbers to the back!
ch.Tower0+\\+\\79 4p20
PG←ch.Close
r←'View PG'

```

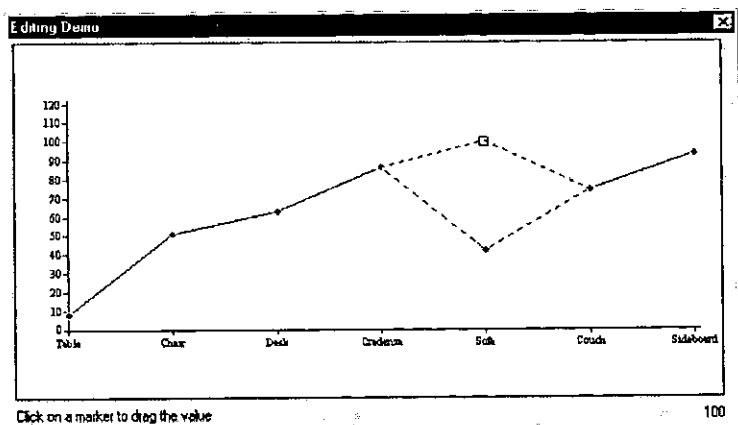
RainPro contains many features but non-APL solutions like OCX/ActiveX controls offer more features and functionality. Adrian would probably call most of this extra functionality chart junk and he is probably right, however, you have to make up your own mind. I have found OCXs sometimes frustrating if you cannot get the control to work properly. OCX documentation is written for Visual Basic users and you can forget about phone support if you tell them you are using APL. On the other hand, RainPro is written in and for APL.

An exciting new feature that will show up in the next release is support for event-driven code. This means that you can now write code and be able to interact with your graph once you view it on screen. I have had a brief look at a beta version of 4.3 and I like what I see. Two sample functions were provided for me to look at called *Drill* and *Edit*.



The *Drill* function shows two graphs drawn as static objects on a form. One was a pie chart – when you click on a section of the pie up popped another graph showing a combined bar and line graph with the underlying detail of the selected pie portion.

You can also drag the line up and down interactively on the combined chart with the slider bar.



The *Edit* function shows a simple line chart. You can again move the line up or down. Being able to interact with a graph narrows the functionality gap between RainPro and OCXs. This is a great new feature for doing what-if scenarios. Another new feature is to be able to display hints when the mouse pointer hovers over a graph object. Hopefully, in the final release, you will also be able to edit/change properties like legends, labels axis names etc.

What follows now is few thoughts for possible future releases. Although purposely not provided, I still would like to see a GUI interface on top of RainPro that can be integrated into *USE* and therefore always available to do some simple graphs in any workspace. I remember that APL/W, version 7, distributed a sample workspace that creates a 2nd level toolbar that worked with MS Chart VBX graph control. I think it would be a good demonstration and a useful tool to modify this workspace to use Rain or RainPro code to generate these simple graphs. Graphs and statistics go hand in hand. There are many APL statistical libraries that have been produced over the years. It would be a useful add-on to RainPro to integrate a statistical library. Another possibility is OpenGL, a 3D graphic standard for scientific charting, which is gaining in popularity on Windows and is perhaps something that a future RainPro release could incorporate.

In summary, RainPro provides a solid graphics package to the APL user. If you are looking for an APL-based solution, RainPro should fit the bill. But before you buy try the (Dyalog only) shareware version, Rain. It provides plenty of functionality to let you know if RainPro is for you. For more information and to purchase, contact Causeway by visiting their web site at www.causeway.co.uk.

Zark Newsletter Extracts

introduced by Jon Sandles

Introduction

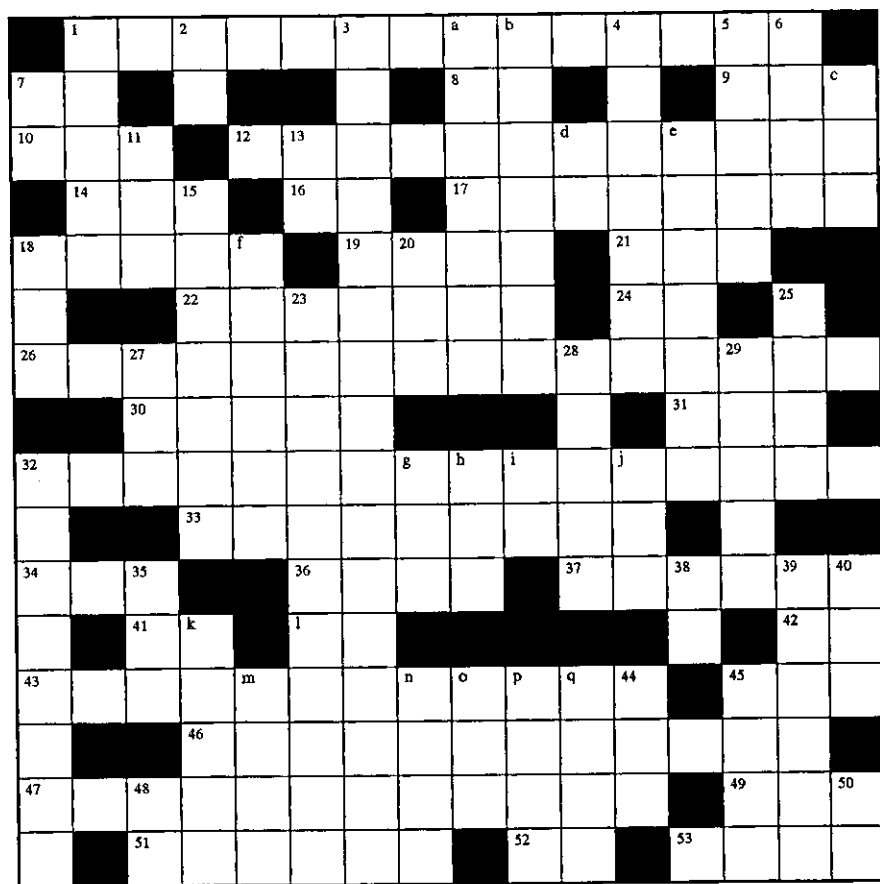
Last issue we reprinted one of the crosswords from the Zark APL Tutor news. This issue we include the solution and the first of a series of articles which I hope will be of use and interest to APLers old and new. Also reprinted is a second crossword. We encourage you all to send in solutions to both the Crosswords and the "Limbering UP" exercise. Your solutions can then be compared to those of 8 years ago.

We received a solution to last issue's crossword from Jonathan Barman who submitted it as a DyalogAPL workspace that also provided a Gui interface for designing crossword layouts. Future crosswords may be typeset using this workspace! We will also make it available on the *Vector* WWW pages so that you can solve the crosswords without soiling your beloved Vectors.

Solution to Crossword from Vector 14.3

(	R	€	ι	9	)	/	ι	ρ	R		M
ι			×		C		M	V	×	1	2
ρ	ρ	T	A	B	L	E		~	N	€	L
A		+		M	E	=	L	M	÷	N	
)	F	N	S		A	+	A		D	<	K
≡		/		B	R	/	M		+	0	
Δ	ψ	V		V		B	T	2	~		-
A	×	×	\	1	+	I		o	R	*	2
	V	2		/		T		o			↓
3	-	5	×	ι	9		L	.	5	+	N
?	1	0	0	ρ	5	2		2	5	/	V
3	0			V		5	ρ	5		M	

Crossword



Across

1. Do all the rows of the character matrix CM contain at least one vowel (AEIOU)?
7. Smallest
8. 3×4
9. The length of an empty character vector
10. The elements of M , scaled by the factor Z
12. The coefficients of $Y=A+Bt$ (where $t=1, 2, 3, \dots$) given a vector Y
14. The rank of 1
16. Convert the matrix D to a vector while looking through a mirror

17. The element of C corresponding to the largest element of N
18. The length of V , ignoring the first MT elements
19. The shape of $3 \times 1 \times 2 \times A$
21. The accumulated value of annual investments U (e.g. 100 250 50) after the last deposit, at accumulation rate $(1 + interest) \times I$ (e.g. 1.07)
22. Given two same-length vectors C and P , the elements of C corresponding to the P largest elements of N
24. $\vdash V$ in conventional maths notation
26. The corner elements of the matrix N (sound familiar?)
30. The absolute value of the result of adding N to the elements of AM
31. The first I elements of V , repeating values if not enough
32. Identify the rows of the matrix C that contain a single distinct character
33. The number of digits in each element of the non-negative integer vector N
34. Truncate the vector V to length N
36. Stick a column of 99s on the end of the matrix M
37. Convert the numeric matrix BAL into a formatted character matrix using the two-element control vector $V2$ (e.g. 10 2)
41. Largest, backwards
42. All
43. Does the character vector J contain the same number of take (+) and drop(-) symbols?
45. Those values in the vector I not found in J
46. Perform the inverse of $I \leftarrow B / \setminus N$, using $\circ$ to separate expressions
47. Perform the inverse of $M \leftarrow + \setminus N$, where N is a matrix
49. The indices of the elements of the vector V
51. The locations within the vector T at which the characters 'M' and 'J' first occur
52. The matrix M transposed
53. What programs are available?
 1. '[1]' [2 1] in origin 0

Down

1. Is the array M nonempty?
2. Any
3. The corner elements of the matrix M
4. The incestuous expression whose result is compared to $\setminus pIV1$ to return the distinct values of the vector $IV1$
5. The indices of the 1 bits in the vector U , written right-to-left
6. The first element of the matrix N , as a scalar
7. Ignore the decimals in the values of M
11. The first Z elements of the matrix T

13. The reciprocal of the scalar D , the hard way
15. Shift the elements of the vector C left one place, filling with N if N is 1, else 0
18. The number of digits in the positive integer scalar N
20. The "ascending" grade vector 6 1 5 3 7 4 2 for $N+12$ 15 14 15 13 12 15
23. Use the integer portion of the scalar M as an index into the vector A , building a vector of that length, containing N copies of M , then 0s. Pad or truncate the result to length P
25. A random number between 1 and $NV1[1]$; another between 1 and $NV1[2]$; and so on to $NV1[pNV1]$
27. The decimal portions of the elements of C
28. Round UP the values in V and reshape them to a vector with as many elements as the matrix whose shape is S
29. Row, row, row your boat
32. Are all the elements of the vector NV integers
35. Sum the elements of the vector V while standing on your head
38. Convert the numeric vector N into a character vector that looks the same
39. Is the Boolean scalar A true and the first element of the Boolean matrix N false?
40. Replicate each column of the matrix J , L times
44. ___ Brown, the chief architect of APL2
45. Set I to be the whole numbers from 1 to F
48. What an array with no elements is called (argh!)
50. ___ APL, IBM's precursor to APL2

- a. `' ',6p3pMATB[CPN;]`
- b. `'A[1]'+[-\ -1 2 0 -1 1 3 1]`
- c. `1 1q3 3p'X'(Y)[Z]`
- d. `'M+. *N'[p'CAT' o.='TAILS']`
- e. `'V[3]-3[101:3*(3+I)-3]~'013-*(V`
- f. `(x4|18)/'V[1]+C+5'`
- g. `q3+(x10x3x2),('$)'`
- h. `-1p'q',x+/113`
- i. `1+1+N'`
- j. `'C123'[1 2 3]`
- k. `'[/B-15'[15]`
- m. `'+(('))>'[-1p14]`
- n. `('STREET'x'E')/''0)(0''`
- o. `(1o.+~4)+'+ooo+`
- p. `qeq'.B0q'`
- q. `'V=V[1],M'~'1V'`

Utility Corner:

Converting from Characters to Numbers

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

The system function `⎕FI` (fix input or format inverse) and `⎕VI` (verify input, or validate input) are extremely useful when writing user-friendly interactive functions. When used with `⎕` input, they provide a simple and effective team for accepting numeric input. For example:

```
[1] ASK:'Enter quarterly sales figures'
[2] RESPONSE←⎕
[3] →(⌵/⎕VI RESPONSE)/OK
[4] 'Non-numeric entry. Please re-enter.'
[5] →ASK
[6] OK:NUMBERS←⎕FI RESPONSE
```

The `⎕VI` function returns a bit vector with one element per group of nonblank characters in its argument. A 1 is returned for each group that can be converted to a number (e.g. 27.5 or -3.6 or 3.1E6); a 0 is returned for each nonconvertible group (e.g. 2.3.4 or -2 or CAT).

The `⎕FI` function does the conversion, returning a numeric vector with one element per group of nonblank characters in its argument. Nonconvertible groups are returned as 0.

Here's an illustration:

```
⎕VI '26 43K -2.0 0 1E6 END -5'
1 0 1 1 1 0 0
⎕FI '26 43K -2.0 0 1E6 END -5'
26 0 26 0 -2 0 1000000 0 0
```

Without `⎕VI` and `⎕FI`, the simple-sounding task of converting a character vector to a numeric vector is difficult. The direct application of execute (`⍎`) may cause an error if the vector is ill-formed:

```
⍎'26 43K -2.0 0 1E6 END -5'
VALUE ERROR
⍎ 26 43 K -2 0 1000000 END-5
^
```

Fortunately, $\square VI$ and $\square FI$ prevent this possibility. Unfortunately, not all implementations of APL support $\square VI$ and $\square FI$. For example, APL*PLUS and Sharp APL do support them, but VSAPL and APL2 do not.

Let's write a utility function named ΔVI to mimic the behaviour of $\square VI$ for those APL implementations that don't have $\square VI$ and $\square FI$. Since $\square VI$ and $\square FI$ perform similar work, we won't bother to write a separate function named ΔFI . Rather, our ΔVI will return the global variable ΔFI . Here's an example of how you could use this utility function:

```
[1] ASK:'Enter quarterly sales figures'
[2] RESPONSE←□
[3] →(∧/ΔVI RESPONSE)/OK
[4] 'Non-numeric entry. Please re-enter.'
[5] →ASK
[6] OK:NUMBERS←ΔFI
```

Notice the similarity to the first illustration in this article.

Here's the utility function. Scientific/exponential notation (e.g. 3E5) has been ignored for simplicity.

```
▽ BV←ΔVI CV;BL;CUM;EI;I;LN;N;NB;ND;NN;NP;SI;□IO
[1]  a Does: BV←□VI CV
[2]  a and: ΔFI←□FI CV
[3]  a returning ΔFI globally
[4]  □IO←1 a Origin 1
[5]  CV←CV,' ' a Trailing blank simplifies logic
[6]  NB←~BL←CV=' ' a Nonblank, blank
[7]  SI←(NB∧1φBL)/∖ρNB a Start inds
[8]  EI←(NB∧1φBL)/∖ρNB a End inds
[9]  N←ρLN←1+EI-SI a No. of grps; lengths of grps
[10] CUM←+BV←CV='-' a Cum no. of neg signs
[11] NN←CUM[EI]-Nρ0,CUM[EI] a No. negs per grp
[12] BV←NN←BV[SI] a One neg okay only if start chr
[13] CUM←+CV='.' a Cum no. of periods
[14] NP←CUM[EI]-Nρ0,CUM[EI] a No. periods per grp
[15] BV←BV∧NP≤1 a Max one period allowed
[16] CUM←+CV∈'0123456789' a Cum no. of digits
[17] ND←CUM[EI]-Nρ0,CUM[EI] a No. digits per grp
[18] a Min one digit; nothing but negs,pers,digits:
[19] BV←BV∧(×ND)∧LN=NN+NP+ND
[20] a Consider only convertible grps
[21] N←ρSI←BV/SI
[22] LN←1+BV/LN a 1+ for trailing blank
[23] a I←(∖LN[1]),(∖LN[2]),(∖LN[3])...
[24] I←LN/Nρ0,+∖LN
[25] I←(∖ρI)-I
```

```
[26] ΔFI←BV\*CV[I+LN/SI-1] * Cvt to num via *
```

v

To help you follow the logic, we've traced the values of the variables in the ΔVI function when it's run with the test argument we used above:

```
[6] CV: 26 43K ^2.0 0 1E6 END -5
[7] BL: 0010001000010100010001001
    NB: 1101110111101011101110110
[8] SI: 1 4 8 13 15 19 23
    EI: 2 6 11 13 17 21 24
[10] LN: 2 3 4 1 3 3 2
    N: 7
[11] BV: 0000000100000000000000000
    CUM: 00000001111111111111111
[12] NN: 0 0 1 0 0 0 0
[13] BV: 1 1 1 1 1 1 1
[14] CUM: 00000000011111111111111
[15] NP: 0 0 1 0 0 0 0
[16] BV: 1 1 1 1 1 1 1
[17] CUM: 12234444556677889999999 10 10
[18] ND: 2 2 2 1 2 0 1
[20] BV: 1 0 1 1 0 0 0
[22] SI: 1 8 13
    N: 3
[23] LN: 3 5 2
[25] I: 0 0 0 3 3 3 3 8 8
[26] I: 1 2 3 1 2 3 4 5 1 2
[27] ΔFI: 26 0 ^2 0 0 0 0
```

By making heavy use of scan and shift techniques, this function can evaluate a character vector of any length without looping.

Limbering UP:

Converting Character Matrices to Numbers

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

You can use the system functions $\square FI$ (fix input) and $\square VI$ (verify input) to convert a character vector into the numeric vector represented by the characters. For example:

```

       $\square FI$  '37 26.5 -24 7R 142'
37 26.5 -24 0 142
       $\square VI$  '37 26.5 -24 7R 142'
1 1 1 0 1

```

(See the ΔVI utility function presented in the Utility Corner column if your version of APL does not support $\square FI$ and $\square VI$.)

The $\square FI$ and $\square VI$ functions work only on character vectors, not on character matrices. As full-screen APL applications become more common, we're often encountering numeric information in the form of a character matrix, not a vector. For example, we might read the contents of six numeric fields from the screen as a six-row character matrix. We need to convert it to a six-element numeric vector.

Your task is to write a function named ΔVM ($\square VI$ on Matrix) that verifies that each row of its character matrix argument can be converted to a single number. Its result is a vector with one element per row of the matrix: a 1 means that row can be converted; a 0 means it can't; a -1 means the row is all-blank.

In addition to this explicit result, the ΔVM function returns the global variable ΔFM ($\square FI$ on Matrix), which is a numeric vector of the converted numbers, one per row of the character matrix argument. For rows that cannot be converted to a single number, the result is 0. For example:

```

      CMAT
36.5
 10

STOP
-28
3 4 5
       $\rho CMAT$ 
6 5
       $\Delta VM CMAT$ 
1 1 -1 0 1 0
       $\Delta FM CMAT$ 
36.5 10 0 0 -28 0

```

The ΔVM function should be as efficient as possible and as small as possible. 200 points will be awarded for the fastest function; 100 points for the smallest. Functions that are not the fastest or the smallest will be awarded points on a pro-rata basis. For example, if your function is 4 times slower than the fastest, you will received 50 points ($200 \div 4$). The highest score possible is 30, but only if the same function is both fastest and smallest.

The highest-scoring function will be listed in the next issue of this newsletter. Also listed will be the names and scores of the top five authors.

The rules:

1. At least 20 percent of the function must be comments. (See the reference manual if necessary to learn how to enter the comment symbol.) The size of the function will be determined after stripping out all comments.
2. The speed of the function will be measured by running it on a 500 row, 10 column character matrix of "representative" data.
3. Pornographic programming techniques (e.g. embedded assignments or branching to line numbers) may be amended by the judge, but will not be cause for disqualification.
4. Entries must be received by January 15, 1990.

Good luck and happy limbering!

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT 06029, USA

Comment by Jonathan Barman

I vividly remember the excitement, 18 years ago, when I found in Quote Quad the most amazing little function for checking numeric input. Like the Zark ΔVI example, I had coded a variety of complex functions to carry out the check, with varying degrees of success. Here, at last, was a simple and elegant algorithm which cut through all the complexity and had the added bonus of being much, much faster than anything I had ever achieved!

The following is taken from the March 1980 issue of Quote Quad, Volume 10 Number 3:

Algorithm 139

Verification of Numeric Input

Gerald Bamberger

There are 24 types of valid APL numeric literals. Each of the four fundamental types — 9, 9., 9.9, .9 (where 9 represents a string of digits) — may receive a prefix and/or suffix:

- no sign or negative sign
- no exponent, positive exponent, or negative exponent

Input

The right argument *A* is a character string containing character groups delimited by strings of one or more spaces.

Output

The function result is a vector containing a 1 or 0 for each character group, indicating whether or not that group constitutes a valid APL representation of a numeric value.

Algorithm

```

v R←VI A
[1] R←' 11111111112345'[' 0123456789.~E'∘'0 ',A]
[2] R←1+⊥((R∈'234')∨R≠' ',~1+R)/R
[3] R←R∈(8 3p0 41 431)+1 12 121 21 31 312 3121 321∘.×1 100 1000
v

```

Example

```

VI '2.93 4.77E~2A ~6.77 +421 1.1.1'
1 0 1 0 0

```

Comment

This capability is available as `⍶VI` on some systems, including STSC and IPSA.

J-ottings 16 or A Composition on Composition

by Norman Thomson

Handling conjunctions is an important part of J — arguably the two most fundamental are *atop* (⊗) and *with* (⋈).

An example of composition using *with* is the verb

```
diag=.(<0 1)&|:
diag i.3 4
0 5 10
```

in which the noun `<0 1` is joined with the verb `transpose, |:`, to form the composed verb `diag` which returns the leading diagonal of a matrix argument. Composing a noun and a verb to form a new verb is a familiar part of the fabric of natural language. For example, "daydream" is a verb whose meaning cannot be fully conveyed by using *day* or *dream* separately, and the same applies to compound verbs such as "headhunt", "facelift", "datestamp", etc. Some compositions of two *verbs* are "sleepwalk", "tumbledry", "stirfry", and again in each case the composition has an extra layer of meaning which exceeds the sum of its two constituents.

The definitions of $\&$ and $\otimes$ in the two-verb case are clearly spelt out in the Dictionary :

$u\&v$ is the composition whose result is $(v\ x)\ u\ (v\ y)$ (dyadic), or $u\ (v\ y)$ (monadic) so that v is always applied monadically.

$u\otimes v$ is the composition whose result is $u\ (x\ v\ y)$ (dyadic), or $u\ (v\ y)$ (monadic) so that u is always applied monadically.

A hook is also a composition of two verbs, for which the two possible results are: $x\ u\ (v\ y)$ and $y\ u\ (v\ y)$, so that, as with $\&$, v is always applied monadically.

Because an absent left argument is given the default value y , the composed verb arising from a hook is always inherently dyadic, unlike the composed verbs resulting from monadic *atop* and *with*, which are identical.

Thinking of *u* as the prime verb, and *v* as a transformation, the dyadic cases can be thought of as

- ⊗ *u* is applied to a transformation which uses both arguments;
- ⊗ *u* is applied after a transformation has been made to both arguments separately;

hook *u* is applied following a transformation to the right argument only.

Composition takes into account a property called *rank inheritance*. Rank inheritance means that the *verb* rank of the transforming verb *v* is inherited by the prime verb *u*. The verb ranks of any verb, primitive or defined, can be obtained by issuing *v b. 0*, the result of which is three numbers, which give respectively the monadic, left and right ranks. These are respectively the maximum ranks of nouns which can be accepted by the operational routine without any shape constraint. One or more of these will often be ∞ , that is infinity, which means that the verb rank is unbounded. The rank of monadic *+* for example is unbounded, which means the operation is valid, whatever the *frame* of the argument, even although *eventually* *+* will be applied at the scalar or *cell* level. The left and right ranks of dyadic *+* on the other hand are 0, meaning that the underlying operational routine can *only* be applied at the *cell* level, since for any higher argument rank compatibility constraints restrict the *frames* of the arguments.

Any composite verb whose transformation component is modified by *each=*, *⊗* has rank 0. The definition of *⊗* includes observance of rank inheritance, whereas its variant at *(⊗:)* ignores it, and so in cases where the transforming verb is adverbially qualified by *each*, *⊗* and *⊗:* offer a rank inheritance option.

Here are some simple examples to illustrate these points. Define a rank 1 verb

```
rotv=.|. "1      NB. Rotates a matrix about its vertical mid-axis
```

so that for a matrix argument *m*, *diag m* returns the leading diagonal, and *rotv m* returns *m* with each of its rows rotated.

Issuing the expression *diag rotv m=.1.3 4* does *not* create a hook; but is an instruction to execute one verb *rotv*, and then perform another, *diag*, on the result of applying the first. On the other hand issuing either *(diag rotv) m* or

```
diagrotv=.diag rotv
diagrotv m
```

does create a hook, since what is executed in both cases is a new verb formed by composition *before* any association with data. To pursue an analogy with English,

frying the pork and *then* stirring it is *not* the way to produce a stirfry! Both (diag rotv) m and diagrotv m give rise to domain errors, because diag can only be applied monadically, whereas hooks are inherently dyadic in their application.

Now consider (diag&rotv) m and (diag@rotv) m. If the rules at the head of this article are followed without taking rank inheritance into account the result of each should be the same as diag rotv m, namely 3 6 9. However both expressions give rise to index errors. Why? Because rotv is a rank 1 verb, and so rank 1 is inherited by diag. But as m is a matrix, at rank 1 it consists of a *vector* of rows, for which, as an argument to diag, the index value 1 is out of range. This rank inheritance argument can be checked by issuing the composition (rotv@rotv)m which returns a result which is identical to m. The conjunction @: allows rank inheritance to be overruled, so that (diag@:rotv) m does indeed return the same result as diag rotv m.

As a further example, consider the differences between the following compositions:

```
n(>.&^.)m
n(>.@^.)m
n(>.^.)m      NB. Hook
```

The first is the function $\max(\ln n, \ln m)$, the second is $\text{ceil}(\log_n m)$ which is the minimum number of digits required to represent a strictly positive number m in number base n , and the third is the function $\max(n, \ln m)$ — in this case the same as $n > .^{\wedge} . m$.

Since with *atop* the transformation verb v uses both x and y , *atop* can be thought of as a sequencing conjunction, that is the prime verb u is applied only after some transformation has been applied to *all* the data, and the result passed on to it.

The verb $n(>.@^.)m$ above is an archetypal case of *atop*. The composed verb addresses the question of, for example, how many hexadecimal digits are required to represent 1000. Step one is to find the logarithm of 1000 in base 16 thereby consuming and transforming *all* the data. This is then passed on to the prime verb $\text{ceil}(>.)$.

An archetypal instance of *with* is multiplication by addition of logs, in which the two multiplicands are both transformed by log before the prime verb $+$ is applied. $\&$. (under) is variant of $\&$ which applies the reverse transformation to the result, in this case completing the multiplication process.

The difference between a hook and issuing the same verbs in sequence verbs is illustrated by the fact that $--y$ and $\%y$ both evaluate to y whereas $(--y)$ is $2y$

and $(\%)y$ is y^2 . As a further example $\%+ / y$ is the reciprocal of the sum of items of the vector y , $(\%+ /)y$ is the vector in which each item of y has been replaced with its proportion of the total.

The "bridge hook" $u \sim v$ is useful in circumstances where transformation is required on the left rather than the right argument. A simple example is using the material in x to construct an object of the same shape as y , for which the appropriate bridge hook is $(\$ \sim \$)$.

```
0 ($~$) m
0 0 0 0
0 0 0 0
0 0 0 0
```

Another example is the transformation of an integer x into a vector of y decimal digits. First transform y into a vector of y 10's by $\$ \& 10$ and then apply this as the left argument of $\#$: to define the hook $(\# : \sim \$ \& 10)$.

As well as rank inheritance, there are two traps to be avoided in sequencing compositions. The first is that it is easy to omit $\&$ when it is vital for correct meaning as in $(+ / \& * :)$ to mean "sum of squares". Without the $\&$ the verb is a hook, and the result is the addition table for the squares of the items of y .

The second trap is the omission of parentheses where these are mandatory. Conjunctions "capture" the object (verb or noun) immediately to their right (or expressed in another way, they have short right scope), and so a verb sumdev defined as

```
mean = . + / % #
sumdev = . + / @ - mean      NB.  sum the mean deviations
```

... has the meaning $(+ / @ -) \text{mean}$ rather than the intended $+ / @ (- \text{mean})$. $(+ / @ -) \text{mean}$ is a hook, for which, following the definition above, the result is $y (+ / @ -) \text{mean } y$. This means perform $y - \text{mean } y$ so that $\text{mean } y$ is calculated first to obtain a scalar at scalar level (rank 0), then apply $+ /$ at inherited rank 0, that is perform $+ /$ on the vector items, which on scalars is a "do nothing" operation, leading to, as the overall result, the vector $y - \text{mean } y$.

By contrast, with

```
sumdev1 = . + / @ (- mean)
```

the hook (-mean) is the transforming verb of the *atop* composition, and so after performing $y\text{-mean } y$, the resulting vector is presented to $+/$ with rank inheritance 1 (that is the rank of the hook).

With

```
sumdev2=. +/@: -mean
```

rank inheritance is overruled, $+/$ applies at the maximum rank, and so the result is equivalent to $+/@(-\text{mean})$. The point is that minus is, in APL terms, a scalar function, whereas mean is not, and *a fortiori* neither is -mean.

It is prudent when using $@$ or $@:$ to check verb rank by issuing $v \ b. \ 0$, which was described earlier. For example

```
(+/@-) b. 0
- 0 0
(+/@: -) b.
- - -
```

so that 5 6 7 (+/@-) 3 5 1 is 2 1 6 and 5 6 7 (+/@: -) 3 5 1 is 9.

In summary, a thorough understanding of *with*, *atop* and *hook* is a major step to confidence in handling J.

Tour de Volute — Around in a Square Whole

by Howard Peelle (hapeelle@educ.umass.edu)

An array of numbers arranged in a spiral is known as a "volute", named after the Latin *voluta* for spiral shells in nature [0]. For example, here is square volute of 25 consecutive integers, beginning with 0 at the centre:

```

12 11 10  9 24
13  2  1  8 23
14  3  0  7 22
15  4  5  6 21
16 17 18 19 20

```

A volute can spiral inward (called an "involute") or outward (an "evolute") and either clockwise or counterclockwise, emanating from any orthogonal direction — for a total of 16 variations in a plane. Since a corresponding involute can be produced easily from an evolute by subtraction, only evolutes will be considered here. Volutes can be readily transformed by reflection and/or rotation into other orientations, denoted here by compass directions of the first leg in each dimension. The example above is a NorthWest (NW) evolute.

Algorithms for producing volutes were coded and compared by Eugene McDonnell in his "At Play with J" column [1] and also by Roy Sykes in his APL "Whizbang" column [2]. (See Appendix for their programs.) This article offers alternative programs and introduces new algorithms — all written in J — including some which increase speed, decrease space, minimize length of code, and generalize. That is, we will reconsider how to fit numbers around and around in square whole arrays.

The programs to follow are grouped by approach:

Algebraic Approach	[Graham, Knuth & Patashnik]
Rings Approach	[Peelle]
Winding Approach	[Smillie; McDonnell; Peelle]
Sum-Scan Approach	[Hui]
Permutation Approach	[Tuttle; Sykes]
Squiral Approach	[Peelle]
Generalizations	[Peelle]

All programs (except generalizations) use a single positive integer (N) input and produce a square evolute of consecutive integers in origin 0 using a convenient orientation. Comparisons of speed and space (based on a sample input of N=55) and length are given progressively throughout, and then relative benchmarks are tabularized for several Ns at the end.

Algebraic Approach

Graham, Knuth & Patashnik [3 p.99 and p.512] compute the x and y coordinates of integer n by the following formulas:

$$\begin{aligned} \text{for } m &= \lfloor \sqrt{n} \rfloor \\ x(n) &= (-1)^m ((n-m(m+1)) \cdot \lfloor \lfloor 2\sqrt{n} \rfloor \text{ is even} \rfloor + \lceil \frac{1}{2}m \rceil) \\ y(n) &= (-1)^m ((n-m(m+1)) \cdot \lfloor \lfloor 2\sqrt{n} \rfloor \text{ is odd} \rfloor - \lceil \frac{1}{2}m \rceil) \end{aligned}$$

These can be expressed together in J as follows:

```
m =: <. @%:
even =: -. @odd
odd =: 2&|
```

```
xy =: (_1: ^m) * ((-m * (m+1:)) * (even, . odd) @ (<. @+; @%:)) + (+, . -) @ >. @ -; @m
```

xy creates a 2-column table for integers $1..N$ input. Then (like McDonnell's evGKPa in the Appendix) we grade-up the table and reshape the resulting permutation list into an EN volute:

```
GKPa =: ,~ $ /: @xy @ i. @%:
```

This program is more efficient (about 64% in time and 66% in space) and more concise (62% in length) than evGKPa. It can be improved a little more by some J gymnastics:

```
GKPA =: ,~ $ /: @XY @ i. @%:
XY =: Sign * (, . -) @ Square + Leg * (, . -) @ Parity
Sign =: _1: ^ m
m =: <. @%:
Square =: >. @ -; @m
Leg =: - (*>) @m
Parity =: odd @ <. @+; @%:
odd =: 2&|
```

This runs in 80% time, in the same space, and is 89% as long as my GKPa above. Although this closed form is mathematically appealing, it appears difficult to

improve its efficiency further without altering the basic algorithm. So, let's consider an alternative.

Graham, Knuth & Patashnik [3] also give a formula for the inverse, that is, to obtain n from (x,y) :

$$n = (2k)^2 \pm (2k + x(n) + y(n)) \text{ for } k = \max(|x(n)|, |y(n)|) \\ \text{where the } \pm \text{ sign is } (-1)^{(x(n) < y(n))}$$

This can be expressed directly in J as follows:

```
n =: *: @+: @k + sign * +: @k + +/
k =: >./@: |
sign =: _1: ^ < /
```

Now we can use catalog (like McDonnell's evGKPB, in Appendix) to generate the (x,y) coordinates in row-major order:

```
x_y =: [: |: >@, @({@(-)@axis
axis =: 1. - <.@-:
```

And then construct a WN volute:

```
GKPB =: ,~ $ n@x_y
```

This runs in about 50% time and 107% space and is 89% the length of evGKPB (which does not handle even N input correctly). It can be improved further by using two planes of coordinates to produce a NW volute as follows:

```
GKPB =: n @ X_Y
X_Y =: (X ,: Y)@axis
X =: # # ,:
Y =: # #"1 ,. @: -
```

This is more efficient than my GKPB above (64% time and 89% space) and shorter (96%). And it is clearly superior to my GKPA (13% time, 78% space, 87% length). Nevertheless, it is expensive to compute the coordinates. So, let's look beyond this algebraic approach.

Rings Approach

Better yet, we can make a WN volute without computing coordinates, by using matrices of 'rings' as follows:

```

HAP =: Rings @ axis
axis =: 1. - <.@-:
Rings =: Squares + Signs * Doubles + Differences
Squares =: *:@Doubles
Signs =: _1: ^ Less
Less =: </ |.@(+ One)
One =: 2: | >:@#
Doubles =: +:@Max
Max =: >./~@:|
Differences =: -/~

```

This program is much faster (9% time), smaller (47% space) and shorter (95%) than my GKPB above.

Winding Approach

Keith Smillie [4] presented a recursive program which turns 90° and winds new legs onto the side of a matrix (see evKS in Appendix). My version of this for a NE (odd N) or SW (even N) volute is:

```

KWS =: Turn @ (Right ,.~ Top , KS) @ <: ` Empty @. (=0:)
Turn =: |."1 @ |.
Right =: + *: + i.@>:
Top =: *: + i.
Empty =: 0 0&$

```

This program is far more efficient (14% time and 1% space) and shorter (75% length) than evKS. Nevertheless, recursion becomes impractical as N becomes large, so let's consider iteration.

Eugene McDonnell [1] recoded his old APL volute program iteratively in J (see evEEM in Appendix). The following alternative program builds a WS (odd) or EN (even) volute faster (42%) using less space (57%) but is much longer (146%) than evEEM:

```

EEM =: Build ^: (|`Empty)
Empty =: 0 0&$
Build =: Turn @ Bottom @ Left
Turn =: |."1 @ |.
Bottom =: , (+ *: + i.@>:.)@#
Left =: ,.~ (*: + i.)@#

```

Here is another program which builds the same volute — but without rotations — by inserting a gerund of verbs in decreasing-sized pairs of boxed legs:

```

BOX =: [: > TOP`RIGHT`BOTTOM`LEFT / @ LEGS
TOP =: ,&.>
RIGHT =: ,~&.>
BOTTOM =: ,~&.>
LEFT =: ,&.>
LEGS =: (CORNERS (+ i.)&.> }:)@REPS
CORNERS =: 2: */\ |
REPS =: ,@SIGN@(.~ >: )@i.@-
SIGN =: * # $ _1 1"_

```

It is faster (76%) than my EEM above but, alas, very space-consuming (221%) and much longer (156%).

Sum-Scan Approach

As reported by McDonnell [1], Roger Hui created a WS volute by sum-scanning a matrix assembled from copies of increments (see evHUI in Appendix). My recoding is:

```

HUI =: [: +/\ head , (drop~ (top,bot)@half)
head =: i. -- *: @(- even) - odd
even =: 2: | >:
odd =: 2&|
drop =: }.~ (.,-)@even
top =: (<: ,. !.@+: ,. ]) #"1 (1: ,. 1: + _8: * |.) .. _1:
bot =: (!. ,. <:@+: ,. |.) #"1 (1: ,. _5: + 8: * |) .. _1:
half =: >:@1.@<.@-:

```

This is slightly faster (94%) than evHUI, significantly smaller (60%) and shorter (63%).

Permutation Approach

Joey Tuttle apparently considered inverses of grade-up and sum-scan of the desired result ravelled [1]. His program evJKT (see Appendix) was the fastest in McDonnell's comparisons.

My revised program for a NW volute is slightly faster (98%), uses slightly less space (98%), but is one token longer (103%) than evJKT:

```

JKT =: ,~ $ /:@(+/\)@Increments
Increments =: _1&|. @ (# Cycles) @ Repeats
Repeats =: ,~ 2: # i.
Cycles =: # $ (.,-)@(.1:)@(:

```

As an alternative, Roy Sykes [2] employed APL indexed reassignment and other programming tricks to improve efficiency. Although Sykes reported that his program performed over 6 times faster than *evJKT* in APL+DOS, his *EVOLUTE* translated into J (see Appendix) is 2 times slower (201%) and larger (229%) than *evJKT*. This seems to reveal differences between systems and optimization of primitives.

My recoding improves on *EVOLUTE*'s speed (90%), space (82%), and length (83%) for a WS volute:

```
RAS =: ,~ $ Indices@]'Items }~
Items =: i.@*:0]
Indices =: [: +/\ _1: }. Center , Repeats # Cycles
Center =: <: * >.@-:
Repeats =: ,~ 2: # 1.
Cycles =: >:@+: $ 1 0 _1 0" _ + 0 _1 0 1&*
```

Squirrel Approach

The following program produces a NW volute by amending positions of an initial zero matrix one at a time, much like drawing a "squirrel" using Logo (see [5]):

```
SQU =: 3 : 0
V =. (,~ y.) $ 0
P =. ,~ <.-:y.
I =. _1 0
N =. 1
while. N < *:y.
do. P =. P + I
V =. N (<P)} V
if. N = {>. * <.) %:N
do. I =. |.I
if. 0 = {:I do. I =. -I end.
end.
N =. >:N
end. V
)
```

This program is awfully slow (72514%) but uses less space (88%) than McDonnell's *evJKT*.

Generalizations

All programs presented here are already generalized for a positive integer *N* input — notably including both even and odd cases — in order to produce a

square evolute of consecutive integers beginning with 0 in the centre (where the 'centre' of an even volute is one of the four central spots). Also note that all of the programs handle the degenerate case of input $N=0$ (whereas `evHUI` and `evJKT` do not).

Any two-dimensional integer evolute or its corresponding involute can be converted into each other by subtracting it from the sum of its largest and smallest items, using `Inv =: -- (>./ + <./)@.`.

Generalizing volute items to any arithmetic progression is achieved simply by multiplying an integer volute (V) by a scaling factor (S) and adding a constant (A): $A + S * V$. Generalizing to a geometric progression (or other functions) would be similar.

Each program here can be generalized to rectangular volutes with shape $N+0\ 1$ or $N+1\ 0$. For instance, the following program (modified slightly from my `JKT`) inputs the shape:

```
EVR =: $ /: @ (+/\) @ Incs
Incs =: _1&|. @ (# Cycles) @ Reps
Reps =: >: @ (+/) {. 2: # 1. @ >: @ {
Cycles =: # $ (,-) @ (.1:) @ {:
```

For example, `EVR 5 5` produces a square NW volute (same as `JKT 5`), and `EVR 6 5` produces a rectangular NW volute with 6 rows and 5 columns. Note that a 5 by 6 NW evolute is impossible although `EVR 5 6` gives a (spurious) result. Benchmarks for this program are slightly slower but slightly slimmer than for `JKT`.

Volutes may be generalized further to any rectangle, as does Smillie's `Sp` program (see Appendix). However, we will constrain volutes here to be square or rectangular with only one extra row or column, corresponding to the original squiral function [3].

A volute can also be generalized for orientation by using a second input to denote the directions of the first two legs. Thus (modified from `EVR` above):

```
EVO =: ] $ /: @ (+/\) @ Inc
Inc =: _1&|. @ (Copy Rep)
Rep =: >: @ (+/) {. 2: # 1. @ >: @ (<./)
Copy =: ] # Cyc
Cyc =: # @ $ (,-) @ [
```

We assign pronouns for the eight different orientations:

```

nw =. _1 0 ,: 0 _1
ne =. _1 0 ,: 0 1
sw =. 1 0 ,: 0 _1
se =. 1 0 ,: 0 1
wn =. 0 _1 ,: _1 0
ws =. 0 _1 ,: 1 0
en =. 0 1 ,: _1 0
es =. 0 1 ,: 1 0

```

Then `nw EV0 5 5` produces a square NW volute. `ne EV0 6 5` is a rectangular NE 6 by 5 volute, and `wn EV0 5 6` is a WN 5 by 6, etc.

Of course, this generalization is not necessary since auxiliary programs could be defined to orient a volute result. Assuming a two-dimensional volute input in NorthWest orientation, the following programs yield all orientations:

```

NW =: ]
NE =: |."1
SW =: |.
SE =: |."1@|.
WN =: |:
WS =: |.@|.
EN =: |:@|.
ES =: |.@|:@|.

```

For example, `WN EVR 6 5` is a 5 by 6 WN evolute. Benchmarks comparing (worst case) `ES EVR 55 55` versus `es EV0 55 55` indicate that using auxiliary programs is much more favorable (23% time and 59% space).

We leave for the interested reader to write a program to orient any given volute into a given orientation, as in `es Orient EVR 5 5`.

Lastly, volutes can be generalized to higher dimensions. Here is a generalized program to produce a ($D > 1$)-dimensional rectangular evolute which fills in a blank array by a "right hand rule" — for 3-D that is Up (1), North (1), West (1), Down (2), South (2), East (2), then Up (3), etc.:

```

EVD =: Indexes onto Blank
Blank =: $ _:
onto =: i.@#@[ ` (;@[) ` ] }
Indexes =: Mid +"1 0: , +/\@I
Mid =: <.@-:
I =: _1: }, R # C
R =: max { . # # i.@>:@(<./)
max =: >:@(+/)
C =: max $ ( , -)@id@#
id =: = @ i.

```

For example:

```

EVD 3 3
2 1 8
3 0 7
4 5 6

```

```

EVD 3 3 3
3 2 -
- 1 -
- - 11

```

```

4 - -
- 0 -
- - 10

```

```

5 - -
6 - -
7 8 9

```

Note that this is not array-filling for $D > 2$. Also note that rectangular evolutes are correct only when they are possible to construct, e.g., EVD 4 3 and EVD 4 3 3, but not EVD 3 4 nor EVD 3 4 3 and EVD 3 3 4.

Finally, all these generalizations can be combined in the following program to produce any dimensional rectangular evolute in any orientation:

```

EV =: indexes onto blank
blank =: ] $ _:
onto =: i.@#@[ ` (;@[) ` ] }
indexes =: mid +"1 0: , +/\@increments
mid =: <.@-:@(- 1: = +/)-
increments =: _1: }, reps@[ # cycles
reps =: max { . # # i.@>:@(<./)
cycles =: max@[ $ ( , ~ -)@[
max =: >:@(+/)

```

For example, $nw \text{ EV } 3 \ 3$ and $unw \text{ EV } 3 \ 3 \ 3$ (for $unw = . \ 1 \ 0 \ 0 \ , \ 0 \ 1 \ 0 \ , :$
 $0 \ 0 \ 1$) produce the examples above.

This program is, unfortunately, rather inefficient (1755% time and 897% space and 202% length) compared to my JKT. After all, that's going around and around in any whole volute!

Comparisons

The following tables compare speed, space, and length of my (fixed) programs for several Fibonacci values of input N. Benchmarks were obtained on a Pentium PC running J3.05c under Windows.

Speed						
Program \	N	5	34	55	144	377
GKPA		8.20	91.74	104.55	102.12	279.14
GKPB		2.13	11.97	13.40	12.79	16.50
HAP		1.25	1.23	1.21	1.15	1.44
KWS		4.22	10.11	9.71	16.85	Limit Error
EEM		2.20	6.21	7.75	17.97	284.32
BOX		2.30	4.61	5.92	16.85	659.87
HUI		2.33	1.88	1.29	1	1
JKT		1	1	1	1.01	1.48
RAS		1.80	1.88	1.82	1.63	1.99
SQU		29.94	430.69	705.10	2224.54	54602.52
EV		3.42	15.08	18.95	40.39	1110.90

The fastest program for each N is denoted with 1, and the others show factors slower.

Space						
	N	5	34	55	144	377
Program \						
GKPA		1.55	6.09	6.51	6.62	6.66
GKPB		1.41	4.20	5.10	4.95	4.71
HAP		1.05	2.24	2.38	2.41	2.41
KWS		1	1.63	1.71	1.68	Limit Error
EEM		1.02	2.05	3.78	4.65	4.33
BOX		1.31	6.68	13.96	30.55	65.05

HUI	1.37	1	1	1	1
JKT	1.16	1.25	1.32	1.33	1.33
RAS	1.20	1.91	1.99	2.00	2.00
SQU	2.71	1.22	1.18	1.09	1.08
EV	3.30	9.67	11.82	11.07	10.36

The program requiring the least space is denoted with 1, and the others show factors larger.

Length

Program \	Tokens Ratio
GKPA	1.66
GKPB	1.45
HAP	1.37
KWS	1
EEM	1.08
BOX	1.68
HUI	2.58
JKT	1.08
RAS	1.32
SQU	1.97
EV	2.18

The shortest program is denoted with 1, and the others show factors longer. The length of a program is its number of tokens (without unnecessary parentheses).

Discussion

Based on the tables above, the fastest program is HUI after accelerating past JKT at about $N=144$. The most economical in space is HUI, except KWS, EEM and HAP for small N and SQU which appears increasingly competitive for large N . The shortest program here is KWS. But note that McDonnell's program evEEM (in Appendix) reigns as most concise (even containing two unnecessary tokens).

To judge which is the best program overall, we can combine the tables — weighted equally — by averaging across the N s for speed and space. And, now ... the winner is JKT! Second place goes to HAP, just ahead of RAS and HUI. An honorable mention goes to program EV for its generality. And an award might be given for elegance. This is left for the reader to judge. Which program do you like best?

References

- [0] Cook, Theodore A. (1903) *Spirals in Nature and Art*, London: John Murray (Edinburgh Press).
- [1] McDonnell, Eugene E. (1996) "At Play with J – Volutes", *Vector* (13) 2, pp. 144-153
- [2] Sykes, Roy A., Jr. (1997) "Evolution", *APL Quote-Quad* (27) 3, pp. 27-31
- [3] Graham, Knuth & Patashnik. (1994). *Concrete Mathematics* (2nd edition). Reading, PA: Addison-Wesley.
- [4] Smillie, Keith W. (1994) "Primes, Spirals and Coffee Tables", *Vector* (11) 4, pp.104-109
- [5] Peelle, Howard A. (1998) "Squirrel Volutes in Logo and J", *International Journal of Computers for Mathematical Learning* (3) 2, (to appear)
- [6] Peelle, Howard A. (1990) "Squirrel Coordinates Problem" (EDUC 651 course material, revised March 1995) University of Massachusetts.

Acknowledgements

During development of this article, feedback was welcomed from Chris Burke, Murray Eisenberg, Roger Hui, Eugene McDonnell, Cliff Reiter, Keith Smillie, Adrian Smith, Roy Sykes, Norman Thomson, and Joey Tuttle.

Appendix:

Previous Volute Algorithms.

From McDonnell [1]:

GKPae =. 0:=2&	NB. (GKPae y)=1 if y is even
GKPao =. 1:=2&	NB. (GKPao y)=1 if y is odd
GKPaq =. <.0+:0%:	NB. floor double sqrt
GKPam =. <.0%:	NB. m is floor sqrt
GKPal =. >.0-:0GKPam	NB. ceiling half m
GKPar =. _1: ^GKPam	NB. parity (1 or _1)
GKPat =. (*>:)0GKPam	NB. m*(1+m)
GPax =. GKPar*([1-GKPat)*GKPae@GKPaq)+GKPal	
GKPay =. GKPar*([1-GKPat)*GKPao@GKPaq)-GKPal	
evGKPa =. ,~ \$ /:0(GKPaX ,. GKPay)@1.0+:	

```

GKPB0 =. +: @(>./"1)@: |
GKPB1 =. >@, @({~)@(>.@-:@- + i.)
GKPB2 =. _1: ^ </"1
GKPB3 =. *: @ [ GKPB2@ ] * [ + +/"1@ ]
GKPB4 =. (GKPB0 GKPB3 )@GKPB1
evGKPB =. ,~$GKPB4

QT=. [ , "2 {.@|:@]
Wd=. [ `((#@{.]) QT [ ] Wd #@{.])@.(0:<#@)]
evKS=. (i.@(0:0:)) Wd i.@*:@]
Sp=. (i. @ ((-/ @ ] ) , 0:)) Wd i.@(*/)@]

evEEM0=. |: @ |. , * / @ $ + i. @ #
evEEM1=. [ : i. 0 0 " _ [ ] NB. constant empty table
evEEM=. evEEM0 ^: (+: 'evEEM1)

even =: 0: = 2&|
odd =: 1: = 2&|
line0 =: *: @ (-even) - odd + i.
c3 =: 1: ,. ] ,. _1:
top=: odd. "1 ( ( | . , +: , . | . ) # "1 c3 @ (1&+) @ ( _8&*) ) @: >: @ i. @ -: @ ( -->: @ even)
bot=: -@ even. "1 ( ( | . , . <: @ +: , . | . ) # "1 c3 @ ( _5&+) @ ( 8&*) ) @: >: @ i. @ -: @ ( -odd)
evHUI =: [ : +/\ line0 , top , bot NB. @: in top and bot for J3.05

evJKT =. ,~ $ /: @ (+/\) @ evJKT2
evJKT1=. <: @ +: $ _1: , ] , 1: , -
evJKT0=. ): @ (2: # >: @ 1.)
evJKT2=. _1&|. @ (evJKT0 # evJKT1)

```

From Sykes [2], translated into J:

```

EVOLUTE =: 3 : 0
Z =. i. A =. y. + y.
B =. (2!y.) + */0 2# : y.
Z =. Z (+/\ A { B, (2#>: i. y.) # (+: y.) $ _1 0 1 0 - + 0 1 0 _1 * y.) } Z
(2#y.) $ Z
)

```

Howard A. Peelle
University of Massachusetts
Amherst, MA 01003, USA

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

APL and J: Working Together, Yet Separately

by Donald B. Pittenger (dbpitt@msn.com)

Other articles have described how APL can access J to have certain specialised tasks performed—for instance, doing extended-precision calculating [1]. Here, I present an example of how APL and J can be used in different aspects or phases of the same project.

The general idea is to exploit some strengths of each language while minimising exposure to weaknesses. This is a cautious strategy for the APLer who wishes to enter (tentatively) the world of J without leaving the safety of having APL remain the basis for mission-critical or enterprise-critical tasks.

Background

It is assumed the reader is acquainted with APL and its operation under Windows in the guise of either Dyalog APL/W or APL2000's APL+WIN. A cursory knowledge of J is also assumed, though ignorance of J should not be a great impediment for understanding what follows: examples of J code are not presented.

Let me start by sketching how I have dealt with APL in the broad sense over the past 15 years. Some (most?) of you might find my approach strange, but I hope you will see how my use of J fits into the scheme.

My behaviour pattern—especially in the last 10 years—is to flit from APL vendor to APL vendor. I started with STSC's APL*Plus for the IBM PC (version 2.6 at first, then a long while with 3.0). Eventually, I switched to APL*Plus II. Favourable reviews in *Vector*, plus a good price, led me to get APL2/PC, though it turned out to be fragile. Three years ago, I felt I needed to run systems under Windows, so I got APL*Plus III (and Iverson Software's Windows version of APL, but it was too slow for serious data production). I bought Dyalog APL/W (version 8 for Windows 95) about a year ago, and keep up to date in APL+WIN as well. I have been using J for about two years.

All this thrashing around is meaningful to this report because it seems like almost every time I get a new APL, I rewrite my main demographic forecasting system. (I do demographic, household, and income forecasts, largely for automobile

industry clients.) I find it cleaner to go to a different APL when it is time for a major re-write. Perhaps this lessens the chance of using old, patched-up code, even though key algorithms are ported from system to system.

I should explain that I have never used APL workspaces for anything but throwaway systems. I have each APL function residing in a separate file, and my systems call in slugs of code according to directory paths fed to file-reading routines. No doubt this imposes some inconveniences, but I think it makes for safer, cleaner systems. Porting programs from one vendor's APL to another's is a matter of using an AV index conversion routine. Obviously, this is only for data-processing code. User interfaces have to be written from scratch in the new system.

As this is written, I have nearly completed a new system in APL/W—the previous system was in APL+WIN. The new system is largely batch-oriented in that it will project data for a large number of countries in one swoop, whereas the old system had each nation as a separate operator task. This change meant a switch to using large, ASCII tables (in *.txt format) containing key parameters or control data such as annual births by nation. And it so happens I find J handier than APL when dealing with *.txt files.

Thanks to the newly perceived need to deal with ASCII files and the availability of an APL-like language that easily works with such data, I find myself using both APL and J for the same general project.

Where APL is Used

I suggested above that caution can be a watchword to many APLers. As Adrian Smith forcefully stated in his APL 91 presentation [2], it is prudent not to be locked into one APL. You should assume that you will switch interpreters or vendors in the future, and plan your software systems and coding practices accordingly. His Causeway graphical interface software, for instance, grew from a desire to avoid being tied to one vendor for interface reasons.

I agree with his philosophy, but must admit I lack the discipline to carry it out rigorously. When I wrote my core systems in APL2/PC, I took to using the 'Squad' index in order to avoid writing a lot of executable strings containing ';;;' blocs sized to fit arrays of variable rank. The Squad was fine, aside from the fact that Squad-expressions tended to be rather long and perhaps a little hard to read. Most other APLs lack the Squad, so when I moved from APL2, I wound up writing a Squad cover function, and of course it was peppered with ';;;'s.

Safety in numbers

To return to the point, a really prudent APLer will not (yet) abandon APL for J. I know of five APLs for personal computers, while there is only one J vendor. The 'safety in numbers' concept, then, is one reason for sticking with APL for critical systems. Yes, J can do some tricks APL cannot, but this does not outweigh the risk of betting everything on one vendor. Nor does any present J speed advantage: if the speed difference were an order of magnitude—or even half that—then a switch might be worth it for speed-critical tasks.

Fully-featured interface

Another APL advantage, at least for now, has to do with the Windows interface. Both APL/W and APL+WIN have more fully-featured interfaces than J. For example, J does not offer MDI (scaleable, overlapping forms such as you find in Microsoft Word or Microsoft Excel). Nor, as this is written, does J have its own grid-object or spreadsheet-interface object. Yes, J can be a DLL server for C++ or Visual Basic (as can some APLs). But dual-language programming involves a learning curve and potential complexities that can be avoided for in-house systems such as those I normally write. To put it differently, it is fast and convenient to write the entire system, including the Windows part, in an APL environment, especially if the interface really needs features J's Windows Driver lacks.

Where J is used

Well, if APL is so wonderful, why use J at all? Recall that I mentioned J was handy for dealing with *.txt files. It also has other features that make it useful as an adjunct to APL. Of course, APL can do most things J can do, including the work described below. But using J can be considered yet another prudence ploy. Should J really 'take off' or should APL 'hit the wall', the prudent system builder will have experience writing in J as well as a set of useful J programs!

I like examples that are concrete rather than theoretical, so let me describe how I use J in conjunction with my core APL-based systems.

Defining a uniform baseline

My original international demographic systems dealt with less than two dozen nations, most of them of the advanced industrial sort with lots of detailed data. Today, I need to project each country in the world with population of a million or more, and might have to include even smaller nations in the future. From the start, I had to deal with the problem of data variation from country to country,

but the move to ever-smaller nations (often with sketchier and sketchier data) has forced me to keep revising the software system. The problem is this: I want to offer uniform and fairly detailed data to clients, but the source data are anything but uniform, and often are not very detailed. I have a further need to keep estimated (historical, non-forecast period) data close to actual values—plugging in actual data directly, if possible. And I have to estimate or fill in details that are absent. What this boils down to is that I must 'calibrate' data from many nations so that a single, baseline set emerges. The baseline set is uniform, and that is what the forecast software is designed to use.

I presently use J in the calibration phase. APL programs do the actual calibration computing. But much of the computing is based on input from a number of tables, and it is J, along with Microsoft Excel, that deals with these tables.

Dealing with text files from Excel

The tables are created and, for the most part, maintained using Excel; J performs critical ancillary tasks that will be described. Excel is for data entry. The spreadsheet tables are saved as tab-delimited text files (Excel's *.txt format noted above). Most tables contain a single time-series such as total births per year. Thus, the table will consist of perhaps 230 rows (one per country) and as many columns as there are years in the time-series. Table topics include total population, total births, total deaths, household totals from censuses, Gross Domestic Product (GDP), Consumer Price Indexes (CPI), and monetary exchange ratios.

In some cases, I have more than one table for the same subject: I have a table of 1 July population totals from the United Nations, and another table with official population estimates or population register tallies and the associated dates. I standardise my populations as of 1 January. Official populations are assumed to take precedence over the more comprehensive UN data (even though many UN data are from official sources). So, I need to *blend* the tables so that I wind up with population totals for 1 January. J routines are used to do this blending, and the resulting table—a key control for both calibration and the projection run—is saved to file.

There are several operational benefits from this practice. For one thing, data from separate sources can remain separate: this makes it easier to update material or to check for data entry errors. Secondly, the blending program takes care of all the reference-date checking and needed interpolation, leaving the source tables with pure content for the tasks just mentioned. Thirdly, once a change is posted to one

of the source tables, it takes only a couple of minutes to fire up J and create an updated blended table.

Where J feels more at home

What follows is purely subjective on my part, though it might be based on technical realities. To me, J seems to feel crisper, more at ease dealing with *.txt data than APL. ASCII files are read into J using fread, and the content is boxed Excel cell by Excel cell using clipunfmt. These are standard J utility programs. Once boxed, the data are generally simple to deal with in further processing. In the case cited in the previous paragraph, the blended table is boxed and then the utility clipfmt transforms it into a tab-delimited string with end-of-line characters (in other words, *.txt format!) before it gets written to file using fwrite.

One reason why APL might seem a shade clunkier is that I wrote my own format/unformat routines because, through ignorance or oversight, I do not know of professionally-written routines that do the deeds well. Still, once the data are boxed, to me J still feels 'more at home' than does APL with nests when it chews on table data.

My J forecast system-related scripts are nested under a top-level directory labelled DataManagement. This is coequal with the top directory for the APL forecast system itself. Besides table-blending, J is used to splice and unsplice historical and projected control numbers, to rummage database directories to inventory files by nation and data subject, and to inspect certain kinds of data in the database directory.

This last task involves more elaborate Windows work than simply selecting from lists: graphs are shown. One inspection routine lets me select a nation, then a data subject, and finally a file containing that data for a single observation. An example would be the population of France by sex and age group from the 1982 census. Once the file has been read, I can query the data source, any comments I might have entered (such as if there were people of unreported age), the age groups represented in the data, and a couple other items of potential interest. In the case of population by age-sex, I can have J draw a population pyramid—a standard demographic graph which is simply back-to-back bar graphs with the age dimension running vertically.

Finally, I have J do certain off-line tasks such as prepare estimates of age-sex-specific household headship rates using a very simple Fuzzy procedure.

Comments

From the above, the division of labour I have imposed on APL and J should be fairly clear. J works with Excel-generated files and Windows directories (or 'folders' in current parlance). So far as APL-created files are concerned, J is used only for inspection of content.

APL creates, modifies, and adds data to certain classes of files that contain only ASCII characters, but are crafted so that the file content is expressed in strand notation for easy conversion to a nested array. (J can read these files and, with a little effort, box the content.) APL also reads *.txt files and converts the content into nested arrays. These files are data controls of the type described above as well as 'forecast script' files. Let me emphasise that APL's role is simply to *read and convert* these files to nested arrays—it is not used to modify them, even though it is capable of doing so.

If, in the future, I switch APLs again, the J-based software will be completely unaffected, so long as system revision does not change the content of control data files, scripts, and so forth. Actually, a system rewrite usually means that some of these will be changed, some will be abandoned, and new files will be added. Still, the net gain ought to be that less new APL code would be needed than if the entire system, including Windows features, were changed from APL/W back to APL+WIN.

J is also being used in my sub-national forecasting system, currently in prototype form. I am using J here so that I will have code that parallels my main system code just in case I am compelled to abandon APL; the sub-national system can be revised into a nation-level system fairly easily. However, given J's still-limited Windows programming capabilities, I am finding myself in a situation complementary to that described above. I really do need MDI and grid objects for certain data-support programs involving county or provincial numbers, so I will need APL to support J rather than having J support APL! Either that or turn to dual-language programming.

Conclusion

Like many APLers, I was initially hostile to J. Truth be told, I am still not very fond of it, though I respect it more than I did. It is ugly to look at, and might well be too large. Documentation and learning tools are improving, but still sub-par, on average.

On the other hand, J seems to be a good data processing language. It is handier than APL in that I can call a J script from Windows Explorer using Notepad and make changes without having to go through the bother of firing up J: to fix APL code, you must be in APL. And, as described above, it is especially handy for working with spreadsheet-based tables and other ASCII text data.

If, having read my example, you can see where something like this might work for you, I heartily encourage you to try it. You don't have to stray too far from APL to do so; for instance, I don't get into particularly J-ish business such as writing tacit functions. But be warned that J is not simply APL without the symbols. In spite of claims that it is just a dialect of APL, I find it bears about the same relationship to APL as German does to Swedish.

References

- [1] Chris Burke, *The J DLL and Extended Integers*, Vector, Vol. 13, No. 1, pp. 149-53, July 1996.
- [2] Adrian Smith, *Supply-Chain Management at Rowntree: Critical Success Factors for APL*, APL Quote Quad, Vol. 21, No. 4, pp. 301-306, August 1991.

Internet Programming with the Internet Server Auxiliary Processor

by Andrei Kondrashev (71303.3224@compuserve.com)

Processes and Threads

Multitasking is the ability of an operating system to run multiple programs concurrently. Under the Windows NT operating system, concurrently running programs are called *processes*. Each process has a private virtual address space, its own copy of the executable code, GDI objects, etc. Very few Windows resources can be shared between different processes. Processes have limited possibilities to communicate with each other. All interprocess communication methods are strictly controlled by the operating system. *Multithreading* is the ability of a process to multitask within itself. Each process starts with a *single thread*. Additional threads can be created later on. All threads of a process run in the same virtual address space and share all resources of the process. A thread can execute any part of a process' code, including a part executed by another thread. This allows many independent and concurrently running threads that are using the same code, to perform similar operations on different data. All threads can access global variables of the process, but they use their own stacks that make the thread's local variables unique. Windows NT implements multitasking at the thread level.

HTTP

HTTP (stands for HyperText Transfer Protocol) is an application-level networking protocol. HTTP can be implemented on top of any reliable transport protocol (usually TCP). Well-known port number for HTTP is 80. HTTP is a multi-purpose data transfer protocol. It can be used to transfer any type of data. HTTP is a "sessionless" protocol. When a client needs to get data from the server, it connects to the server's port 80 and sends an *HTTP request*. The server replies with the *HTTP response* and closes the connection. Thus, the term "sessionless" means that the session consists of only a single act of message exchange. A good example of a "session-oriented" application protocol is FTP. HTTP is very easy to implement, but relatively tricky to use in interactive network applications. Despite the fact that modern Web browsers and servers are able to re-use the TCP connection to increase performance, that does not change anything for the application

programmer. From his point of view, each HTTP request initiates an absolutely new session with the server. No previous exchange history is directly available.

An HTTP request consists of three parts: the HTTP request line, HTTP request headers, and the HTTP request body. The structure of the request line is:

<METHOD><SP><URL> [<?><PARAMETERS>] <HTTP/1.0><CRLF>

Where:

Characters "<" and ">" are used to separate different elements of the line;

<METHOD> can be "GET", "HEAD", or "POST";

<SP> means the blank space ASCII character;

<URL> is a universal resource locator;

<PARAMETERS> is an optional list of parameters;

<CRLF> means two ASCII characters: carriage return and line feed.

The *universal resource locator* (URL) is a character string that identifies the resource (data file, or something that will produce the data file) on the Internet. Generally, it consists of the address of the Internet server and the path information. Both IP and DNS addresses can be used to address the server. The path information indicates the location of the resource on the server. The server will map this virtual path using its own internal rules to find the physical location of the resource requested.

Parameters (or query string) may specify additional information for the server's data-producing process. Parameters are separated from the URL by the "?" character. For example:

`www.lingo.com/isap/isap.dll?ws=counter`

The *method* defines how the additional request information is being transferred to the server. If the method is "GET" or "HEAD", the request does not contain the request body. If the method is "POST", the request body contains additional information to be used by the server's data-producing process.

The *request headers* follow immediately after the request line. Header fields (each must be terminated with CRLF) carry additional information about the request and the client itself. If the request body is present, there will be at least the Content-Type and the Content-Length headers. An empty line (contains CRLF only) indicates the end of request headers.

The *request body* can be anything. The type of the information is defined by the Content-Type header. If it is missing, plain ASCII text is assumed. This is an example of an HTTP request (note the empty line between the request's headers and body):

```
POST www.lingo.com/isap/isap.dll?mydoc.htm HTTP/1.0
Accept:image/gif,image/jpeg,*/*
Content-Type:application/x-www-form-urlencoded
Content-Length:7
- empty line here -
ws=demo
```

The structure of the HTTP response is similar to the structure of the HTTP request. Only the request line is replaced with the *response status line*. This is an example of the HTTP response:

```
HTTP/1.0 201 OK
Content-Type:text/html
Content-Length:44
- empty line here -
<HTML><BODY>This is a response</BODY></HTML>
```

The response body can be any binary output: plain text, HTML, GIF or JPEG images, movie clips, etc. It also can contain client-side-specific data, such as a Microsoft Excel spreadsheet, for example. There are two distinct ways the HTTP server can prepare the response:

- server returns a static document defined by the URL;
- server runs a data-producing program defined by the URL, and that program prepares the document that is returned by the server.

The client doesn't distinguish between these cases, and doesn't care how the HTTP response was produced.

HTTP Servers and APL

A program that understands HTTP requests, and is able to send back to the client HTTP responses, is called an HTTP (or *Web*) server. As we saw above, HTTP is a very simple protocol. All Web "magic" happens on the client side. A Web browser is a rather complex program that parses HTML, creates forms, decides what method and URL to use, optimizes communications with the server, etc. The server, in its turn, is relatively simple. One of the most important features that any HTTP server should support, is the ability to run external data-producing programs that generate HTTP responses dynamically. This feature drives all

commercial use of the Internet. A data-producing program can be run as a separate process on the server's computer or as a separate thread of the server's process.

The simplicity of an HTTP server is very tempting. As soon as the socket programming became available under all major APL (and J) implementations, it was possible to develop HTTP servers purely in APL. If the socket interface has a callback feature, a pretty decent simple HTTP server can be implemented in about 30 lines of APL code. This server, of course, will be able to use APL programs to generate dynamic HTTP responses. Let's discuss what's wrong with this idea.

Poor performance. Up to 99% of the work that any Web server has to perform is simple file transfer. It means that the APL program will frequently `⎕NTIE`, `⎕NREAD`, and `⎕NUNTIE` the files. APL is not very suitable for these operations. There will be multiple moves of data between various buffers before the data will be, finally, dumped to the TCP output buffer. Note that Windows NT is able to send files over the TCP connection very efficiently, in the background, taking almost no CPU time. Internet servers use this facility, so the performance of an APL-based server that has to send many static documents will be much worse in comparison with, let's say, Microsoft Internet Information Server, that has been specifically designed to perform such operations very efficiently. Next, as of this writing, an APL-based server cannot take direct advantage of the fact that it is running under a multitasking operating system. Only non-preemptive multitasking can be implemented under all currently available APL systems, until real multithreading is available. As a result, the simple file transfer operations that IIS does in parallel on all active connections, would be executed sequentially by an APL server. Of course, the performance is something application-dependent. You may decide that the performance of an APL server is acceptable for you. My point here is that standard tools will perform better anyway.

All industry-standard Internet servers must support various **security and encrypting features**. In order to implement the necessary level of security, the server must be deeply integrated with the operating system. APL programs run under control of an APL interpreter that isolates them from the operating system. Of course, anything is achievable, but is the result worth the effort spent? Probably not, especially because full-scale security and encrypting cannot be implemented purely in the APL environment anyway.

The **maintenance** issue is rather important. Usually, in large organizations a special department performs everyday routine maintenance of all servers such as data backup, setting the permissions on the server's resources, and so on. In most cases those people are not programmers at all. They are "certified" users (network

administrators, etc.), who completed special training and who know exactly how to do certain things. Infrastructure departments don't like to use non-standard critical software developed by a third-party vendor, because it might be difficult to find and/or to train the supporting personnel. This is why, for example, it is so easy to invent yet another programming language, but it is so difficult to make someone else use it. It's the same story with Internet servers: clients prefer to use system software developed by Microsoft, IBM, or Netscape, but not by MyCompany, Inc.

An APL HTTP server won't be able to run anything except APL. This is not good. There are many other Internet programming tools around. There will be more, for sure. Not all tools are equally suitable for all purposes. The freedom to choose the right tool and to use more than one of them simultaneously on the very same Internet server is important for many companies that use the Internet commercially.

Résumé

APL Web servers are possible, but their usage is limited to Intranets (because of lack of sufficient security). They can be acceptable for really heavy applications that are used by a very limited number of concurrent users, in cases when a data-producing program is so complex that other issues become less important in comparison with the advantages of direct use of APL. There are various tricks that allow deploying APL servers indirectly and under control of "standard" Web servers. As an example, you may look at <http://www.1travel.com>. The hotel booking part of this Web site is supported by a hidden APL server (implemented by *Lingo Allegro USA, Inc.* and *Shepp & Associates LLC*).

The access to this server is fully controlled by another Internet application written in a language other than APL.

However, APL is a very suitable language for Web-based applications. Much better, from my point of view, than anything else available on the market now, including Perl. The only (and usual for APL) problem is how to use it together with other existing software systems and only in the area where APL is unbeaten: the data processing. Let's look at what is currently available.

CGI vs. IISAPI

One of the most popular and simple methods of creating dynamic HTTP responses is called *Common Gateway Interface* (CGI). The idea of CGI is very simple. When the server determines that the URL defines a data-producing

program, it creates a new process using the code defined by the URL. The server sets the process's environment variables in order to pass the parameters of the request: `REQUEST_METHOD`, `CONTENT_TYPE`, `CONTENT_LENGTH`, and others. The server also re-directs the process's standard input and standard output devices to the TCP socket that supports the HTTP connection. Thus, a CGI program can simply use the standard input/output functions to read and write to the socket. The CGI process terminates when the program has sent all data to the socket. The main advantage of CGI is that virtually any programming language can be used, including APL. All that you need is the ability to access the environment variables, and to read and write to the standard output device. Because the processing of HTTP requests heavily involves string manipulation operations, special languages have been designed specifically to use with CGI.

What's bad about CGI? First of all, each CGI request must be supported by its own process. For example, if there are ten CGI requests processed by the server, there will be ten concurrently running processes (probably, executing exactly the same code) on the server computer. Despite the fact that NT tries to re-use code and resources from the previous instances of the same program, by mapping portions of physical memory to the virtual memory of different processes, it is not always possible to avoid the replication of the same code. When APL runs, the workspace will be duplicated for sure. You can use the Task Manager to analyse how much it costs you to run more than one instance of your application.

Secondly, it takes some time to start a CGI process. For such a "heavy" system as APL, this time can be substantial (we will need to load a workspace too). Next, because a CGI process terminates when it's done with the request, it cannot take direct advantage of previous runs of the same program. For example, if a CGI program must access a database server in order to create the response, it will have to log on to the server every time, when it starts. For many industry standard database servers, the logon is a very unpleasant and long procedure (may take dozens of seconds), which will add an additional overhead to the HTTP server response time. All these reasons make CGI practically unusable for sophisticated server-side processing. CGI programs can be used only for very simple operations and should be coded in compiled languages that produce small executables.

Different vendors of HTTP servers introduced various alternatives to CGI for developing data-producing programs. The rest of this article will discuss so-called *server extensions* for the Microsoft Internet Information Server (IIS).

IIS exposes a special application interface (IISAPI) that can be used by developers to create very effective data-producing programs. Server extensions are Windows dynamic link libraries (DLL) and they run as threads of the IIS process. When IIS

receives a request that requires running an extension, it loads the corresponding DLL and uses the DLL's code to start yet another thread. The DLL remains in the IIS virtual address space even after the request is completed. Because threads can share the same executable code, IIS needs to have only one copy of the extension DLL to run any number of concurrent independent threads, which require this DLL in order to generate HTTP responses. Many different extension DLLs can be used at the same time.

Thus, the first obvious advantage of IISAPI versus CGI is that IIS extensions require zero loading time. Secondly, they do not waste memory, if more than one concurrently executed request requires running the same data-producing program. Extensions are more effective than CGI, because they can take advantage of the fact that they are part of the server. The IIS exports functions that allow the program to perform data exchange operations more efficiently, rather than just reading and writing to the standard input/output device. IIS extensions can also use the fact that they are not removed from the memory. They can be permanently connected to a database server, for example. IISAPI provides the best possible performance in comparison with all other available server-side Internet programming tools.

The only disadvantage of IIS extensions is that they are relatively difficult to develop. Only the C language can be used for programming. An extension DLL must be *thread-safe*. It means that the DLL must work correctly, even if more than one thread, based on the same code, is run by the same process. Any additional DLLs that could be used by an extension, must be thread-safe too. If you are planning to use ODBC, be prepared that most ODBC drivers *are not* thread-safe. So, for us, as APL programmers, it would be very desirable to use APL as an IIS extension. Lingo Allegro USA, Inc. developed the Internet Server Auxiliary Processor (ISAP) that can be used with Dyalog APL to develop IIS extensions in APL.

Internet Server Auxiliary Processor

The idea of ISAP is to provide the interface between Microsoft Internet Information Server and Dyalog APL. From IIS's point of view, it has to be just an extension DLL. From Dyalog APL's point of view, it should be something that can accept commands and data from APL. ISAP needs to inform APL about incoming HTTP requests, and to be able to execute IIS functions on behalf of APL, such as getting information about requests, reading client data, and sending back HTTP responses. And it has to be able to handle multiple concurrent requests too.

Figure 1 shows the architecture of the IIS-ISAP-Dyalog APL interface.

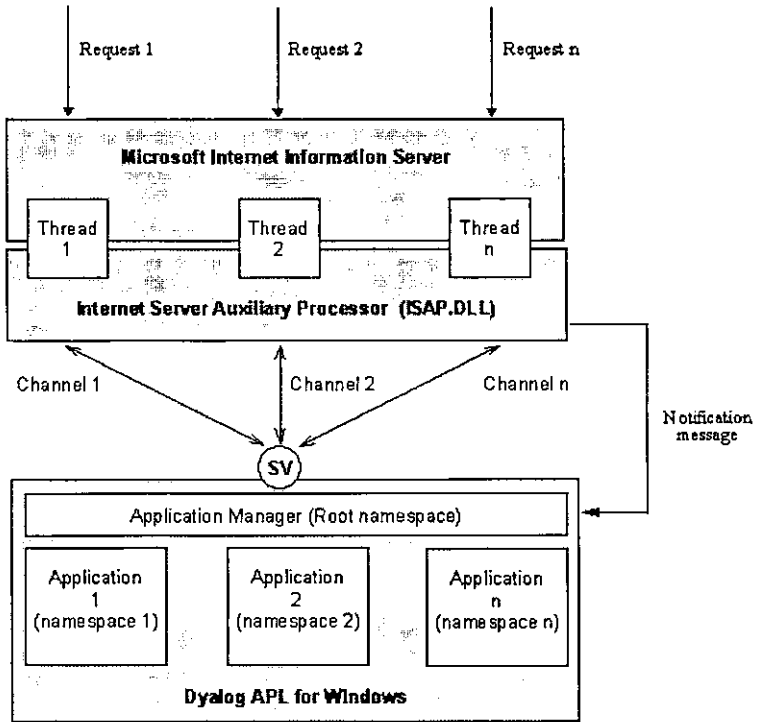


Figure 1

When ISAP.DLL is loaded by IIS for the first time, there is only one main ISAP's thread running. This thread handles communications with Dyalog APL using one shared variable. Immediately after loading, ISAP starts Dyalog APL with an application workspace. How to start APL and what workspace to use is specified in ISAP's initialization file. When an HTTP request that refers to ISAP.DLL in its URL arrives at the server, the server starts a new thread of ISAP. This new thread is called a *channel*. The channel posts a notification message to APL and switches into a wait state, effectively allowing other threads to run. The notification message has only one parameter - the channel number. After the notification message is received, an APL application can execute commands on this channel. More than one channel can exist at the same time. The APL application can work with the channels in any order it wants. It can process more important requests before less important ones, for example. It can split time-consuming requests into

steps and process short requests between the steps of long requests. All of this is decided by the APL programmer.

The *SERVER* function below is a generic IIS-based application.

```

SERVER;_ISF
[1] SETGLOBALS
[2] →SHARE+EXIT
[3] '_ISF'□WC'FORM'('COORD' 'PIXEL')('EVENT'5'SERV_CB')('VISIBLE'0)
[4] SAY'NOTIFY'('_ISF'□WG'HANDLE')
[5] '._□WS'EVENT' 131 'SERV_CB'
[6] □DQ'.'
[7] EXIT:CLEANUP
[8] END

```

When Dyalog APL starts the application workspace, this function is executed as the result of the execution of the latent expression. First, the application performs its initialization by running the *SETGLOBALS* function. This is application-specific. Next, it executes the *SHARE* function, which is a part of ISAP. This function establishes the connection between ISAP and Dyalog APL by sharing a variable. After that, the application creates an invisible window, called *_ISF*, that will receive the notification messages from ISAP (of course, you can use any other name for the form object). *_ISF*'s callback function (*SERV_CB*) must process at least one message, message 5, which is ISAP's notification message. Other events can be processed too, if the application is going to implement any kind of non-preemptive multitasking within itself. When the notification window is created, the program sends the *NOTIFY* command to ISAP, which informs it about which window should receive the notification messages. The callback function should also handle event 131 - Windows shutdown - in order to perform any necessary clean-up (such as disconnection from a database server, updating and closing files, and so on). Finally, the *SERVER* goes into the message waiting state. When *□DQ* exits, which can happen on IIS shutdown, the application performs the necessary exit procedures by executing the *CLEANUP* function. The *END* function is a standard ISAP procedure that terminates ISAP.

The *SAY* function is a part of ISAP and is used to send commands to the auxiliary processor. The general syntax is:

R←SAY COMMAND ARGUMENTS

COMMAND is a simple character vector—the command's mnemonic. *ARGUMENTS* is a general APL array that depends on the command used. The function returns the result of the command, as its explicit result, or generates error 667 that has to be trapped by the application. Most of ISAP's commands should pass the channel

number as one of their arguments. ISAP will execute such commands in the context of the appropriate thread of the IIS. Many commands are executed by ISAP asynchronously. It means that in cases where it is possible ISAP will return control to APL before IIS completes the execution of the command. This allows APL and IIS run simultaneously, increasing overall server performance.

All real work on processing HTTP requests takes place in the *SERV_CB* function. This function could be very straightforward and simple, if the application only needs to process a single HTTP request, or it can be rather sophisticated, if you want to run more than one APL application under IIS. It can process HTTP requests in the order it receives them, or it can implement a non-preemptive multitasking to process many requests simultaneously. These are choices that are left completely up to the APL programmer. ISAP doesn't care which approach you use. Figure 2 shows the principal diagram of HTTP request processing under ISAP.

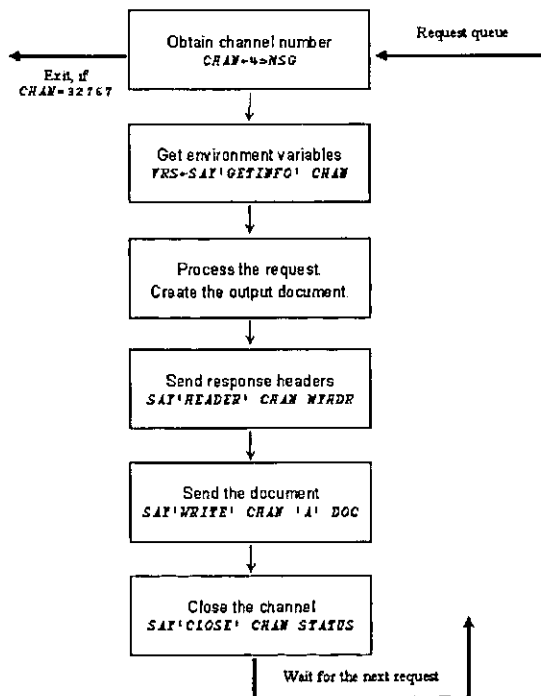


Figure 2

The processing always starts with the GETINFO command (unless you don't care what request you are processing). This command returns a 19-item nested vector that carries the complete information about the IIS channel and the HTTP request. It also contains the posted data from the client, if the POST method was used in the HTTP request. All data, returned by this command, are translated to the Dyalog APL character set, including any characters that are transmitted as escape sequences.

The processing of any request must end with the CLOSE command. This command informs IIS that the processing of the request has been completed. IIS terminates the corresponding ISAP's thread and closes the TCP connection with the client.

An APL programmer can use anything available in the APL language and other ISAP's commands between GETINFO and CLOSE in order to send an HTTP response back to the client. Any type of response body can be used: HTML, image, or a custom binary file. It is also possible to redirect the request to another URL on the same server, or to a URL on some other server on the Internet. ISAP supports several commands that produce GIF and x-bitmap images that can be constructed using the standard graphical functions of Dyalog APL.

You can download the complete ISAP documentation from the Lingo Allegro USA, Inc. Web site at <http://www.lingo.com>.

Sample Application: Page Reference Counter

One of the most popular applications running on Web sites is a page reference counter. This application displays (usually in the form of a nice looking picture) the number of references to a page (usually, the site's home page). Of course, if some Web page is generated dynamically, the page counter can be generated as text simultaneously with the page content. However, this is not convenient to use. Many pages are very complex and the most of their content is static. A better approach would be to take advantage of the fact that some HTML elements require an additional HTTP request to be sent by a browser. One such element is an embedded image. Images can be inserted in the HTML document using the tag. The SRC parameter of the tag denotes the URL that the browser should use in the HTTP request in order to obtain the image file. Thus, if we use the URL to a picture-producing program, instead of a static picture file, it will do the job. This approach allows the use of the same counter-generating program to count any page on the Internet by simply inserting the appropriate tag in the document we want to count references to.

The following is the example of an HTML file that can be found at <http://209.19.63.4/docs/count.htm> (the IP address is fictional):

```
<HTML><HEAD>
<TITLE>Internet Server Auxiliary Processor</TITLE>
</HEAD>
<BODY BGCOLOR="#FFFFFF" TEXT="#000000">
<CENTER>
<H1>The Counter Demo</H1>
<P>Number of hits to this page:
<P><IMG SRC="/ISAP/ISAP.DLL" BORDER=0 ALT="Counter">
</BODY></HTML>
```

We are assuming that the ISAP.DLL file can be found in the /ISAP virtual directory on the server. To make our example as simple as possible, we will create an application that is able to handle only one HTTP request. This is why the SRC parameter of the tag refers to ISAP.DLL without any additional information, such as the application name, which would be needed, if we wanted to execute more than one APL application using ISAP on the same server.

First of all, we need to decide how we will keep the information about pages and their reference counters. In APL the simplest way to do this is to use a two-column nested matrix. The first column is a nested vector of simple character vectors that are URLs of pages that we are counting. The second column is a simple numeric vector, each element of which is the current number of references to the corresponding URL from the first column. We could store this matrix in an APL file. The application will read this file at initialization time, and will write the updated matrix back to the file when it terminates. The file will have the same name as the application workspace, and will be stored in the same directory. We need to create this file manually and to initialize its first component with an empty matrix.

There are many ways to construct the counter's image. One possibility is to use bitmaps that represent elements of the counter. How the counter will look is a matter of taste. Let's use a fixed length counter with, let's say, seven decimal places. We will fill unused decimal places with a "blank" element. It means that we need eleven bitmaps of the same size that represent "blank" and ten digits. I have created such bitmaps that look like the elements of an LCD display. It is a good idea to read these bitmaps from the disk during the initialization to save time while processing requests. We should delete them, when the application ends. We will assume that all bitmaps are stored in the same directory where the application workspace resides, and that the file names are B0.BMP, ..., B10.BMP,

where B10.BMP represents the blank element. It will be possible to change the appearance of the counter by replacing the bitmaps.

Now we are ready to write the *SETGLOBALS* and *CLEANUP* functions for our server application (see the *SERVER* function above).

```

SETGLOBALS;PATH
[1]  [WSID] [FSTIE TN+1+[/0,[FNUMS
[2]  URL+[FREAD TN,1
[3]  COUNT+URL[;2] * URL+URL[;1]
[4]  PATH+(-(φ[WSID)\'\')+[WSID
[5]  PATH+(cPATH,'B'),"(*~1+11),"c'.BMP'
[6]  ('B',"~11)[WG"(c'BITMAP'),"c"(c'FILE'),"c"PATH
[7]  HEIGHT WIDTH+B1'[WG'SIZE'

```

The *SETGLOBALS* function creates eleven bitmap objects with names B1, B2, ..., B11. It also leaves five global variables in the workspace: *URL* - the vector of previously referenced URLs, *COUNT* - the vector of current numbers of hits for each URL, *TN* - the database file tie number, *HEIGHT* - the height of the counter image, *WIDTH* - the width of one element of the counter image.

```

CLEANUP
[1]  (PAGE,[1.5]COUNT)[FREPLACE TN,1
[2]  [FRESIZE TN
[3]  [FUNTIE TN
[4]  [EX [NL 9

```

The *CLEANUP* function writes the updated content of the database back to the file and destroys all GUI objects. The text of the application's callback function is shown below.

```

MSG+SERV_CB MSG;CHAN;DATA;I;MF;SIZE;PNT;PIC;SIZ;[TRAP
[1]  ^ IS WINDOWS SHUTTING DOWN?
[2]  +(131=2>MSG)+SHUT
[3]  ^ IS IIS SHUTTING DOWN?
[4]  +(32767=CHAN+4>MSG)+EXIT
[5]  ^ SET ERROR TRAPPING
[6]  [TRAP+667 'C' '->DONE'
[7]  ^ GET URL OF REFERER
[8]  DATA+cTOUPPER 8>SAY'GETINFO'CHAN
[9]  ^ FIND THIS URL IN THE DATABASE
[10]  +((pURL)>I+URL\DATA)+L1
[11]  ^ NEW REFERER. ADD IT TO THE DATABASE
[12]  URL+URL,DATA * COUNT+COUNT,0
[13]  ^ INCREMENT THE REFERENCE COUNTER
[14]  L1:COUNT[I]+COUNT[I]+1
[15]  ^ REPRESENT COUNTER AS A SEQUENCE OF BITMAPS

```

```

[16] I+ 'B', "0123456789" '17 0*COUNT[I]
[17] A CREATE METAFILE
[18] SIZE+HEIGHT, 7*WIDTH
[19] 'MF' WC METAFILE ('COORD' 'PIXEL') ('SIZE' SIZE)
[20] A WRITE BITMAPS IN THE METAFILE
[21] PNT+ c" (c'POINTS'), "0, "WIDTH*1+17
[22] PIC+ c" (c'PICTURE'), "I
[23] SZ+ c" 'SIZE' (HEIGHT, WIDTH)
[24] (c'MF.') WC (c'IMAGE'), "SZ, "PNT, "PIC
[25] A SEND IMAGE TO THE CLIENT
[26] CHAN SENDIMAGE ('MF' WG HANDLE') (SIZE)
[27] A CLOSE THE CHANNEL
[28] DONE: SAY 'CLOSE' CHAN 0
[29] +0
[30] A WINDOWS IS SHUTTING DOWN
[31] SHUT: CLEANUP
[32] CHAN+ SVR NL 2 o CHAN+ DL 1 2 o NQ'.' 131
[33] +0
[34] A IIS IS SHUTTING DOWN
[35] EXIT: EX'.'

```

When the client's browser loads the `count.htm` file (without the participation of our program) and finds the `<IMG>` tag, it will send another request using the URL that is specified in the `SRC` parameter of the tag, in order to get the image. The `REFERER` parameter of the HTTP request will be the URL of the parent Web page (the `count.htm` file). IIS will recognize that the extension program should be run and will call ISAP. ISAP sends the notification message to APL signaling that it is time to start the processing of the request.

The `SERV_CB` function that will be called by the interpreter, when the notification message arrives, first checks whether Windows or IIS is going to be shut down. If so, the application terminates. Otherwise, the fourth element of the message is ISAP's channel number. The function sets the trap for error 667 that can be generated by ISAP commands. If everything is OK with the code, this error can be generated when, for example, a client presses the browser's **Stop** button before our program sends the response. In this case, IIS will close the TCP connection with the client, and the attempt to send the response will fail. If this happens, we simply need to close the channel. The program obtains the URL of the referrer as the 8th element of the result of the `GETINFO` command on this channel (line 8 of the function). This is a simple character vector. It is converted into upper case and stored in the `DATA` variable. Lines 9-16 update the database and retrieve the current value of the counter for the URL. The value of the counter is represented as a nested vector of seven names of the bitmap objects that we created before. Lines 17-22 of the function create a metafile and assemble the counter picture from those bitmaps. The `SENDIMAGE` function is used to create

and to send a GIF image back to the client. After that the program closes the channel and the callback function exits.

The *SENDIMAGE* function uses two ISAP commands: *IMAGE* and *WRITE*:

```
CHAN SENDIMAGE ARG
[1]  n SENDS GIF IMAGE
[2]  SAY 'WRITE' CHAN 'N' (' IMAGE ARG)
```

In the above context, the *IMAGE* command converts the metafile into a GIF image and returns the complete HTTP response, including all necessary headers, that is sent back to the client by the *WRITE* command. Figure 3 shows the browser's window after loading *count.htm* file a few times.

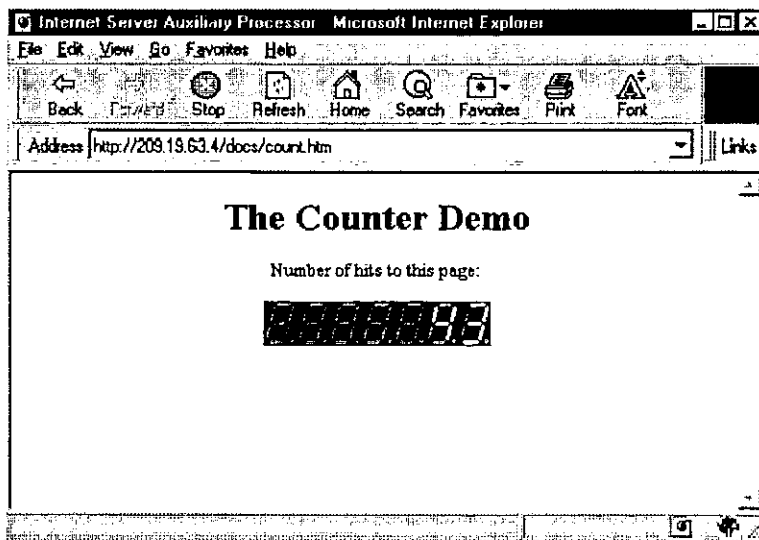


Figure 3

Multitasking

The idea of using APL for server-side Internet programming assumes that we should be able to run more than one APL application from the same server. The simplest, but not the best, way of doing this is to run more than one copy of *ISAP.DLL* (must be placed in different virtual IIS directories) connected to its own copy of *Dyalog APL*. The only advantage of this approach is that Windows NT will run the applications in parallel without any effort spent from our side.

Another approach would be to implement the multitasking using only a single copy of ISAP and Dyalog APL. This is much better, because it saves a lot of computer resources and in many cases can work even faster than straightforward replication.

Figure 4 shows the idea of an application manager that can run multiple APL applications using only one copy of Dyalog APL.

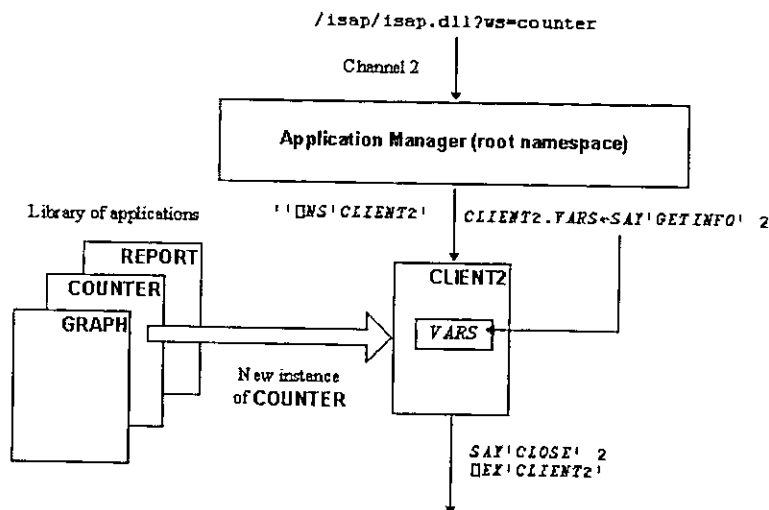


Figure 4

The application manager consists of the root namespace (the manager itself) and the library of applications (several namespaces). When an HTTP request invokes the application manager, it obtains the request's channel number (in our case 2) and creates an empty namespace, called `CLIENT2`. So, the channel number serves as a thread identifier. After that, the manager reads the request parameters and stores them as a thread global variable. The request must include the application name. In our example, the value of the request's `WS` parameter identifies the namespace to run. The manager copies all objects from the corresponding library namespace to the `CLIENT2` namespace and starts the execution of the thread by executing the value of some global variable in the namespace (the namespace's "latent expression"). When the execution of the "latent expression" is completed, the application manager closes the channel, deletes the namespace, and exits the callback function.

This idea of running many applications using a single copy of Dyalog APL is used on Lingo Allegro USA, Inc.'s Web site. You can download a sample workspace that will explain all the details of this approach. Other similar solutions are also possible.

This method, however, will not allow us to run more than one "application thread" at the same time. The reason for this is that the application manager cannot receive any new messages until the callback function exits. In order to implement non-preemptive multitasking, the program manager must interrupt the execution of a "thread" and exit from the callback function more or less often. Of course, the manager cannot really "interrupt" the execution of a program in a namespace, but a namespace itself can yield control to the program manager. A good opportunity for that, for example, could be a situation when a program places a request to a database server and waits for its answer. The program manager should be able to recognize such situations and to exit the callback function without expunging the namespace. The namespace will continue its execution, when the notification message arrives from the database access tool, and the program manager calls the namespace from the point where the namespace's execution was interrupted. Of course, a database auxiliary processor has to have a callback mechanism, like Lingo Allegro's AP747 does. Any external operation that is executed by Windows in a separate thread, and which causes an APL program to wait for its results, is a suitable moment for a namespace to yield control to the application manager. However, there must be a callback feature in place that is able to signal back to APL that the program can continue its execution.

Not all applications need to be interrupted. For example, the page counter program that we showed above needs just a fraction of a second to complete. Such applications can be run in a single step of execution. If a namespace just runs, performing long calculations, interrupts can be organized artificially. In this case, the namespace must post (not send) an appropriate message using the `⌈NQ` function in order to resume its execution at a later time.

As soon as APL interpreters have real multithreading, server-side applications would become much simpler and would have much better performance.

HTML Basics for APLers (IV): Forms and Servers

by Adrian Smith (causeway@csi.com)

Introduction

This is the fourth article in our open-ended series on HTML. It covers the HTML 'Form' tags, but you will quickly see that these are only the very beginning of what you need to know in order to begin constructing an interactive net-based application. Clearly, there is little point in generating an input document if you have no idea what to do with it when the user presses 'Submit', so I will try to cover some of the basics of constructing a useful Web-server as part of the necessary background to generating effective forms. Part (V) of the series will focus in more detail on the server side of the equation, and will show how to put some decent application code behind the simple forms developed here.

The first thing to appreciate is how incredibly primitive this all is! APLers who learned their trade on 10cps '2741' terminals are way ahead of AP124 programmers, and quite out of sight of serious Windows hacks. You *can* write quite effective applications with this stuff, but turn your brain back a good 20 years before you start.

As before, all the sample code and the demo application are available on the Causeway web site (www.causeway.co.uk/html.zip). The Web-server is currently Dyalog only (adapted from WWW.DWS as supplied with Dyalog APL) but a comparable +Win workspace will be around in time for the next issue of *Vector*.

Basic Principles of Application Design

To write a Web-server in Dyalog APL you need an elementary knowledge of TCP/IP (gleaned from reading their manual) and a function to read native files. To write a Web-server that does something useful, you need to be very aware of the utter simplicity of the HTML protocol and to employ some rather strange tricks to work around it. The fundamental point is as follows:

- each interaction between client and server knows nothing about anything that happened before. Every evening when you come home from work you meet afresh the attractive lady who lives in your home. You introduce yourselves, negotiate who will cook supper tonight, and so on. The following day you do it all over again, from the beginning. Sound familiar?

Consequently ...

- any 'session-based' information must be carried with the HTML messages that go to and fro. If you want to remember your loved one's name, she must write it on a scrap of paper and give it to you in the morning. She might also note that it's your turn to do the dishes tonight. When you come home in the evening you can refer to this (assuming you carried it with you) to resume the relationship where you left it.

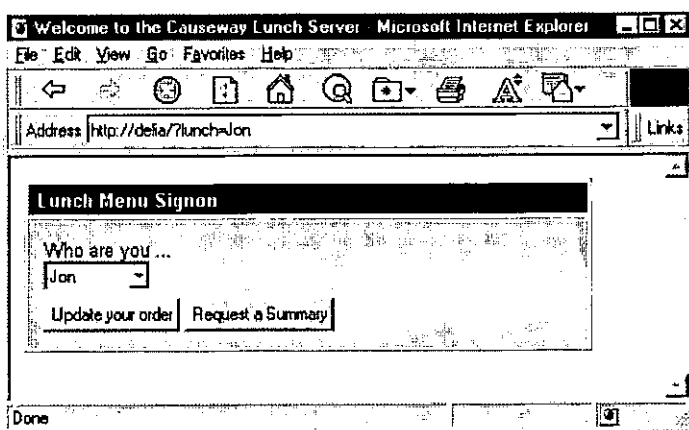
... which makes absolute sense if you are carrying on with several wives and have no idea which one you will go home to tonight! All you need to do is pick up the right scrap of paper before you walk out of the office.

As usual, let's begin with a real example and then build it up in easy stages.

The Causeway Lunch Server

One of the best things about working in Malton is Mr Skinner the Butcher, who does the best belly-pork sandwiches in the world, bar none. Normally we all wander across the road together, but occasionally someone is on the phone, or just buried on a really horrible line of VBA. When this happens you need to get his lunch order and remember it for at least 2 minutes - a tough challenge for even the best APLer.

Why not run a simple intranet application that sits on the server taking lunch orders? The default should be 'what I had yesterday'. Come 1.00 anyone can check the summary and go and get the sandwiches! It could look a bit like this:



The string 'lunch=Jon' was sent to the HTTP server on network address 'delia' which chose the correct name from the list of known users and offered a signup screen. I suppose we might one day institute some security ...

Assuming Jon presses 'Update your order' he should then see a screen like this ...

The screenshot shows a Microsoft Internet Explorer window titled "Causeway Lunch Server". The address bar displays "http://127.0.0.1/". The main content area contains a form titled "Please complete your lunch order". The form includes a "Sandwich" section with a dropdown menu currently showing "Pork Belly", and radio buttons for "White" and "Brown" bread. Below this is a "Special Instructions ..." text area containing "Salt and Pepper". At the bottom of the form are "Update" and "Cancel" buttons.

It will be pre-filled with his usual order, ready to be modified and returned to the server. At this point, you should begin to smell a small rat ... nowhere on the form is Jon's name mentioned! When he presses 'Update' all that the server sees is a set of token-value pairs like:

SANDWICH = Double Egg
BREAD = Brown
SPECIAL = Salt and pepper
SUBMIT = Update

... so there is no way it can know who submitted the order. (We will find out how those tokens were assigned in the next few pages.) You might think that you can rely on the fact that it is Jon who just signed on, but of course this whole process

is completely asynchronous, and Duncan and Marc might well have requested their orders in the meantime. You have no way of knowing what order the requests will return in, or if they will return at all! This is where the first 'dirty trick' of HTML application development comes in - we need to add a 'hidden field' to the form to record the user's name so that it is returned to us when he submits it for update. The form looks just the same, but now we get back a crucial extra token which tells us who sent it:

WHO = Adrian
SANDWICH = Double Egg
BREAD = Brown
SPECIAL = Salt and pepper
SUBMIT = Update

This way we send the user-name out to the browser in the hidden field, and it gets returned to the server as one of the token-value pairs. Of course you can have as many of these fields as you need, and in a real system you would probably generate a meaningless (unique) value for the 'WHO' token afresh at each login, rather than passing this kind of data backwards and forwards for all to see.

So now you can begin to see how it all works. It's time to go back to basics and introduce the raw HTML tags you will need to build these input forms.

The HTML Form Controls

In HTML, a form is simply defined as all the controls enclosed between ...

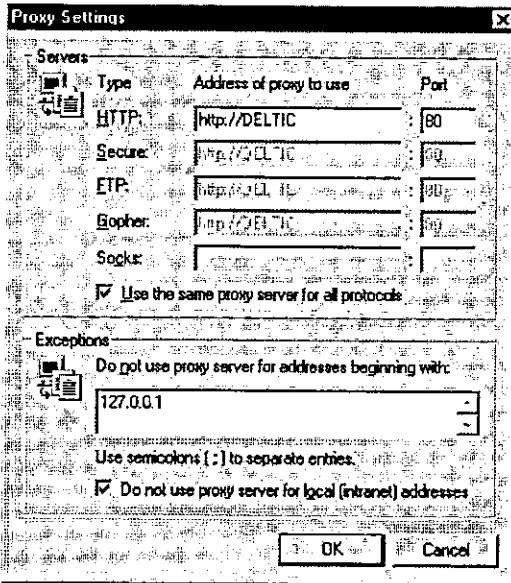
```
<form action="your url here">
```

```
</form>
```

... which does allow you the freedom to have several 'forms' on the same HTML page. This can be very useful if you want to be selective about the fields you send down the line, but for the moment I will assume we are building a simple one-form page.

The <form> tag itself always includes an 'action=' element. This gives the target internet address and port number (80 is usual) where the completed data should be sent, for example "`http://localhost:80/`". The special node-name "localhost" always translates as 127.0.0.1 which is hard-wired into TCP/IP as a 'loopback' to your own machine. Very handy when you are testing this stuff on a portable in the train! If you want to add other name-address mappings, just edit the file `\windows\hosts` and add all the entries you need. There is a sample called `\windows\hosts.sam` to get you started.

One small aside here – in IE3 you have the option of connecting to the internet through a 'proxy server' (often called a firewall). This will be the normal setup in any large organisation. For some reason IE3 seems unaware that the loopback is a local address, so you should find the 'Options, Connections, Settings' screen and make sure it looks like this:



... as nothing will work otherwise.

Between the `<form>` tags you add any number of data fields, which all have the generic form:

```
<input type=ty name="NAME" value="Hello">
```

... but as you would expect there are lots of minor inconsistencies which you just have to learn to live with. However you can now see how those token-value pairs were generated – when you submit a completed form (usually by pressing a button labelled 'Submit') you get back a long string of name=value pairs, corresponding to all the fields with non-empty values which occur on that form.

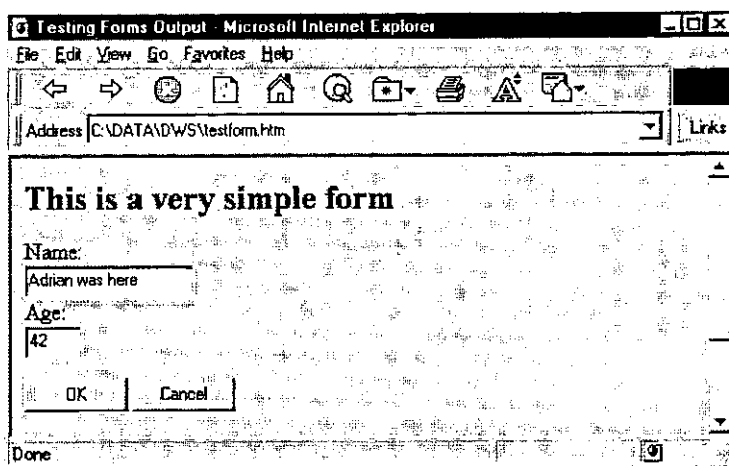
See any competent HTML book (such as [1] or an updated version of it) for the full list of valid input fields and attributes. As usual, stick with HTML 2 tags unless you are willing to be very browser-specific.

Here is a very simple example of a fully-working HTML stream:

```
<HTML><HEAD>
<Title>Testing Forms Output</title>
</HEAD>
<BODY>
<h2>
This is a very simple form
</h2>
<form action="http://localhost:80/">
Name:<br>
<input type="text" size=20 name="NAME" value="Adrian"><br>
Age:<br>
<input type="text" size=4 name="AGE" value=42><br><br>
<input type="submit" name="EN" value=" OK ">
<input type="submit" name="QT" value=" Cancel ">
</form>

</body></HTML>
```

... and here is how it looks in the browser:



So far, control over the field layout is limited to some `<br>` tags to get each element on a new line and the 'size' attribute which is the field width in characters. As you can see, the result is quite acceptable, and it is easy for our server to handle the results. For the moment, I will ignore the technical stuff, and assume that submitted text arrives by magic. If you want to know how to set up a listening socket, just read the documentation that came with your APL system!

When you press <OK> the browser will connect to the address in the action field, which just happens to be an APL socket object, and send it a text string such as:

```
http://localhost/?NAME=Adrian+was+here&AGE=42&EN=+OK+
```

... which you can parse out into a nice 2-column matrix of token-value pairs quite straightforwardly:

```
NAME = Adrian was here
AGE = 42
EN = OK
```

Note that giving these <input> tags unique, easily recognised, names is essential - I have used upper-case throughout this article just to make them stand out a little.

Things get a little trickier when the text involves 'special' characters, for example:

```
http://localhost/?NAME=M%FCller+was+here%21&AGE=42&EN=+OK+
```

It is not just the hi-bit characters that get hexed - because the syntax uses '+' and '&' and several others it has to protect these too, so the string can get quite strange. Here is my function to reverse the mappings:

```
txt←BackFlip mme;pc;nn;ch
⍝ Reverse standard MIME-encoding
⍝ First switch the '+'->' ' as real '+' signs are hexed!
((mme='+)/mme)+' '
⍝ Preserve '?=&' from the original string as something improbable
((mme<'?=&')/mme)+'␣' ⍝ Section sign
pc←('%'=mme)/␣mme
:If 0<ppc
  nn←mme[pc+.1 2] ⍝ hex digits (upper case)
  ch←Ascii[1+16 16␣1+␣'0123456789ABCDEF'␣nn]
  mme[pc]←ch
  txt←mme[(␣mme)~.pc+.1 2]
:Else
  txt←mme
:End

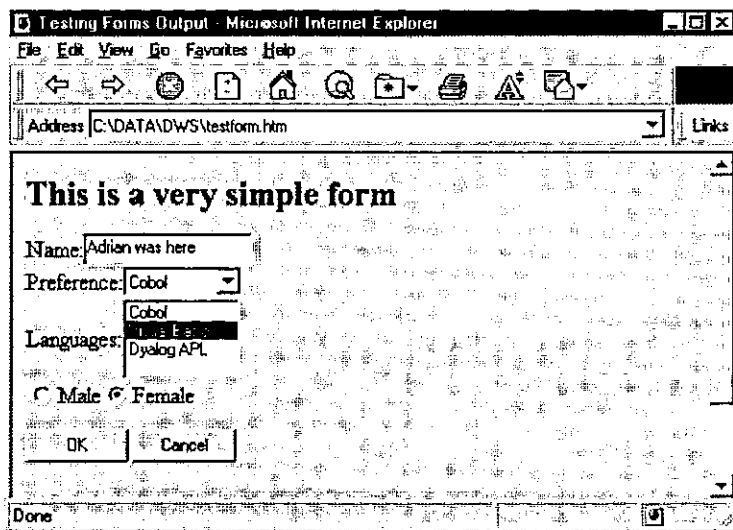
BackFlip 'NAME=M%FCller+was+here%21&AGE=42&EN=+OK+'
NAME␣Müller was here␣AGE␣42␣EN␣OK
```

Now we simply partition on the '␣' and we have our tokens and values. Of course everything is still character - so □VFI comes back into the picture and you will have to decide what to do about invalid entries. Unfortunately there is no field type of 'number' or 'date' yet available, so we are back to the pre-Windows approach of checking the input in our APL code.

The next set of field types to look at are the 'multiple choice' elements - in this case we can use a basic list box (single or multiple selection) or a drop-down list (single selection only). We also have the possibility of groups of radio buttons. Here is our basic form again with these elements added:

```
<form action="http://localhost:80/">
Name:<input type="text" size=20 name="NAME" value="Adrian"><br>
Preference:<select name="JOB">
  <option selected>Cobol</option>
  <option>Visual Basic</option>
  <option>Dyalog APL</option>
</select><br>
Languages:<select name="LANG" multiple>
  <option>Cobol</option>
  <option selected>Visual Basic</option>
  <option>Dyalog APL</option>
</select><br>
<input type="radio" name="SEX" value="male">Male
<input type="radio" name="SEX" value="female" checked>Female<br><br>
<input type="submit" name="EN" value=" OK ">
<input type="submit" name="QT" value=" Cancel ">
</form>
```

... and here is what IE3 makes of it:



As you can see, the result is really ugly, with a hotch-potch of colliding fields and no control at all over the layout.

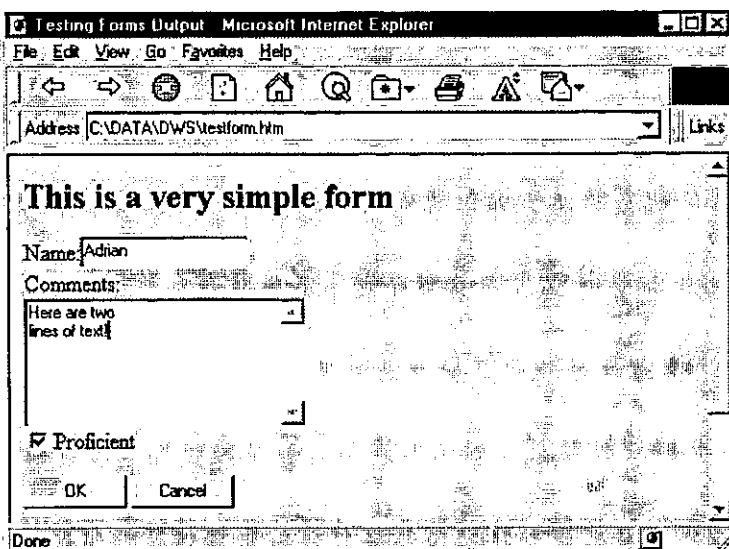
Of course it still works OK, and the drop-downs at least give you some up-front data checking, so you can rely on legal values here:

NAME=Adrian+was+here&JOB=Cobol&LANG=Visual+Basic&SEX=female&EN+=OK+

Finally, there is the 'notepad' field for free-form text input, the 'hidden' field which simply serves to pass name-value pairs out to the form so that they come back with the query, and a simple check box completes the set:

```
<form action="http://localhost:80/">
Name:<input type="text" size=20 name="NAME" value="Adrian"><br>
Comments:<br>
<textarea name="COMMENTS" cols=40 rows=5 wrap=virtual></textarea><br>
<input type="hidden" name="HI" value="one">
<input type="checkbox" name="PROFICIENT" checked>Proficient<br><br>
<input type="submit" name="EN" value=" OK ">
<input type="submit" name="QT" value=" Cancel ">
</form>
```

This one looks more reasonable:



NAME=Adrian&COMMENTS=Here+are+two%0D%0Alines+of+text%21&
HI=one&PROFICIENT=on&EN+=OK+

That pretty well wraps it up. There is a 'password' field, but anything you type gets shown in the browser window as soon as you submit the form! It all seems

quite primitive, but there are still a few sneaky tricks left over which will help us make the layout look a lot more like a 'real' Windows dialogue. What we need is *tables* – read on!

Designing a Simple Form

You may remember from the second article in the set [2] that tables are one of the few HTML structures which give you any detailed control over layout. You can set cell widths, and you can span cells across more than one row or column. You might begin to get really wild ideas about defining a table with lots of small square cells and using *rowspan* and *colspan* to get the layout exactly how you want. Forget it! I tried some experiments here and my conclusions are:

- *colspan* is absolutely fine in all browsers from Netscape 1.2 upwards
- *rowspan* is only reliable for the leftmost column of a table. It is clearly intended for 'stub' row titles and the implementers never considered the possibility of spanned rows deep in the body of the table. It usually fails at the fourth or fifth occurrence and the rest of the table disintegrates.

So – we can span columns but things just have to take their chance vertically. Sounds bad, but actually you can get a surprisingly good effect with just this simple enhancement. Here is that sample form again, this time embedded within a table:

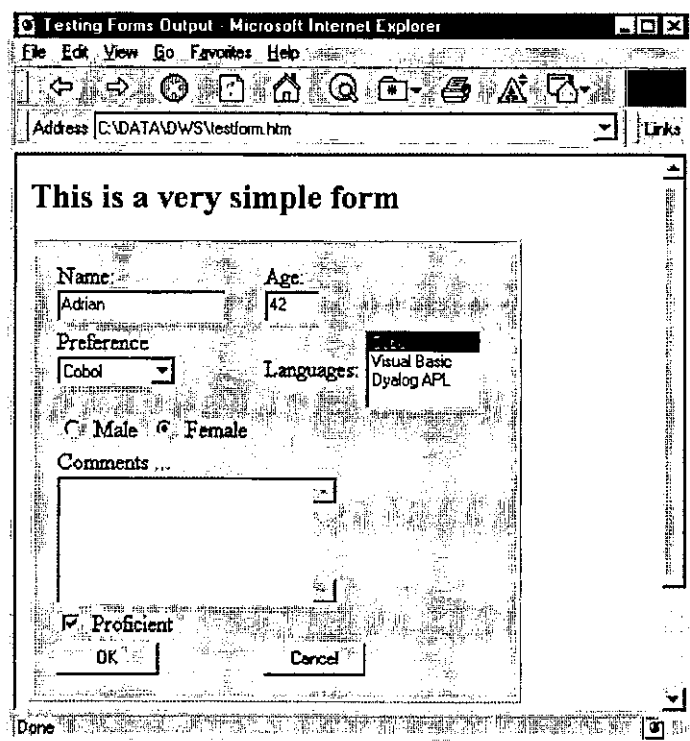
```
<table border cellpadding=12>
<td bgcolor="#C0C0C0"><table>
<form action="http://localhost:80/">
<td width=120>Name:<br>
<input type=text size=20 name="NAME" value="Adrian"></td>
<td width=48>Age:<br>
<input type=text size=4 name="AGE" value=42></td><tr>
<td valign=top>Preference<br>
<select name="JOB">
<option selected>Cobol</option>
<option>Visual Basic</option>
<option>Dyalog APL</option>
</select></td>
<td valign=top>Languages:
<select name="LANG" multiple>
<option>Cobol</option>
<option>Visual Basic</option>
<option>Dyalog APL</option>
</select></td><tr>
<td><input type=radio name="SEX" value="male">&#160;Male&#160;&#160;<input
type=radio name="SEX" value="female"
checked=&#160;Female&#160;&#160;</td><tr>
```

```

<td colspan=2>Comments ...<br>
<textarea name="comments" cols=40 rows=5 wrap=virtual></textarea></td><tr>
<td><input type=checkbox name="proficient" checked>&#160;Proficient</td><tr>
<td><input type=submit name="EN" value=" OK " ></td>
<td><input type=submit name="QT" value=" Cancel " ></td><tr>
</form>
</td></table>
</table>

```

Which now looks like this:



Hopefully, you can see how we used table cells to get the captioned fields lined up, and to get several fields on the same line. You can also see how we used *colspan* to get the notepad to occupy a reasonable width. The tendency of table cells to increase vertically to accommodate their content did the rest. Finally, you can see how the whole form is wrapped in an outer table (with just one cell) to get a nice grey background and a suitable border. Incidentally, use of words like *SEX* may be dangerous – you will probably get your page blacklisted!

HTTP 'Methods' - GET or POST?

So far, all the forms have used the default 'GET' method which submits all the fields and their content in a query string, which you see displayed in the browser window. This has the advantage that astute users can bookmark the entire string and bypass your initial forms completely! It also is less likely to alert the browser to the dangers of transmitting data over the internet (thus triggering annoying warnings). However you might also consider:

```
<form action="http://localhost:80/" method="POST">
```

... which shows nothing in the browser window (password fields are more useful here) and is unrestricted on the amount of data transmitted (some browsers cut off the 'GET' string at 256 characters). However it is a little trickier to parse - the sample Dyalog server makes no attempt to deal with these - so if you do use my server code from the web site, check it out carefully and test it with strings over 8K in length, which is where TCP/IP typically chops messages into packets.

Coming Soon ...

The next article in the set will cover ...

- Converting a *CausewayPro* Form Definition to an HTML Form
- Constructing a useful web server from the samples provided with the APL interpreters
- Parsing and processing the user's request string, using either GET or POST
- Using images in forms to provide simple graphical 'drill-down' capability

If you want to read ahead, HTML.ZIP on the causeway web site has all the examples from this Vector and many of those from the next one.

References

- [1] *The HTML Sourcebook*, Ian S. Graham, John Wiley 1995
- [2] *HTML Basics for APLers - Tables*, Vector 14.1 page 83.

J4 and OOP

by Chris Burke

J release 4.01 has extensions to locales that not only enhance their utility in general, but in particular make OOP easy in J.

A prime motivation for the new extensions was the development of the new Grid Control. The underlying primitives in J4 allow you to follow the OOP model exactly, as has been done for the Grid Control. However the new facilities are of general applicability, and so powerful and easy to use that we expect them to dramatically change the way we build systems in J.

For a good introduction to the ideas behind the new locale extensions and OOP, see the labs *Locales* and *Object Oriented Programming* included in the J4 system.

The design of the new locales, and the development of the Grid Control, was largely the work of Eric Iverson.

In this article, I will illustrate with a simple example, then discuss the new language facilities, and finally show how they are used in the Grid Control.

Example: Plot Package

Let's begin with a problem: how to create multiple plot windows with the plot package.

Suppose you draw a plot, as follows:

```
load 'plot'  
plot i.10
```

Even a simple plot like this generates a host of globals describing the plot - the labels, the axis placements and so on - in fact, about 140 of them. In J3, in order to avoid name conflicts, the code is run in a single locale, which has about 500 definitions in total for the plot system.

If you now try a different plot, with:

```
plot sin i.10
```

this redraws the plot in the same window. The code is run in the same locale and simply overwrites the plot globals.

What if, instead of redrawing the plot, you wanted to create another plot window? In J3, you would have to load the entire plot system into another locale and run it from there; and similarly if you wanted to attach a plot to your own form.

Clearly, the use of locales helps – without them you would have to draw both plots in the same namespace – but there is unnecessary duplication of code in the two locales.

J4 permits an easy solution to this. You can still draw plots in exactly the same way as in J3, but if you want to draw two plots, you can now do so by creating two plot objects, as follows:

```
load 'jzplot'      load the plot class
                   load 'plot' works just as well
                   since it also loads the plot class
```

(Read jzplot as “j” meaning a J system facility, followed by “z” meaning a basic class, followed by the name “plot”.)

```
a=. conew 'jzplot'  create plot object a
b=. conew 'jzplot'  create plot object b
```

(The *co* prefix stands for class/object. In J, you can use the OOP term *class* to refer to the script defining the class, or to the definitions once the script has been loaded in; and use the OOP term *object* to refer to the locales created by *conew*, i.e. *numbered locales* as described below.)

```
plot__a i.10      draw plot in a
plot__b sin i.10  draw plot in b
```

The objects *a* and *b* contain only the globals specific to each plot; while the main plot code is stored elsewhere, in *jzplot*. Thus J4 trivially permits more than one plot to be created, while not duplicating the main plot code.

Now, the problem solved relates only to the separate sets of globals defining the plots. But exactly the same mechanism permits customization of the code for one plot, leaving the other unchanged.

For example, the verb *drawlines* specifies the lines to be drawn; if you wanted to create a customized version for the plot in *a*, you could define, for example:

```
drawlines__a=:    new definition...
```

and this definition would be used when you next called `plot__a`.

Of course a problem of this type (creating multiple plot windows) can be solved in other ways. But the point is that in J4, it is completely trivial, whereas before, a fair amount of work would be involved in setting up a suitable mechanism.

New Locale Facilities

There are essentially three new facilities, which are upward compatible with J3:

- the ability to create a locale, returning a reference to the locale as a result
- the ability to specify locale paths
- the ability to switch the current locale under program control

In contrast, in J3:

- locales were created automatically by defining a name in one. There was no concept of returning a result from creating a locale
- the only locale structure was that there was a single parent locale (the `z` locale), and a path from any other locale to the parent
- you could only switch the locale under program control, by making a name reference to that locale.

Locales in J3 had a simple definition that worked surprisingly well; and the new facilities are in turn, pretty simple, and fit nicely on top of the old structure.

I was amused to see that in my previous *Vector* article on Locales in J (Vol.11 No.4), I specifically mentioned two of the new facilities (locale paths and switching locale under program control); and said they could be provided as an extension to the current system, but seemed more trouble than they were worth! But it is just as well that we did not add them then - likely without a compelling reason such as the OOP definition of the new Grid Control, we would have got it wrong!

Now let's see how these new facilities are used in the plot example above:

```
load 'jzplot'
```

This loads the plot definitions from a script file. At the beginning of the script is a command to switch the locale to `jzplot`, and the plot definitions are then

loaded into that locale. In J3, you could load a script into a specific locale (giving the locale name as the left argument to `load`), but in this case, it only makes sense to load `jzplot` into a locale of that name, hence it is better to specify the locale in the script itself.

In common with all new locales, the `jzplot` locale has a path that points to the `z` locale only.

```
a=. conew 'jzplot'
```

This creates a new locale (object), sets its path to point to `jzplot` and then to `z`, and returns the locale reference in variable `a`. Locales created by `conew` have a single definition `COCREATOR`, which identifies the creator locale, but otherwise they are empty.

```
plot__a i.10
```

The name `plot__a` (with 2 underscores) can be read "plot in a", or "plot in indirect a". Informally, we can talk of locale `a`, with the understanding that this is the locale referenced by `a`.

In evaluating the expression `plot__a i.10`, the system searches the locale path, finds the plot definitions in `jzplot`, and executes them in locale `a`. All global definitions created will then be defined in `a`.

Note that the name of the locale created by `conew` has not been used, only its reference. To make this clear, compare this with what you have to do in J3, where the name itself had to be used. First, you would have to create a suitable name for the temporary locale. Then to use it, you would need an execute statement, for example:

<code>a=. 'temp0'</code>	temporary locale name
<code>". 'plot_',a,'_ i.10'</code>	execute to use the locale name

Clearly, the indirect reference is better!

Since, in general, the names generated by `conew` are not used directly, they are chosen to avoid any likely name conflict, and in fact are simply character representations of the integers. Thus in a new session, `a` would be `'0'` and `b` would be `'1'`. We call these *numbered locales*. The numbers are never reused in a session.

Grid Control

The new Grid Control in J4 is for the most part written in J. The display is an *isigraph* control, and there are some extra wd (Window Driver) facilities to enable drawing a grid in the control.

There is a great deal of J code, and it needs to be organized. As with other facilities, it should be defined in a locale to avoid name conflicts. But it also must be easy to extend and customize; and easy to run multiple grids. The new language extensions are a natural for this, and the definition exactly follows the OOP model.

The Grid Control is more complex than Plot, in that there is much greater use of event handlers (e.g. for scrolling and keyboard entry), and we would expect a much greater customization of the basic facilities. To that end, the control is implemented in several classes.

The basic grid class is `jzgrid`, and this provides facilities that should be required for any grid, including:

scrolling

mouse events

keyboard events

setting attributes, e.g. font, colour, border, cell size

While the `jzgrid` class could in theory be used directly (for a very simple grid), there would typically be another class on top of the basic grid class that provides a specific type of grid, for example class `jwgrid`. This would include data-related definitions such as providing formatted data for the grid and support for copying and editing. Also, yet another class is needed for the Windows form definition.

For example, provided with J4 are three specific grids:

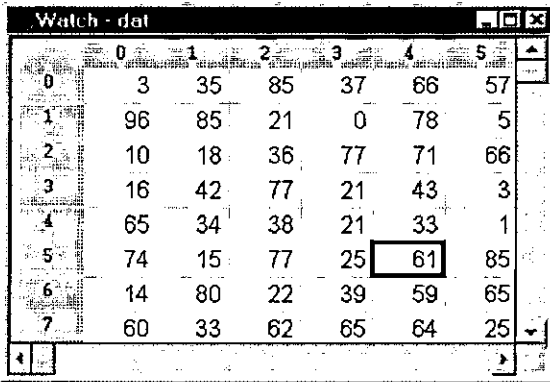
- a Watch Control (class `jwatch`), to display and edit a matrix in a global variable.
- a Table Control (class `jtable`), that displays a function table. This grid is unlimited in size – as the grid is scrolled, new values for the grid are calculated as required.
- a Report Control (class `jfreport`) is a demo form that displays a report, with multi-row headers, and automatic totalling.

The following is a typical session:

```

dat=. 730 30$100          data to be watched
load 'jwatch'             load the jwatch class
a=. conew 'jwatch'        creates a watch object
create__a 'dat'           displays dat in the watch

```



	0	1	2	3	4	5
0	3	35	85	37	66	57
1	96	85	21	0	78	5
2	10	18	36	77	71	66
3	16	42	77	21	43	3
4	65	34	38	21	33	1
5	74	15	77	25	61	85
6	14	80	22	39	59	65
7	60	33	62	65	64	25

Let's examine what happens in detail.

```
load 'jwatch'
```

This loads the script (class) jwatch into the jwatch locale, which in turn loads the scripts jwgrid and jinput, each into their own locale. jwgrid in turn loads the basic grid script, jzgrid. At this stage, all that has happened is that these scripts have been loaded, and the locale namelist looks like:

```

conl''
+-----+-----+-----+-----+
|base|j|jinput|jwatch|jwgrid|jzgrid|z|
+-----+-----+-----+-----+

```

No locale path settings have been made, so each locale has a path of z, as is the case with all new locales.

```
a=. conew 'jwatch'
```

This now creates a new locale, referenced by `a` and named `'0'` :

```
conl''
+-----+-----+-----+-----+
|0|base|j|jinput|jwatch|jwgrid|jzgrid|z|
+-----+-----+-----+-----+
```

The new locale is empty, but has a locale path that includes `jwatch` and `z`.

```
create__a 'dat'
```

`create__a` is called in the new locale, and gets its definition from `jwatch`. This verb creates a new form to display the grid, and then itself calls `conew` to create a `jwgrid` object and sets its path to include `jwgrid` and `jzgrid`:

```
grid=: ''conew'jwgrid'
```

Therefore the locale list now includes two numbered locales:

```
conl''
+-----+-----+-----+-----+
|0|1|base|j|jinput|jwatch|jwgrid|jzgrid|z|
+-----+-----+-----+-----+
```

The paths in these two numbered locales are:

```
copath a                locale '0'
+-----+
|jwatch|z|
+-----+

copath grid__a          locale '1'
+-----+-----+
|jwgrid|jzgrid|z|
+-----+-----+
```

The structure of numbered locales is returned by `costate`:

```
costate''
+-----+-----+-----+-----+
|refs|id|creator|path|
+-----+-----+-----+-----+
|a   |0|base   |jwatch z|
+-----+-----+-----+-----+
|    |1|0      |jwgrid jzgrid z|
+-----+-----+-----+-----+
```

Now suppose the mouse is clicked on one of the grid cells. The form is defined in locale a, so the mouse event handler `watch_grid_mbltdown` will be called there. Since this locale has no event handlers of its own, the definition in the `jwatch` locale is used. This is defined as:

```
watch_grid_mbltdown=: 3 : 'mbltdown__grid sysdata'
```

Therefore the actual handler called is `mbltdown` in the `grid` locale, which will pick its definition up from either `jwgrid` or `jzgrid`, whichever it sees first.

At first sight, these may seem like a lot of mechanism to set up a form with its event handlers. But the point is that each of the classes loaded can be customized independently (except perhaps for `jzgrid` which should work for all grids). Therefore, this provides a simple and effective way of utilizing code modules in a variety of applications, and avoiding name conflicts between two instances of an application.

	1st quarter			2nd quarter			3rd quarter
	Jan	Feb	Mar	Apr	May	June	July
item 0	334	693	144	291	666	854	314
item 1	516	344	264	290	833	124	634
item 2	665	11	794	560	288	415	320
item 3	373	50	73	202	566	831	422
item 4	589	369	463	627	944	405	50
item 5	104	665	833	581	658	982	18
item 6	719	590	837	834	659	730	643
item 7	442	917	276	951	117	416	836
item 8	929	662	243	941	596	126	246
item 9	623	838	671	391	871	667	115

Example Report Grid

OOP is easy in J!

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Six Megabytes in 640K?	Anthony Camacho	112
Technical Correspondence Cannon, Clough, Henriod		117
At Play with J: A Newer Random Link Generator	Gene McDonnell	122
Using OpenGL Graphics in Dyalog APL	Sasha Skomorokhov	128

Six Megabytes in 640K?

by Anthony Camacho

This is unfinished work. I expect you will have to do some experimenting yourself if you need to use these techniques. I would be glad to hear from anyone who has solved more of the problems than I have.

I maintain a membership records system written in APL*PLUS/PC version 8.1. The organisation is growing fast and we have had problems with *WS FULL*, because it searches for a target name in a matrix of all names beginning with the target letter. Would you believe that there are more names beginning with M than any other letter? Anyway the size of the M matrix is about 42K, and it is appended to, which explains why my experiments are done with variables that take up 64K. I want headroom!

The author of the system had also had *WS FULL* problems and introduced a virtual workspace in an attempt to get over them. Because he only had 2 Mbytes of memory and was using a RAM disk to hold the virtual memory he had not explored the limits of the technique.

Although the system is running under Windows 95 it is necessary to have the DOS CONFIG.SYS and AUTOEXEC.BAT set up correctly or you don't get the APL characters. When running under DOS it used to be possible to run VGAFONT.COM but this doesn't work under Windows 95 so I have to resort to the codepage method. Here are my config.sys and autoexec.bat.

Config.sys

```
device=C:\WINDOWS\Himem.Sys
DOS=HIGH,UMB
devicehigh=emm386 noems
installhigh=c:\windows\command\nlsfunc.exe
devicehigh=c:\windows\command\ansi.sys
device=c:\windows\COMMAND\display.sys con=(ega,1)
Country=044,850,c:\windows\COMMAND\country.sys
devicehigh=c:\windows\ramdrive.sys 76800 /E
```

The aim is to use as little of the base 640K memory as possible; even though Windows provides the DOS memory by emulation (so it needn't really steal TSR stuff out of the 640K) it seems to emulate the DOS memory limitations too! The key items here are nlsfunc.exe, display.sys and the country line which are

all essential for the APL font. The APL font supplied with APL*PLUS is an ega-compatible font which explains the display.sys setting.

Autoexec.bat

```
C:\PROGRA~1\MCAFFEE\VIRUSS~1\SCANPM C:\
@IF ERRORLEVEL 1 PAUSE

SET TEMP=C:\WINDOWS\TEMP
SET PROMPT=$p$g
SET winbootdir=C:\WINDOWS
SET PATH=c:\windows;c:\windows\COMMAND;d:\utils
mode con codepage prepare=((850) c:\netapm\aplfont.cpi)
mode con codepage select=850
keyb uk,,c:\windows\COMMAND\keyboard.sys
```

The important lines here are the two lines beginning 'mode'. They provide the APL font. It is essential that the properties of the batch program that starts APL are set to run FULL SCREEN. If you run in a window you won't get the APL font.

APL.bat

```
REM APL.BAT SETS UP APL*PLUS 8 FOR NETWORK
rem amended Feb 1998 by AJC
C:
cd\
CD\NETAPM
lh aplprint

REM A=1 retain video attributes when scrolled off screen
REM | C=0 no comms buffer
REM | F=40 file tie quota (probably not needed under Win 95)
REM | K=U unified keyboard
REM | LIBS=C:\NETAPM where to look for APLLIBS.ASF
REM | M=2000 specifies top of memory (probably not needed)
REM | S=16 sets scrolling screen memory to 16 K bytes
REM | W=APM2 workspace to load
REM | VWS=4c in 4Kb granules allocate
REM | 5.938 Mb of virtual WS and
REM | swap out anything bigger
REM | than 32 bytes
REM | M=2000
APL A=1 C=0 F=40 K=U LIBS=C:\NETAPM S=16 W=apm2 VWS=f:\APL.VWS,1,5938,32
c:
CD\
```

I hope the the file is clear enough not to need explanation, except for the VWS parameter. VWS= sets the virtual memory options. First is the file ID and path, where the virtual workspace is to be held, second is the granule size in KB (which defaults to 1), third is the amount of disk space to reserve in KB (5938 is the maximum value and the manual says '*' would also reserve the maximum value). 32 is the size in bytes of the smallest object that will be written to the virtual

workspace when space is short. According to the manual 32 is the smallest permitted value.

When APL.BAT is double clicked the system should start and, if you have its properties set to full screen you will be able to type quad AV and see all the APL characters. Note that you cannot *maximise* the window to get to full screen. There may also be unexpected consequences of the screen driver; on my portable, which has an 800 by 600 pixel screen, the APL full screen comes up in a 640 by 480 patch in the centre and I guess this is because the portable cannot switch screen modes to vary the size of the pixels.

Now I have a workspace called *HUGE*. It contains so little that you will not suffer much by typing it in. There are two functions and a single variable. Here they are:

```

V A ADD A;ΔΔ;ΔΔΔ
[1] * Adds or replaces variables Δ+A-Z with contents A then next letter
    after Δ, and so on.
[2] ΔΔΔ←~1
[3] NXT:
[4] →(25<ΔΔΔ+ΔΔΔ+1)/END2
[5] ΔΔ←[AV[⌊IO+65+ΔΔΔ]]
[6] ΔΔ,ΔΔ,'+A'
[7] 'ΔWA is ',▽ΔWA
[8] SPACE
[9] →(1024<ΔWA)/NXT
[10] VWSINFO ◊ (Δ,ΔΔ),' added.',⌊TCNL
[11] END:'Please press enter or X enter to end.' ◊ →(¬'X'ε⌊)/NXT
[12] END2:END2 of ADD reached. ',(Δ,ΔΔ),' last variable created.'
[13] ΔΔΔ←~1 ◊ Δ←[AV[1+⌊AV;Δ]] ◊ →NXT
V

```

The right argument started out as a 32750 reshape of a character string. I think anything will do. I believe that some interpreters keep only one copy of variables (e.g. $a+b+c+d+65536\rho 32+[AV]$) which are exactly similar, making separate items only when one is changed. I don't believe that APL*PLUS/PC is one of these. The size was chosen to make the quad-size of the variable 32768.

SPACE is necessary because ΔWA returns the workspace available in the real memory, and if variables and functions are swapped out, their size is not included.

```

V SPACE
[1] * Reports the whole space taken by this WS
[2] 'The size of the workspace (ΔWA plus ⌊SIZE of VARS + FNS) is:
    ',▽(+⌊SIZE ⌊NL 2)+(⌊SIZE ⌊NL 3)+ΔWA
V

```

As you can see 'A'ADD A simply creates copies of A with names 'AA', 'AB', 'AC' and so on, running SPACE and □WA after each to tell you how things are going. The reason for lines 10 and 11 is that I discovered that line 9 sometimes drops through when □WA returns less than 1024 and it may do this well before the virtual workspace is full.

VWSINFO on line 10 is a function from the supplied workspace VIRTMODE and it returns the granule size in Kb, the maximum size of the virtual workspace in Kb, the number of granules, the number of granules free and the minimum size of object that would be swapped out. The VIRTMODE workspace contains several other useful functions, including some to prevent an object from being swapped or to reduce the likelihood of a swap. This dates from the time when PCs were slow. The variables to fill the six megabytes were created and swapped out on the test machine (a Dell D266) in under three seconds!

The value □WA returns seems to vary and I don't know why. I haven't found any programmable way of forcing the garbage collection to run or forcing the system to swap anything out into the virtual workspace.

There is a □POKE (121) which, according to the manual, *forces a garbage collection in the workspace every time the keyboard unlocks for input*. I hoped that it would work without actually insisting on a formal request for input, but it doesn't seem to. Perhaps someone who provided support on this system when it was state-of-the-art will be able to tell us how to force the system to make space.

The system stops with WS FULL when the space used is over six megabytes. Here is a screen dump at the end of a run:

```

□WA is 30176
The size of the workspace (□WA plus □SIZE of VARS + FNS) is: 6074432
END2 of ADD reached. GZ last variable created.
VWS FULL
ADD[6]: HA+A
      ^
      SPACE
The size of the workspace (□WA plus □SIZE of VARS + FNS) is: 6107152
VWSINFO
1 5938 5938 19 32
□WA
62944
Y+16376+A
'J'ADD Y
□WA is 46464
The size of the workspace (□WA plus □SIZE of VARS + FNS) is: 6107152
VWS FULL

```

```

ADD[6]← JB+A
      ^
      VWSINFO
1 5938 5938 17 32
  □WA
63104

```

```

2 HUGE                Unified      Caps      Ins

```

As you see I tried reducing the size of the variable A to 16376 (to make it occupy 16384 bytes) but it only added 'JA' before the workspace was full again. I would have expected that with 17 granules not used and 63Kb of workspace available at least one or two more variables would be possible. Does anyone know why it behaves like this?

Erratum: Windows NT Registry Access

I tried (with some enthusiasm but little success) to use the John Sullivan's code recently published in Vector. There are perhaps other APLers who experienced the same problem as I did, i.e.

- *Reg.Open* would happily create a new key and return a sensible handle, but
- it would fail to open an existing key, returning a handle of 0.

Those readers with the same experience will probably be users of Windows NT, as I am. The answer lies in the access variable. In the original code (Dyadic Systems's function *GetRegKeyHandle*) this was assigned:

```
KEY_ALL_ACCESS ← 2035711
```

I quizzed them about this, and they admitted that this value was incorrect. It works fine on operating systems with inadequate registry security (e.g. Windoze 95 - in fact any old value would probably work) but not with NT. The correct statement should be:

```
KEY_ALL_ACCESS ← 983103
```

With this adjustment, John's code seems to work flawlessly. I hope this assists other confused readers.

Bob Hoekstra

Working with Compressed Data

From: Ray Cannon

April 1998

I developed a Dyalog operator for working with compressed data. I am using a simple Boolean mask to compress out zeros from sparse data. The following function will produce a compressed data structure from a simple numeric vector:

```

v r←Compress vec;bo;values
[1]  a Return vector <vec> in a compressed form
[2]  a vec←(1>r)\2>r
[3]  bo←vec≠0
[4]  values←bo/vec
[5]  r←bo values
v

```

The operator is:

```

v z←la(fn Compressed)ra;lab;lاد;rab;rad;zab;bo
[1]  a Operator to use dyadic fns on compressed data
[2]  lab lad←la      a Split LHarg
[3]  la←' '          a Save space
[4]  rab rad←ra      a Split RHarg
[5]  ra←' '          a Save Space
[6]  zab←lab∨rab      a Create Boolean for result
[7]  lad\~←zab/lab    a Expand LH data to shape of result
[8]  rad\~←zab/rab    a Expand RH data to shape of result
[9]  lad fn←rad      a Apply function
[10] a Note the following is required if zero data
[11] a is to be treated differently from "missing data"
[12] :If -∧/bo+lاد≠0 a Check for zeros in result
[13]     lad/\~←bo    a Compress out zeros from data
[14]     zab∧←zab\bo  a Adjust compression Boolean
[15]     :End
[16] z←zab lad      a Return compressed result
v

```

Lines 7 and 8 could be re-written in ISO standard APL as

```

[7]  lad←(zab/lab)\lad
[8]  rad←(zab/rab)\rad

```

a Example:

```

lad←Compress 0 0 1 0 0 2 0 0 3 0 0 4 0 0 5
ra←Compress 0 1 0 2 0 3 0 4 0 5 0 6 0 7 0
la + Compressed ra
0 1 1 1 0 1 0 1 1 1 0 1 1      1 1 2 5 4 3 5 10 7 5
0 1 1 1 0 1 0 1 1 1 0 1 0 1 1 \ 1 1 2 5 4 3 5 10 7 5
0 1 1 2 0 5 0 4 3 5 0 10 0 7 5

```

Niven Number Computing Speed

From: Eddie Clough

29 March 1998

In his interesting article in *Vector* Vol.14 No.2, pp 121-124, Josef De Kerf notes that, so far, no algorithm has been found to calculate a given Niven number $F(N)$ from N . I may of course be wrong but this seems to me to be a problem comparable with that of finding an algorithm to compute the N th prime; i.e. there may little chance of eventual success.

A partial answer to the question of finding a faster algorithm may be to note the ease with which sums of digits can be found *en bloc*, so to speak, by repeated application of sum tables. Using J3 on a (Pentium 233) computer with 64MB RAM, I can find the digit sums for the first million integers in less than half a second. With these data, the Niven numbers can be identified in about 3 seconds. Beyond one million, the number of numbers being handled becomes so large that the swap file plays an increasing role, slowing the process down, leading to 'out of memory' for 10^8 integers. However, the Niven numbers in the second, third, etc. million can be found equally quickly by adding 1, 2, etc. (or their digit sums) to the digit sums found for the first million. In this way the Niven numbers less than 10^{12} can be investigated. There are in fact 27975 Niven numbers in the millionth group of millions of integers, the largest being 999999999903. (I can't tell you which position it occupies in the complete Niven sequence.)

In the following realisation, the J verb, *niv*, may be used monadically to produce the digit sums for 10^y integers (including zero) and then dyadically to find the Niven numbers among the $x \cdot 10^y$ to $(x+1) \cdot 10^y - 1$ integers, where x and y are respectively left and right arguments of the verb. If y is set to 6, the integers are explored in groups of one million. If 'n' successive millions are analysed by one J statement, the verb reports, for each successive group, the total number of integers examined up to that point, the number of Nivens in the group, the Nivens in the group (recorded for later interrogation in a vector *FN* but overwritten by the Nivens in the next group, if there is one), and the total number of (overwritten) Nivens in the previous groups, this last being given to enable an 'N' to be associated with each finally recorded 'FN'. The example provided shows the analysis of 10 successive groups each of 10^6 integers; analysis time is proportional to the number of groups and this analysis took 30 seconds. (If a smaller group size is used, e.g. 10^5 integers, the time per group falls so that there is very little difference in the overall time to examine the same total number of integers — but the number of Nivens stored for interrogation falls.)

[In another article, *Vector* Vol.14 No.3, pp 138-142, De Kerf looks at Armstrong numbers. The above approach can be adapted to study this domain. The analysis is quite straightforward as far as the first group of integers is concerned, but beyond that matters become slightly more complex. For example, if the sums of 8th powers of the digits of the six-figure integers up to 10^6 were found, they could only be used as a basis for finding the Armstrong numbers in the 10th to the 99th million groups, i.e. for the addition of two and only two digits.]

```

a=.i.10      [   n:=0
int=.<.=]      NB. verb: integer test ( yields 0 or 1 )
sumt=.+./      NB. verb: Sum table
f=.,@sumt&a    NB. verb: Ravelled table of sum of a and ?
niv=.3 : 0
y=.<:y.        NB. y is y. minus 1
#k=: 1.#s=: f^:y a  NB. Repeated application of f on a
:
S=.s+x.(s      NB. Add x.th s to each s
K=.k+x.*#k      NB. Add x. tallies of s to each k
FN=:K#~int K%S  NB. Niven numbers in the x.th group
D=:n            NB. D is total No. of Nivens overwritten
n=:n+FN
z=:(_1{K),D,(#,_1&{)FN  NB. 4-vector:
                        NB. (1)Total No. of integers exam'd, (2) D,
                        NB. (3,4) No. of Nivens and largest in final group
)

o!:'0<'c:\j305c\temp\niven.js'
niv 6
1000000
(1.10)niv"0 0 y=.6

```

-----Niven Numbers-----

Integers Examined	Overwritten	In this group	Largest
999999	0	95428	999990
1999999	95428	91133	1999998
2999999	186561	87403	2999970
3999999	273964	83959	3999996
4999999	357923	80872	4999968
5999999	438795	78193	5999994
6999999	516988	75678	6999966
7999999	592666	73361	7999992
8999999	666027	71069	8999984
9999999	737096	68999	9999990

E.A.Clough
 'Conifers', Leighton Beck Road
 Slack Head, Beetham
 Milnthorpe
 Cumbria LA7 7AZ

More on Niven Numbers

From: Claude Henriod

31 March 1998

I am not going to repeat here what a NIVEN number is as I also did not know its meaning (see the articles in *Vector* Vol. 14.2 & 14.3). I am, on the other hand, astonished that Joseph De Kerf should show a function with scalar loops. It must surely be a copy from a version in C or Fortran. In fact, one of the best qualities of APL is handling in bulk (here a vector) which, by contrast, gives a bad performance in other cases. It is not thus a question of finding a new mathematical trick (algorithm) but simply of understanding the APL language and applying it.

Example: the last character of a vector is not:

```
max←(+pcandidates)÷candidates
```

but simply

```
max←~1÷candidates
```

I have chosen a solution which differs a little from those of Joseph De Kerf and David Piper. The argument of the function is not the n^{th} NIVEN but the tranche which terminates:

```
z NIVEN_NO
```

You will see I have an example in g) *NIVEN_E*

Description and/or examples of each function in the logical sequence of development (the dossier is a single session log in direct execution mode):

- Presented in IBM-APL2 (ver. 1.02/PS2/DOS 7.0)
 - Similar results in IBM APL2/2 (OS2 ver .1 service level 11)
- a) *NIVEN_X* simple vector; corresponds to *NIVENE* by David or $A=W$
 - b) *NIVEN_Z* recursive form in diminishing tranches (if N is large)
 - c) *NIVEN_L* same function but in loops by tranche (DO_UNTIL)
 - d) *NIVEN_V* question "Is it a Niven?" with possible limits
 - e) *NIVEN_B* list of Niven numbers in another base (2 to 10)

- f) *NIVEN_H* Niven numbers to base 16 in hexadecimal notation
- g) *NIVEN_E* Similar to *NIVEN_L* (external loop) for *EACH* tranche

I have saved this variant to the last to show you what it allows me to see about the progressive distribution of Niven numbers. In fact if one takes the intervals between the numbers the difference is not significant. On the other hand we can see that the ratio between one tranche and another is constant (see n00000). It is now up to you to extract from this a new theory in order to receive the APL Nobel prize!

- h) *P_NIVENE*

To complete the account of the problem I have added my version of David's code, as well as an execution of David's *P_NIVENE*, for those of you who find that my code is "Write Only".

- i) & j) Timing tables of results for N=10,000 and N=100,000.
- k) *XTIME* A simple timing routine (without graphics).

All this material is available on disk:

2 WinWord documents of about 30K

1 listing .log (APL2 - PS2, Editor 3) of 10K

2 workspaces .atf & .ws of approx. 100 K each

Please send 5 1/4" disk (either capacity) or 3 1/2" disk (either capacity) to:

Claude Henriod
Rte de Monteiller
CH-1965 Savièse
Switzerland

with your return address

At Play With J: A Newer Random Link Generator

by Eugene McDonnell

The random link generator used in many J\APL systems is an example of the linear congruential kind introduced in 1948 by the mathematician D. H. Lehmer, of the University of California in Berkeley, California. It depends on two numbers, 16,807 and 2,147,483,647. The first is 7^5 , a primitive root of the second, which is Euler's prime, $<: 2^31$. The basis of the algorithms for the roll and deal primitives is the expression for computing the next link $r1$ in the chain of random links:

```
r1 =: 2147483647 | 16807 * r1
```

The chain is 2,147,483,646 links long, and contains all of the integers between 1 and 2147483646, but not in any easily discernible order. The first link in the chain is 16807, and the next few links are

```
282475249 1622650073 984943658 1144108930 470211272
```

The values seem to be suitably random. What's more, they satisfy many of the tests mathematicians and computer scientists have devised for judging randomness.

This column discusses a generator of a very different kind, that has its roots in a method devised in 1958 by G. J. Mitchell and D. P. Moore. Donald Knuth's book *Seminumerical Algorithms* had its first edition in 1969 (the year I bought my copy) and discusses random numbers extensively, but doesn't mention this method. It appears in the book's second edition (1981), but is treated warily. Knuth writes, "The only reason it is difficult to recommend [it] wholeheartedly is that there is still very little theory to prove that it does or does not have desirable randomness properties". By the time of the third edition (1998) it is obvious that a great deal of effort has been expended on it, and it is almost part of the canon. I first saw the technique described in Knuth's book, *The Stanford Graph Base*, Addison Wesley, Reading, Massachusetts, 1993. He describes it in the section called GB_FLIP, the name of the program used to generate random links for the other programs described in the book. The generator described here comes from this book.

The basis of the generator is a bit of mathematics the gist of which Knuth gives as follows:

The subtractive method. If m is any even number and if the numbers $a_0, a_1, \dots, a_{54}$ are not all even, then the numbers generated by the recurrence

$$a_n = (a_{n-24} - a_{n-55}) \bmod m \quad (1)$$

have a period of at least $2^{55}-1$, because the residues $a_n \bmod 2$ have a period of this length. Furthermore, the numbers 24 and 55 in this recurrence are sufficiently large that deficiencies in randomness due to the simplicity of the recurrence are negligible in most applications.

If something has a period of n , it means that it can't be made up as an integral number of repetitions of a smaller sequence. Knuth says that $2^{55}-1$ is the smallest the period can be, but it is plausible that it is $2^{85}-2^{30}$. Furthermore, the low-order bits of the generated numbers are just as random as the high-order bits. These are both very large numbers.

```
<: 2^55x
36028797018963967

(2^85x) - 2^30x
38685626227668132516855808
```

There is a usage problem in giving a name to them. In the UK the names of numbers go up in powers of a million. Thus billion, trillion, and quadrillion are a million to the second, third, and fourth power. In the US they go up in powers of a thousand, so million, billion, trillion, quadrillion are a thousand to the second, third, fourth, and fifth powers. The names are more appropriate in the UK usage, but the size of the denominating numbers are smaller and usually more convenient in the US usage. The first large number would be about 36029 billion in UK usage, and 36 quadrillion in US usage.

36028,797018,963967 (UK) 36,028,797,018,963,967 (US)

The larger number would be about 39 quadrillion in the UK and 39 septillion in the US. Whichever way you call it, it's big.

38,685626,227668,132516,855808 (UK) 38,685,626,227,668,132,516,855,808 (US)

Since there are only $<: 2^{31}$, or 2,147,483,646 positive integers, it is clear that each integer will appear many times in the chain, but the period of the chain is indeed enormously long.

Here's how the generator works: if a random link is needed, select the one indexed by `gb_fptr`, from a list `A` of 55 random numbers, and subtract one from `gb_fptr`. Thus getting a random link requires trivial computation, an indexing and a subtract, simply

```
gb_fptr { A
gb_fptr =: <: gb_fptr
```

There must be more to it than that, you say, and you are right. The tricky problems are how to get more random links when we've used all that are in the list, and, ultimately, where did the initial list come from? The processes involve a fair amount of crabwise sidling and snakewise slithering. I'll make it as clear as I can.

What does one do when all 55 links in the list have been selected? A program `gb_flip_cycle` is used to provide a new list of 55 links, guided by recurrence (1). Knuth's C program uses two successive `for` loops; the first subtracts in sequence the last 24 items of `A` from the first 24; the second subtracts the last 31 from the first 31 in sequence, depending on the explicit sequencing in the `for` loop specification to make sure that the overlap of 7 items in the middle of the two has the proper values each time it is traversed. We could do that, too, using the shiny new control words available in J, but prefer instead to emphasize the potential for parallelization by using the following scheme: subtract the last 24 from the first 24, then the new first 24 from the second 24, and lastly the new 7 following the first 24 from the last 7.

All these subtractions are mod m , and this is easily computed at the machine level by anding the result of subtraction with `16b7fffff`. In the function below it is shown at the J language level by taking the residue mod `16b8000000`, yielding the same result.

The function `gb_flip_cycle` takes as argument a list of 55 random numbers, and yields a new such list. In terms of recursion (1) its first use replaces a_0 through a_{54} by a_{55} through a_{109} . Each of `a`, `b`, and `c` is a two-row matrix of indices to the list. For example, `c` is the matrix

```
48 49 50 51 52 53 54
24 25 26 27 28 29 30
```

The first row gives the indices of the minuend, which are also the indices of the result. The second row gives the indices of the subtrahend. For example, the amend `c g` } subtracts a_{79} through a_{85} from a_{48} through a_{54} , yielding a_{103} through a_{109} , and these replace a_{48} through a_{54} .

```

gb_flip_cycle=: monad define
a=. 0 31 +/ i. 24 NB. 1st 24 get (1st 24) - (last 24)
b=. 24 0 +/ i. 24 NB. 2nd 24 get (2nd 24) - (new 1st 24)
c=. 48 24 +/ i. 7 NB. last 7 get (last 7) - (7 after 1st 24)
NB. v0 is difference mod m of selected parts of argument
v0=: 16b800000000"-![: -/ {
NB. v1 is top row of 2-row left argument -- result address
v1=: [: {. [
NB. v2 is right argument: a 55 item list
v2=: ]
g=. v0`v1`v2 NB. gerund for amend
c g } b g } a g } y. NB. amend in three overlapping steps
)

```

If you look closely at the numbers, you'll notice that recurrence (1) isn't faithfully followed. Instead we have

$$a_n = (a_{n-55} - a_{n-24}) \bmod m \quad (2)$$

Knuth claims that this doesn't make the results any less random.

You are probably asking, "Yes, but where did you get the original list of random links?" This is the part that happens only once, and that's good, because it is the most complicated and time-consuming part of the whole method.

In outline, a next random link is formed from three values: the current value *next*, the previous value *prev*, and the *seed*. The current value is subtracted from the previous value, and the modified seed is subtracted from this, mod *m*.

The random links are stored as they are developed in list A, but not sequentially. The first one is stored in the 21st position, index 20. The next number found will be stored 21 to the right, in position 41. And the next one 21 further to the right, in position 62. Oops. That addition is made mod 55, so it will actually be stored in position 7. And so on. There's a trifle of Fibonacci magic here: there are 34 positions to the right of the first one, and 55 in all, and the numbers 21, 34, and 55 are consecutive Fibonacci numbers. Also, 21 is relatively prime to 55, which means that the index will take on all values from 0 to 54, inclusive, once each. Knuth comments that the successive values are thus stored as far as possible from each other, and that the initializing would be rather poor if this dispersal were not done.

The value of *seed* is not constant; it can be any one of 31 possible values. For each of the initial list of random links, a new value of *seed* is obtained by rotating its 31 rightmost bits one position to the right.

```
bwr=: [: #. _1: |. (31$2)□_ #: ] NB, rotate bits 1-31 to the right
```

```
seed=: bwr seed
```

and then performing

```
temp=: next
next=. m | prev - next + seed
prev=: temp
```

The seed used by Knuth is $_{314159} \text{ mod } m$, or 2147169489. This is also the initial value of *prev*. The initial value of *next* is 1, chosen because the method requires that there be at least one odd number in the first set of links.

```
gb_init_randx=: monad define    NB. initial list of random numbers
A=. 55 # 0
m=: 2^31
seed=. m | y.
next=. 1
prev=. seed
x=. 20
whilst. i -: 20 do.
  A=. next x } A
  seed=. bwr seed
  temp=. next
  next=. m | prev - next + seed
  prev=. temp
  x=. 55 | i=. i + 21
end.
A
)
```

To verify that A has been properly formed, you can look at the first few links formed. These should be:

```
A=: gb_init_randx _314159
20 41 7 28 { A
1 2147326568 1073977445 536517481
```

Knuth explains that the first set isn't random enough to be used. Once the sequence gets far enough from its beginning, however, the initial transients become less perceptible. Thus after the initial A is formed, in order to ensure the necessary degree of randomness, a new list is formed from it by using *gb_flip_cycle* five times.

When this is done, the list is ready to be used. The complete initializing procedure is thus

```

    gb_init_rand=:monad define
    gb_fptr =: _2
    A =: gb_flip_cycle^:5 gb_init_randx y.
    )

```

The initial value of A is obtained by

```
A=: gb_init_rand _314159
```

A link in the random link chain is obtained using the gb_next_rand function.

```

    gb_next_rand=: dyad define
    if. gb_fptr > _56 do.
        z=: gb_fptr { A
        gb_fptr =: <: gb_fptr
    else.
        A =: gb_flip_cycle A
        z =: _1 { A
        gb_fptr =: _2 end.
    z
    )

```

The links are accessed using the index value gb_fptr, and in reverse order. This doesn't affect the randomness of the links. Thus the first link taken would be the one at index position 54, which may be accessed (using contracurrent indexing) with gb_fptr set to _1, and then subtracting 1 from gb_fptr. The links are taken from the current list of links in the if clause until gb_fptr reaches _56, at which time the else clause is used to form a new list of links. Item _1 of this new list is returned, and gb_fptr is set to _2.

To verify that initializing has been done correctly, check the result of gb_next_rand:

```

    x: gb_next_rand ''
119318998

```

Unlike the linear congruential generators, subtractive generators are much less choosy about the seed value used. I believe that any value with a generous balance of 1s and 0s will do. Thus this one method gives rise to an enormous number of random link chains, each enormously long, obtainable simply by varying the seed.

Using OpenGL Graphics in Dyalog APL

by Alexander Skomorokhov and Alexei Zalivine

Introduction

Many APL people know about OpenGL from the delightful presentation made by Chris Burke at the APL97 conference in Toronto. At least the authors focussed on OpenGL after this event. There are many good things about the Open Graphics Library:

- It is open, cross-platform and free
- It is the *de facto* standard
- Many manufacture are adding hardware support of OpenGL
- It is a powerful and flexible way to produce complex 3D graphics

You may refer to Vector publications [1-2] and the book [3] for more details on OpenGL in general. Many impressive applications of OpenGL are mentioned there, including Hollywood movies. The authors are far away from an interest in the animation of dinosaurs. Our interest in graphics is limited to scientific and financial data visualization, as a part of complex data analysis. Why did we touch such a monster as OpenGL? It was a kind of bet. One friend of Alexander Skomorokhov, a Dyalog APL user, told him after the conference in Toronto that he was going to learn J. Why? Because of OpenGL support in J. The answer was "it must be easy to do in Dyalog APL" and it was the start of a "project".

We won the bet. But looking back we think it is better to allow people to learn J.

What is bad about OpenGL from the APL programmer's point of view?

- You should deal with many (about 150) OpenGL calls stored as functions in Dynamic Link Libraries
- You should pass proper APL arrays as OpenGL function arguments, described as C language data types
- You should provide a low-level connection between OpenGL and your Windows application written in APL

After Dyadic Systems put our DEMOGL workspace on their web site we received many responses from APL users. That shows the interest in powerful graphics

tools in the APL world and makes some sense of our work with OpenGL. The main purpose of this paper is to show how to use OpenGL from Dyalog APL and to simplify this for APL programmers.

Getting Started or Stones to Build a House

OpenGL includes two Dynamic Links Libraries: *opengl32.dll* and *glu32.dll*. The direct access to the OpenGL functions from APL uses the system function name association. We implemented an interface to OpenGL DLL functions by writing a cover function for each of them. The following general conventions were used:

The name of the APL cover function exactly matches the name of the OpenGL function it calls

- The name of the associated OpenGL function is a composition of its original name plus the suffix *_DLL*
- Name association is conditional and is performed only when the cover function is called for the first time

These conventions make the OpenGL part of the APL code very similar to the C code used in most books and papers on OpenGL. Well, it is relative pleasure to see C code. But you can just copy it from a book to the APL session and that is helpful.

Some of the OpenGL functions require a pointer to data as an argument. In this case the corresponding APL parameter is given as an enclosed array of rank 0. Many arguments of OpenGL functions are passed as predefined constants with fixed names. Using these constants makes the look of the APL cover functions even closer to the C syntax. We prepared a whole bunch of such constants to be used from APL.

All OpenGL functions and constants are stored in the namespace *gl*.

Raw GL-programming for APLers

Preparing cover functions and name association of OpenGL commands and all constants is a large piece of work. But all this stuff can not be used directly. OpenGL is a so-called "rendering" system. This means that a programmer should provide "a place" for drawing, and a connection between OpenGL and this place. After you "explain" to OpenGL where to put graphics output, you may start the painting itself. This explanation depends on the platform (if you use a Silicon Graphics workstation or Windows on a PC) and on the language you use. These

connections were made so as to provide portability, but they add headache if you want work just under Windows, for instance. In this section we will try to hide the difficulties and to allow as easy a use of the OpenGL calls as possible.

Let us start from the OpenGL minimum:

```

)OPS
OpenGL
)FNS
Cylinder      GL_Close      GL_Configure  GL_Expose  GL2GUI Sphere
)VARs
)OBS
gl

```

All building blocks, such as APL cover functions to associate OpenGL functions stored in the DLLs and all the predefined OpenGL constants, are stored in namespace *gl*. The main thing in the root is the user-defined operator *OpenGL*:

```

vOpenGL[[]]v
[0] obj(GL_Paint OpenGL)par;wgl;conf
[1] wgl+GL2GUI obj
[2] obj [WS'Event' 'Close' 'GL_Close'wgl
[3] obj [WS'Event' 'Configure' 'GL_Configure'
[4] obj [WS'Event' 'Expose' 'GL_Expose'(1>wgl)
[5] GL_Paint par
[6] conf+ε2+''obj [WG'Posn' 'Size'
[7] [NQ(obj'Configure'),conf
[8] (1>wgl)GL_Expose 0

```

The purpose of this operator is to provide communication between an APL GUI object, given as a left argument *obj*, and OpenGL functions. The operand *GL_Paint* is an APL function, which contains the "pure" OpenGL code you would like to execute to produce an image on a given APL GUI object. Such a communication needs an identification of the GUI object, such as a window handle and descriptor. These low-level calls to the Windows and OpenGL DLLs are hidden in the function *GL2GUI*:

```

vGL2GUI[[]]v
[0] z+GL2GUI obj;handle;DC;hrc;PFD;IPX
[1] handle+obj [WG'Handle'
[2] [CS'gl'
[3] 'wglCreateContext_DLL'[NA'U4 OpenGL32.DLL.C32|wglCreateContext U4'
[4] PFD+40 1 37 3,24,(13p0),32 0 0 0 0 0 0
[5] DC+GetDC handle
[6] IPX+ChoosePixelFormat DC PFD
[7] SetPixelFormat DC IPX PFD

```

```

[8]  hrc+wglCreateContext DC
[9]  wglMakeCurrent DC hrc
[10] z+handle DC hrc
[11] []CS'#'

```

It is necessary to define OpenGL behaviour on three main (from the OpenGL view) events: *Expose*, *Configure* and *Close*. Callbacks to these events are assigned in lines [2-4] of the operator *OpenGL* and are listed below.

```

      vGL_Close[[]]v
[0]  arg GL_Close msg;handle;hrc;DC
[1]  handle hrc DC+arg
[2]  []CS'gl'
[3]  wglMakeCurrent 0 0
[4]  wglDeleteContext hrc
[5]  ReleaseDC handle DC
[6]  []CS'#'

      vGL_Configure[[]]v
[0]  GL_Configure msg
[1]  []CS'gl'
[2]  glMatrixMode GL_PROJECTION
[3]  glLoadIdentity
[4]  glViewport 0 0,msg[5 4]
[5]  []CS'#'

      vGL_Expose[[]]v
[0]  DC GL_Expose msg
[1]  []CS'gl'
[2]  glClearColor 1 1 1 1
[3]  glClear GL_COLOR_BUFFER_BIT+GL_DEPTH_BUFFER_BIT
[4]  glMatrixMode GL_MODELVIEW
[5]  glLoadIdentity
[6]  glCallList 1
[7]  SwapBuffers DC
[8]  []CS'#'

```

Callback *GL_Close*, for instance, cleans up and releases the resources used to provide OpenGL-to-GUI object connection on window close. If you do not process this event you waste Windows memory.

The main operator *OpenGL*, the three callbacks listed above and the function *GL2GUI* do not depend on:

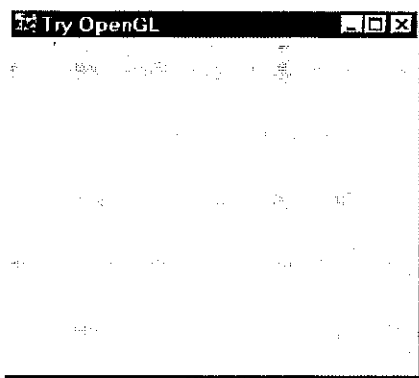
- the APL GUI object you would like to paint on and
- the OpenGL commands you would like to execute

All together they encapsulate the system requirements of using OpenGL from APL and allow "pure" OpenGL programming in an APL session.

We are ready to start. Let us create a container form to do some OpenGL painting:

```
'F'⎕WC'Form' 'Try OpenGL'('Size' 230 280)('Coord' 'Pixel')
```

Now we have a form, not very impressive, but clean enough to paint on:



We have an executable system (operator *OpenGL*), we have a place to draw on (Form *F*), but we have not yet decided what to draw. It may be a sphere. An APL (or OpenGL?) function to create the image of a sphere is shown below:

```
[0] Sphere par;sphere
[1] ⎕CS'#.gl'
[2] glNewList 1 GL_COMPILE
[3] glColor3f 0.8 0 0
[4] glRotatef 45 1 1 1
[5] sphere+gluNewQuadric
[6] gluQuadricDrawStyle sphere GLU_LINE
[7] gluSphere sphere,par
[8] gluDeleteQuadric sphere
[9] glEndList
[10] ⎕CS'#'
```

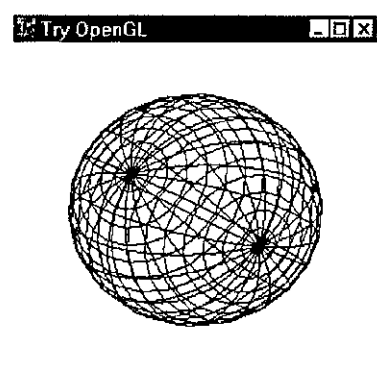
In line [1] we change space to *gl* to access the OpenGL functions (and return in line [10] after the job is done). Lines [2] and [9] are service lines and should be used in all other functions. They open and close a list of OpenGL commands to be executed to create an image. Lines [3-8] are the main content to draw a sphere. Remember that OpenGL is a procedural, rather than a descriptive graphics language. We describe what should be drawn and how an image should look step

by step. Lines [3-4], for instance, describe RGB colour (Red, Green, Blue intensity, ranged from 0 to 1) and rotation around a vector (1,1,1) with angle of rotation equal to 45°. Other parameters of the image are taken from the right argument of the function. It is a numeric vector of: radius of the sphere, slices (number of lines of longitude on the sphere) and stacks (number of lines of latitude on the sphere).

Well, enough on OpenGL. Let us draw on Form 'F' a Sphere using OpenGL calls with parameters 0.7, 20 and 20. This English is directly translated to APL as:

```
'F' Sphere OpenGL 0.7 20 20
```

And our form view is now changed to:



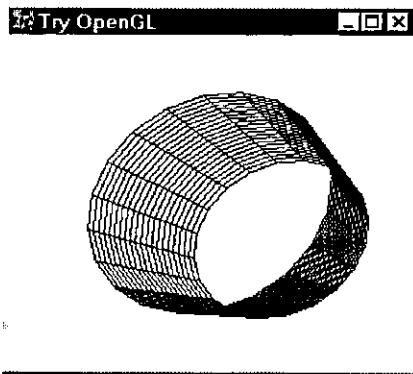
If you would like to draw a "cylinder" you should prepare in the Dyalog APL function-editor an OpenGL script like:

```
[0] Cylinder par;obj
[1] [CS'#.gl'
[2] glNewList 1 GL_COMPILE
[3] glColor3f 0 0 1
[4] glRotatef 45 1 1 1
[5] obj←gluNewQuadric
[6] gluQuadricDrawStyle obj GLU_LINE
[7] gluCylinder obj,par
[8] gluDeleteQuadric obj
[9] glEndList
[10] [CS'#'
```

And ask APL to draw it on Form 'F' as:

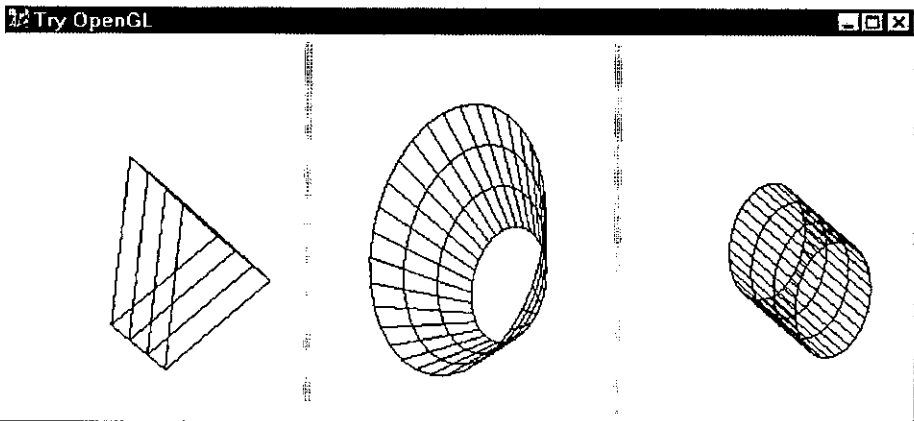
```
'F' Cylinder OpenGL 0.7 .5 .7 20 20
```

The right argument above defines base radius, top radius, height, slices and stacks. You will get the image:



You may even create in APL several GUI objects, like forms or subforms of the same form, and draw OpenGL on each object separately, as illustrated below:

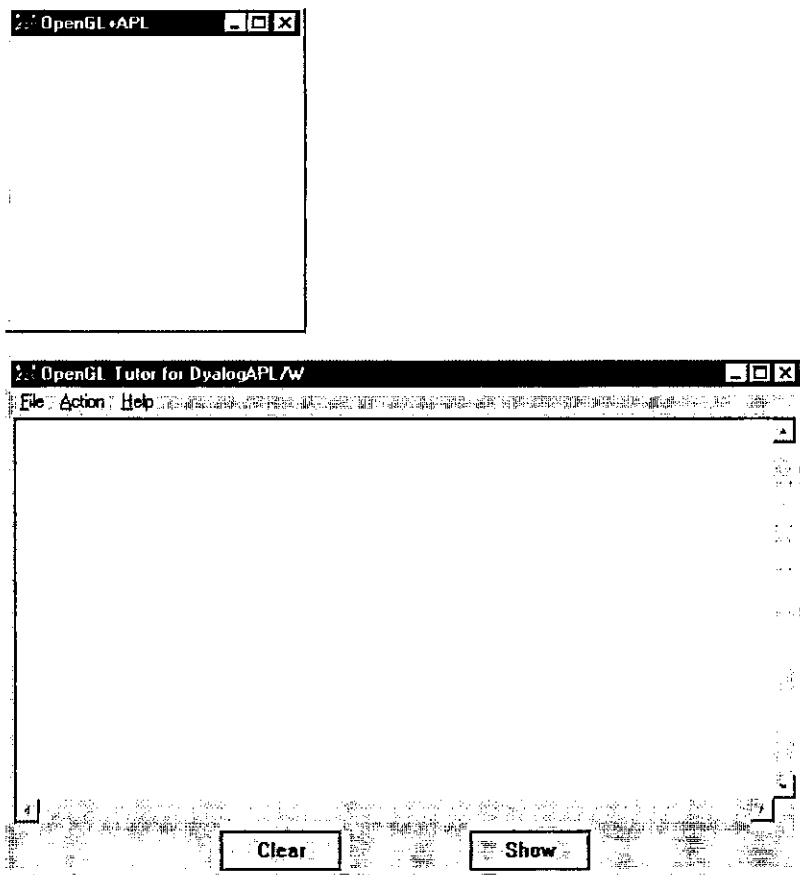
```
'F'⎕WC'Form' 'Try OpenGL'('Size' 35 60)
'F.S1'⎕WC'SubForm' ''('Posn' 0 0)('Size' 100 33)
'F.S2'⎕WC'SubForm' ''('Posn' 0 34)('Size' 100 33)
'F.S3'⎕WC'SubForm' ''('Posn' 0 68)('Size' 100 32)
'F'⎕WS'Coord' 'Pixel'
'F.S1' Cylinder OpenGL 0.5 0.5 0.7 3 3
'F.S2' Cylinder OpenGL 0.7 0.3 0.7 33 3
'F.S3' Cylinder OpenGL 0.3 0.3 0.9 33 3
```



This way allows you to create a multi-window graphical application. It also helps you to see and compare the influence on an image, of different OpenGL settings and parameter values. We will tell more on this in the next section.

Learn GL-basics With the Help of APL

To assist APL programmers to learn OpenGL comfortably we have implemented a GUI application called *GL Tutor*. On startup you get the two forms:



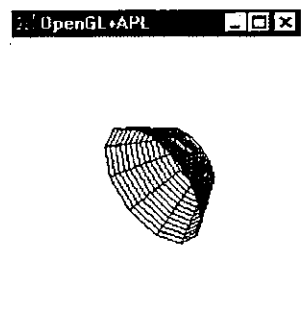
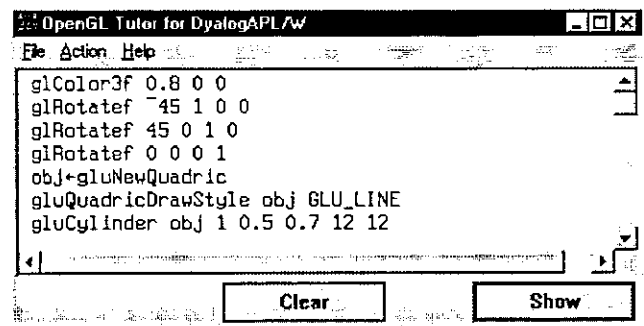
The small one is a container for OpenGL images. The larger one is the work place of an OpenGL student. The main white area is a multiline edit field, where you may enter and edit your "pure" OpenGL code, like the code to draw a cylinder

from the previous section. The application takes care of the OpenGL-to-APL interface. In some sense it is a GUI implementation of the operator *OpenGL* from the previous section.

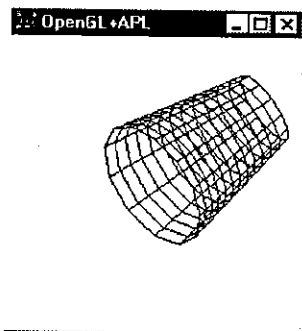
The *File* menu contains a list of prepared examples from the popular book "*OpenGL Superbible*" [3]. Original examples are listed for using by C programmers. We have adapted them for Dyalog APL, but you can see that the texts are very close and the difference is mostly about setting function arguments (representation of C data types in APL). You may select an example using the *File* menu, or enter your own.

The *Action* menu provides the possibility of generating events, like Create, Expose, Configure, Keys, Paint and Close. The *Show* button performs the execution of OpenGL code and creates an image in the OpenGL container form. The *Clear* button does the same, but it clears the previous content of the graphics window before execution. You may also rotate the image in the graphics window using the cursor keys, or move the image around using *Shift*+cursor keys.

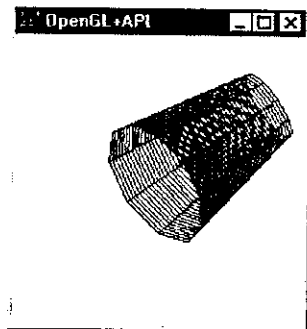
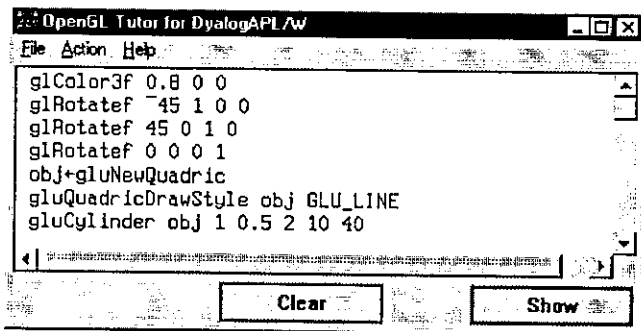
Let us select *Example2* from the *File* menu and press the *Show* button:



Now edit line `gluCylinder obj 1 .5 2 12 12` to change the height of the cylinder from 0.7 to 2.



Now change the number of filling polygon lines from 12 12 to 10 40 and see how it affects the image:



We like the fact that APL is an interactive language (well, not always). Now we have an interactive OpenGL as well.

OpenGL Function Designer

Why use GL Designer?

Raw OpenGL programming with the use of a bunch of cover functions to call appropriate DLL functions allows you to use all the power and flexibility of OpenGL. But it costs too much effort to learn OpenGL and it leads to low productivity of programming.

To avoid these difficulties you can develop utility functions. That allows you to hide most of the OpenGL context and to use high-level calls. It is possible at the expense of flexibility. Many parameters and object properties should take some default values in this case. The rest of the parameters and properties are still a valuable set. The user should understand the meaning and influence of each argument, or spend much time experimenting with different function calls.

The main idea of OpenGL function design under GUI control is to combine the advantages of both approaches. The purposes of GL Designer are:

1. Make it easy to try and select parameters and properties of a GL image
2. Automatically create auxiliary functions for further independent usage

What is GL Designer?

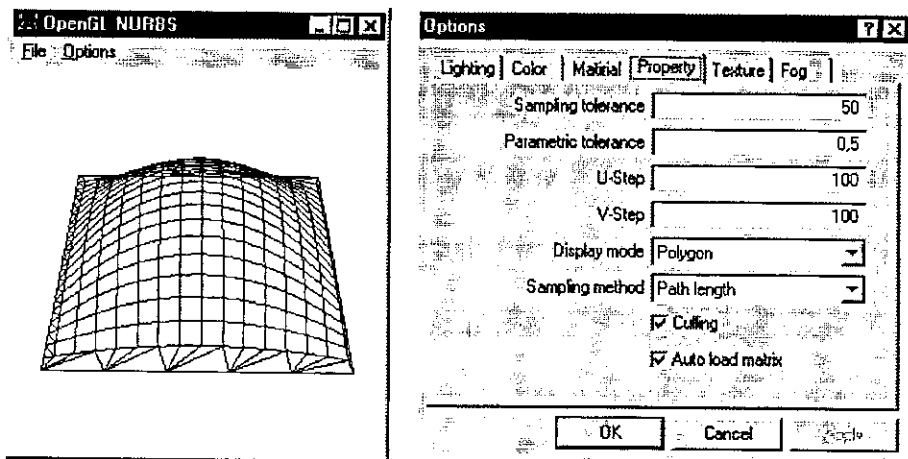
GL Designer is an APL function (and probably a number of subfunctions), separate for each class of OpenGL tasks, like drawing a surface. The *Designer* function uses a minimum number of arguments, first of all the data to be drawn. When you call the Designer function you get two main forms, one a container for an OpenGL image and the other a form to define and select parameters. After any change of parameters you may see their influence by selecting the *Apply* button. When you finish the definition of parameters you select the *OK* button and the resulting APL code is generated automatically.

An example of using GL Designer

Here we describe using GL Designer oriented to build GL-objects, like NURBS-type surfaces (Non Uniform Rational B-Spline). This task is widespread in scientific data visualisation. The call of the Designer function is as follows:

```
Designer arg1 arg2 arg3
```

Where the arguments are: array of control points, array of texture and array of colours. The rest of the parameters take their default values. The user gets two main forms shown below:



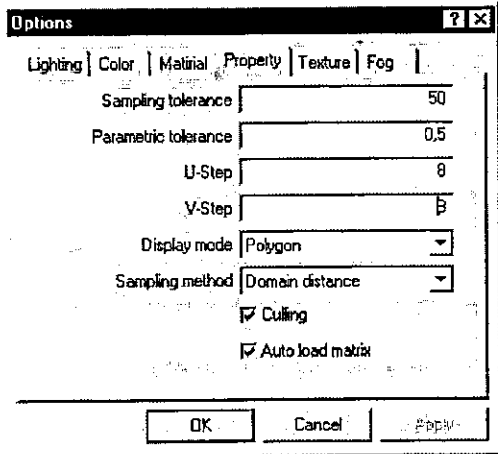
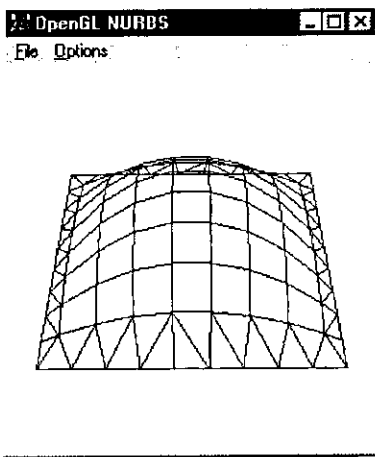
The small form is the OpenGL image container. It shows a surface using a mesh of points data which you passed as an argument, and default values of the other parameters.

The larger form stands for the designer itself. You can see six tabs, which group the different sets of surface parameters and properties as follows:

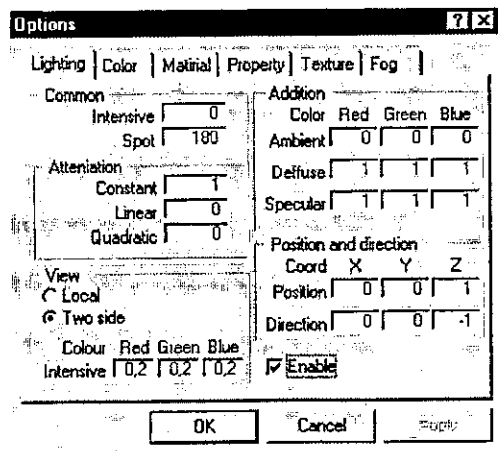
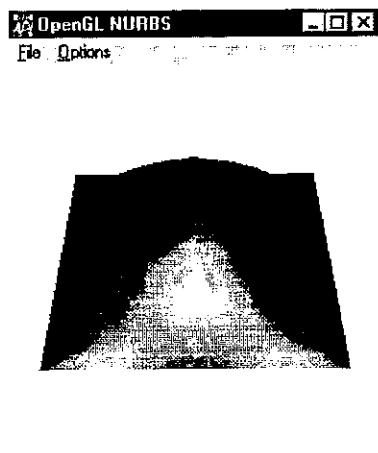
1. Lighting (intensity, angle of scatter of the source of light, point of view and location of watcher, diffused, dispersed or reflected light etc.)
2. Colour, and a possibility of governing the colour of an object and a graphic
3. Material (characteristics such as surface, degree of reflection, etc.)
4. Properties (style of nets and outlines, distances between net nodes, etc.)
5. Texture (level of details of a texture, style of texture imposition, etc.)
6. Effect of Mist (or even fog, type of diffusing a colour, density and other effects).

The next figures illustrate some effects of playing with the property and lighting parameters. The default surface is built in accordance with given data. The display mode is "Polygon" and the sampling method is "Path length" (net with shown numeric parameters).

Now let us change the polygon filling parameters (U-Step and V-step):



To finish this demonstration set Display mode to "Fill", and allow lighting (CheckButton "Enable" is on):



Generating APL functions

After a developer has got a surface, satisfying the requirements, all necessary functions are generated automatically on leaving the Designer. "Compilation" also checks for used OpenGL commands and minimizes namespace *gl*. Designer stuff is removed also. You may now save the resulting workspace of minimum size, and use it as stand-alone piece of OpenGL graphics in the future.

Conclusion and Future Directions

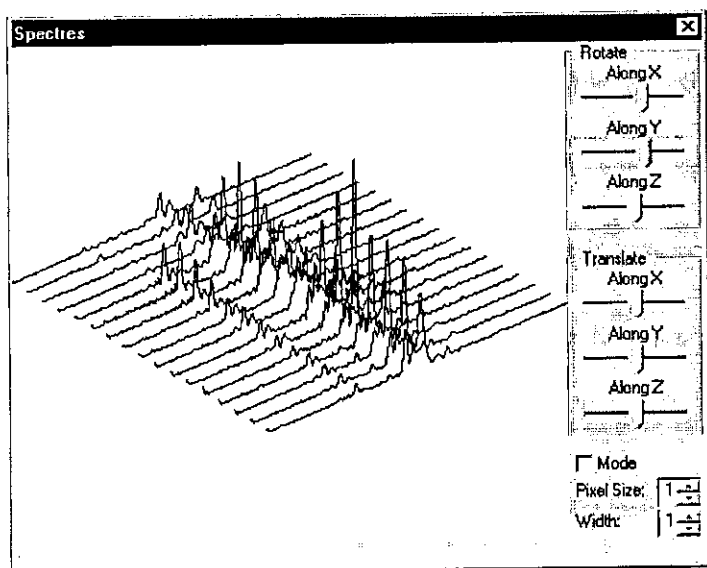
The paper illustrates that with the use of the predefined namespace *gl* and the interface operator *OpenGL* it is easy to create OpenGL graphics from Dyalog APL. The GUI application referred to as *OpenGL Tutor* may help in learning the OpenGL language for APL programmers.

An approach based on the *OpenGL Designer* allows implementation of complex OpenGL functions under GUI control and with minimum knowledge requirements of OpenGL.

The other way to increase the productivity of OpenGL programming in APL is in writing auxiliary functions to be used as macros to create complex images. We do not describe this topic here. It will be covered at the *OpenGL and Dyalog APL Tutorial* planned for the APL98 conference.

Another interesting issue is the incorporation of an APL-to-OpenGL interface into the CPro development system from Causeway Ltd. Adrian Smith created an experimental CPro class on OpenGL to allow use of OpenGL as part of a GUI application designed under CPro. The results are quite impressive. We are planning to pay special attention to this topic at the APL98 Tutorial.

Last, but not least, is what we call application-oriented functions. It is a stand-alone small GUI application using OpenGL. An example is shown below:



It was created for the visualization and analysis of noise spectra in Nuclear Reactor Diagnostics. Why is OpenGL very helpful here? It makes it easy to implement an image rotation and allows one to see the data from different viewpoints. Altogether it is a simple program, but adds a new dimension to visual data analysis.

Acknowledgements

The authors thank everybody who was interested in this work from an early stage and that way pushed the authors to the finish. We would like specially thank Adrian Smith for discussion and testing the use of OpenGL from Causeway Pro, and Alexander Kornilovsky who participated in OpenGL namespace and DEMOGL workspace development.

References

- [1] Chris Burke, *OpenGL Graphics and J*, Vector, Vol.14 No.1, p.58.
- [2] Adrian Smith, *Mostly on OpenGL*, Vector, Vol.14 No.2, p.50.
- [3] Richard Wright and Michael Sweet, *OpenGL Superbible*, Waite Group Press, 1996.

Index to Advertisers

Dyadic Systems Ltd	2
Lingo Allegro	6
Strand	8
Vector Back Numbers	7

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐

UK private membership	£12	☐
Overseas private membership	£14	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£135	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only) £6		☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1998/99 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737-771643 106173.3347@compuserve.com	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	c/o Vector Administration 01439-788385 apl385@compuserve.com	Gill Smith, Brook House, Gilling East YORK YO62 4JJ
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@compuserve.com	138 Burton Stone Lane, YORK YO3 6DF
Education	Dr Ian Clark 01388-526803 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Vacant Post	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (0121-3546550), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths (01653-691745), Bob Hoekstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01853-696760
Fax: 01853-697719
Email: causeway@csi.com
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2832@compuserve.com

Insight Systems Aps
Nordre Strandvej 119C
DK-3150 Hellebæk
Denmark
Tel: +45 49 76 20 20
Fax: +45 49 76 20 30
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337